



UNIVERSIDAD DE BUENOS AIRES

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Codificación de series temporales dinámicas en redes neuronales

Tesis presentada para optar por el título de Doctor de la
Universidad de Buenos Aires en el área Ciencias Físicas

Gonzalo Uribarri

Director de tesis: Dr. Gabriel Mindlin

Director adjunto de tesis: Dr. Marcos Trevisan

Consejero de estudios: Dr. Pablo Mininni

Lugar de trabajo: Laboratorio de Sistemas Dinámicos, Departamento de Física, FCEyN,
UBA

Fecha de defensa: 17/3/2022

Dynamical time series embeddings in neural networks

Abstract

In recent years, data-driven modeling has gained remarkable momentum in all areas of knowledge. The accelerated increase in data availability and computing power, coupled to the constant development of machine learning algorithms, has allowed data-driven models to position themselves as the state of the art in a wide variety of tasks in science and engineering. The main idea behind this framework is to use algorithms capable of automatically discovering the relevant variables of a given system from raw experimental data, even in cases where the data is high-dimensional and unstructured. Despite their remarkable success, the main drawback of this type of models is that they are very difficult to interpret. In this thesis we investigate the use of data-driven models for cases where the measurements come from dynamical systems. Our goal is to develop methodologies that allow us to leverage the potential of current machine learning models to build interpretable dynamical models of the systems under study directly from data. To this end, we study the relevant features extracted from the data by neural network models in different time series related tasks. We show that this type of models has the ability to learn a representation of the data that is equivalent to the flow of the original dynamical system from which the time series is measured.

Resumen

En los últimos años, el modelado *data-driven* (basado en datos) ha tomado un notable impulso en todas las áreas del conocimiento. El acelerado aumento en la disponibilidad de datos y el poder de cómputo, sumado al constante desarrollo de algoritmos de *machine learning* (aprendizaje automático), ha permitido a los modelos *data-driven* posicionarse como el estado del arte en una gran variedad de tareas en ciencia e ingeniería. La idea central en este programa de trabajo es que los algoritmos sean capaces de descubrir por su cuenta cuales son las variables relevantes de un dado sistema a partir de datos experimentales crudos, incluso en casos donde los mismos son de dimensión alta y no están estructurados. A pesar de su notable éxito, la principal desventaja de este tipo de modelos es la dificultad para interpretarlos. En esta tesis se investiga el uso de modelos *data-driven* para casos donde las mediciones provienen de sistemas dinámicos. El objetivo de la misma es desarrollar metodologías que permitan aprovechar el potencial de los modelos actuales de *machine learning* para construir modelos interpretables de la dinámica del problema a partir de mediciones del mismo. Con este fin, estudiamos las características relevantes extraídas de los datos por modelos de redes neuronales en diversas tareas relacionadas a series temporales. Mostramos que este tipo de modelos tiene la capacidad de aprender una representación de los datos que resulta equivalente al flujo del sistema dinámico original del cual proviene la serie.

Agradecimientos

A Gabo, por su sabiduría y su paciencia, ambas tendientes a infinito.

A mis colegas (y amigos) del laboratorio, por generar un ambiente de trabajo inmejorable.

A Marina, por acompañarme todos los días.

A mis padres, por su apoyo y cariño incondicional.

A mi familia y amigos, por enriquecer mi tiempo con su presencia.

A la Universidad Pública y al Estado Argentino, por darme la oportunidad.

Índice general

1. Introducción	1
1.1. Una historia de predicciones	1
1.2. Modelado: Primeros principios vs. <i>Data-driven</i>	3
1.3. El enfoque dinámico a la predicción basada en datos	3
1.4. La era del <i>Machine Learning</i>	5
1.5. <i>Machine Learning</i> y modelos interpretables	6
1.6. Objetivo de la tesis	8
1.7. Estructura de la tesis	9
2. Sistemas dinámicos, caos y análisis no-lineal de series temporales	10
2.1. Sistemas Dinámicos	11
2.1.1. Definiciones y consideraciones generales	11
2.1.2. Retrato de fase	12
2.1.3. Comportamiento asintótico de las soluciones	13
2.1.4. Cambio de coordenadas	15
2.2. Caos y Atractores Extraños	15
2.2.1. Comportamiento caótico	15
2.2.2. Exponentes de Lyapunov	16
2.2.3. Atractor Caótico	17
2.2.4. Mecanismos del caos	19
2.2.5. Organización topológica y <i>linking numbers</i>	21
2.3. Análisis no-lineal de series temporales	23
2.3.1. Reconstrucción del flujo	24
2.3.2. Teorema de Takens	24
2.3.3. <i>Embeddings</i> en la práctica	25

3. Redes Neuronales	28
3.1. Aprendizaje supervisado	28
3.1.1. El paradigma de aprendizaje supervisado	28
3.1.2. El compromiso sesgo-varianza	30
3.1.3. El teorema de <i>No Free Lunch</i>	31
3.2. Redes Neuronales y <i>Deep Learning</i>	32
3.2.1. La neurona artificial	32
3.2.2. Red neuronal <i>feedforward</i>	33
3.2.3. Entrenamiento de una red	34
3.2.4. ¿Por qué redes neuronales profundas?	36
3.3. <i>Deep Autoencoders</i>	37
3.3.1. Definición	37
3.3.2. Aplicaciones a dinámica	38
3.4. Redes neuronales recurrentes	39
3.4.1. Definición	39
3.4.2. <i>Long Short Term Memory</i>	40
3.4.3. Aplicaciones a dinámica	42
4. Codificación en Autoencoders	43
4.1. Introducción	43
4.2. Metodología	44
4.2.1. Generación del Dataset	45
4.2.2. Implementación de la Red Neuronal	45
4.2.3. Cálculo del <i>linking number</i>	47
4.3. Resultados	48
4.3.1. Estimación de la Dimensionalidad	48
4.3.2. Evaluación de la topología	49
4.3.3. Evaluación de intersecciones	52
4.3.4. Evaluación del Exponente de Lyapunov	52
4.3.5. Robustez Frente a Ruido	53
4.4. Discusiones	54
5. Codificación en Redes Neuronales Recurrentes	58
5.1. Introducción	58

5.2. Métodos	59
5.2.1. Generación del Dataset	59
5.2.2. Implementación de la Red Neuronal	60
5.2.3. Cómputo de Linking Numbers	60
5.3. Resultados	62
5.3.1. Predicción del próximo segmento	62
5.3.2. Predicción de N pasos adelante	65
5.4. Discusiones	66
6. Conclusiones	68

Capítulo 1

Introducción

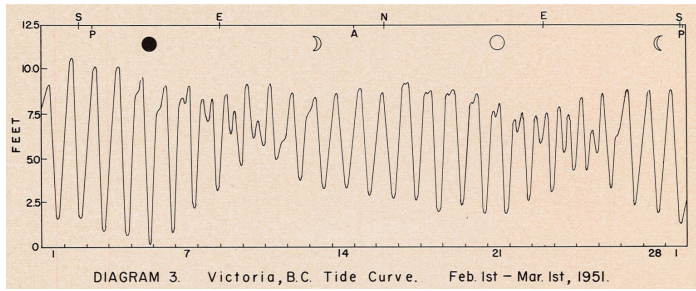
“ *Lo desconocido es una abstracción; lo conocido, un desierto; pero lo conocido a medias, lo vislumbrado, es el lugar perfecto para hacer ondular deseo y alucinación.* ”

Juan José Saer, *El entenado*, 1983

1.1. Una historia de predicciones

La necesidad de predecir fenómenos naturales ha sido el principal motor de los avances científicos y tecnológicos a lo largo de la historia. Y, sin duda, un fenómeno natural que toda civilización navegante anheló poder pronosticar es el de las mareas. En la figura 1.1(a) se presenta un típico gráfico de altura de la marea en función del tiempo tomado de [1], esta clase de registros son realizados en los puertos desde hace siglos [1, 2]. Conocer cuál será el nivel del mar en una dada costa al momento de la llegada de un barco es información que puede significar la diferencia entre un encallamiento y un amarre exitoso al puerto. Es por esto que predecir los valores en la serie temporal de la marea resulta esencial para todo tipo de actividades navales, tanto comerciales como bélicas.

A pesar de su ubicuidad y relevancia, los patrones de ascenso y descenso de las aguas costales se mantuvieron elusivos durante miles de años. Como todo problema importante que se resiste a ser resuelto, atrajo la atención de grandes intelectuales a lo largo de la historia. Desde Eratóstenes y Posidonio en la antigua Grecia, pasando por Aristóteles, Galileo Galilei y Descartes, muchos fueron los que intentaron explicar el fenómeno [3]. Incluso el propio Newton en su obra Principia abordó el problema de las mareas [4]. Para hacerlo tuvo que asumir un planeta simplificado, cubierto enteramente por un océano uniforme [1]. Sin embargo, a pesar



(a)



(b)

Figura 1.1: (a) Registro de la marea en función de tiempo para una costa canadiense durante en año 1951. Imagen adaptada de [1]. (b) Máquina de predicción de mareas de 10 componentes de 1872, concebida por Sir William Thomson (Lord Kelvin). Fotografía tomada por William M. Connolley en el Museo de Ciencias de South Kensington, Londres.

de los esfuerzos, ninguna de estas descripciones resultaba lo suficientemente exacta como para realizar predicciones útiles a los navegantes.

El primer gran avance teórico en el tema fue realizado por Pierre-Simon Laplace en 1775. Basado en el trabajo de Newton, Laplace derivó un sistema de tres ecuaciones diferenciales en derivadas parciales que explica correctamente cómo la interacción entre el cuerpo de agua y los forzantes naturales que recibe dan lugar a corrientes horizontales y al desplazamiento vertical de la superficie oceánica [1]. Esta formulación del fenómeno, conocida como su teoría dinámica de las mareas, resultó sumamente exitosa. Tanto es así que continúa siendo la base del cálculo de las mareas hoy en día. Pero a pesar de su éxito, no fue posible utilizar esta teoría para realizar predicciones del nivel del mar en las costas hasta siglos más tarde, ya que resultaba técnicamente imposible relevar condiciones de contorno realistas y resolver el sistema de ecuaciones [2].

Fue recién William Thomson, también conocido como Lord Kelvin, quien un siglo más tarde realizó por primera vez pronósticos útiles sobre las mareas. Conociendo el trabajo realizado por Laplace y las dificultades para resolver sus ecuaciones, Kelvin abandonó la idea de resolver de forma analítica el problema de la marea para todo el planeta y decidió enfocarse en pronosticar la marea en un dado punto de la costa a partir de mediciones experimentales [2, 5]. La idea de Thomson fue aplicar el recientemente desarrollado análisis armónico de Joseph Fourier a los registros temporales de la altura del agua para un puerto dado. Él propuso que la información acerca de la forma particular de esa costa y la profundidad del mar en esa región quedaría codificada en las amplitudes y las fases obtenidas a partir de los datos. Además, gracias al

trabajo de Laplace, sabía que la serie temporal de la marea sólo podía contener las frecuencias presentes en los forzantes, es decir, que solo era necesario descomponer a la serie en frecuencias coincidentes con las de ciertos fenómenos astronómicos. Fue así que, con la ayuda de Edward Roberts y Alexander Lége, en 1872 construyó la primera máquina de predicción de mareas cuya imagen puede observarse en la figura 1 [5]. Este tipo de computadoras analógicas predictoras de mareas se utilizaron durante casi un siglo, llegando incluso a jugar un rol decisivo en el planeamiento del desembarco de Normandía durante la segunda guerra mundial [6].

1.2. Modelado: Primeros principios vs. *Data-driven*

Esta breve historia sobre la predicción de las mareas es sólo un ejemplo de la relevancia que el análisis de series temporales tuvo a lo largo de la historia. Pronosticar el estado futuro de una secuencia de valores que evolucionan en el tiempo es una tarea esencial en innumerables problemas de aplicación práctica en campos tan diversos como el clima, la biología, el tráfico y las finanzas [7–11]. En el ejemplo podemos observar que, frente a un fenómeno dinámico, existen dos enfoques posibles para construir un modelo matemático.

Uno es el enfoque utilizado por Laplace, el cual consiste en derivar nuestro modelo a partir de conocimiento previo del sistema en estudio: cómo evolucionan sus elementos constituyentes, cómo interactúan entre sí, qué cantidades del problema se conservan, qué simetrías se respetan, etc. Utilizando esta información y realizando aproximaciones adecuadas podemos construir modelos matemáticos que describen la evolución temporal del sistema. Esto es lo que denominamos un modelado de primeros principios. Muchas veces, particularmente en física e ingeniería, estos modelos matemáticos de evolución temporal se expresan en forma de ecuaciones diferenciales.

El otro enfoque es el utilizado por Kelvin y consiste en construir un modelo a partir de los datos experimentales. En este caso buscamos un modelo matemático, no necesariamente interpretable, cuyas predicciones se ajusten a las observaciones. Este es el enfoque que debemos tomar cuando nos es imposible plantear un modelo de primeros principios, ya sea por falta de información del fenómeno o por una imposibilidad matemática. En general estos modelos, llamados *data-driven*, son más difíciles de interpretar y su rango de validez suele ser restringido al dominio de los datos disponibles. Por ejemplo, sería muy difícil decir algo sobre la forma de la costa a partir de los valores de las amplitudes y las fases de la descomposición de Fourier propuesta por Kelvin.

1.3. El enfoque dinámico a la predicción basada en datos

La temática de la presente tesis es el análisis basado en datos de series temporales. En particular nos enfocamos en el estudio *data-driven* de series temporales dinámicas, es decir, de

series cuya evolución está determinada por un sistema de ecuaciones diferenciales. El estudio *data-driven* de este tipo de series se desarrolló fuertemente durante los años 80s y 90s, años en los cuales el caos determinista se volvió un área prolifera de estudio. En esta época, los dinamicistas desarrollaron un conjunto de ideas y técnicas para la caracterización y el pronóstico de series temporales dinámicas [12–19]. A este conjunto de técnicas se las conoce como análisis no-lineal de series de tiempo [20, 21].

En contraposición a los métodos tradicionales, donde la serie se analiza en el dominio temporal o espectral, la característica central del análisis no-lineal es la representación de la serie de tiempo en un espacio de fases. La clave de este enfoque es que contamos con un mecanismo por el cual es posible reconstruir un flujo (una trayectoria en un espacio de fases) que resulte topológicamente equivalente al flujo original del cual provienen los datos [20, 21]. Es decir que, partiendo de la serie temporal para una única variable del problema, podemos obtener un *proxy* de la evolución de todas las variables dinámicas involucradas. A este proceso de reconstrucción del espacio de fases se lo llama *embedding* y es posible gracias al teorema de Takens, el cual nos provee un mecanismo para hacerlo y nos garantiza, bajo condiciones adecuadas, la equivalencia topológica entre los flujos [22].

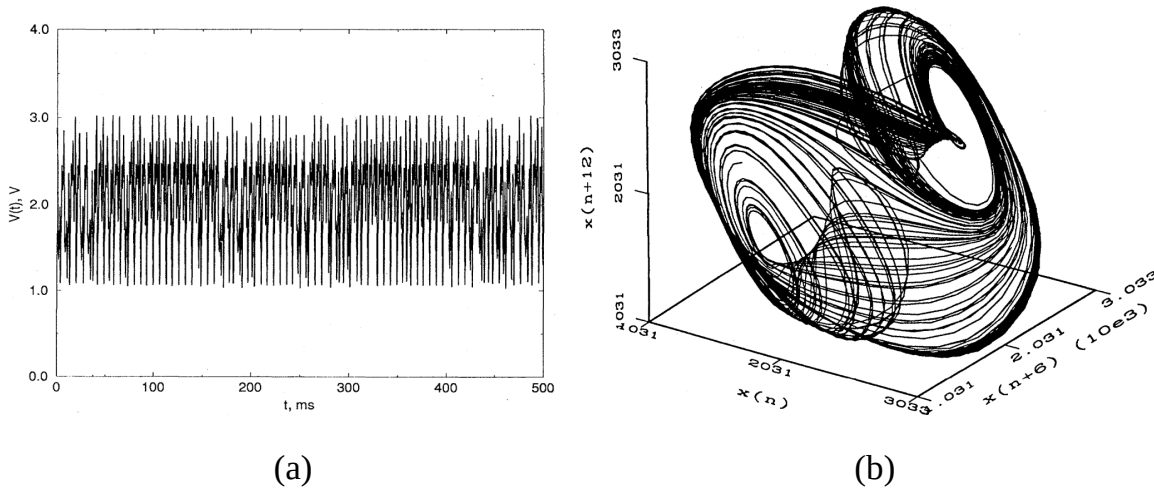


Figura 1.2: (a) Serie temporal de valores de la tensión en un circuito electrónico en régimen caótico. (b) Embedding de la señal de tensión realizado mediante la metodología de retrasos temporales propuesta por Takens. Ambas imágenes son adaptadas de [23], realizadas a partir de datos del experimento presentado en [23].

Este tipo de análisis resultó muy exitoso en el estudio de series temporales experimentales, particularmente para sistemas en regímenes caóticos o fuertemente no lineales. En la figura 1.2 se presentan mediciones experimentales de un circuito electrónico en régimen caótico, tomadas de un experimento publicado en 1991 [23]. En el panel (a) se muestra la serie temporal de la tensión, la cual posee la irregularidad característica de los procesos caóticos. Mientras que en el panel (b) se presenta el atractor reconstruido mediante el mecanismo de *embedding*. En esta

representación es fácil observar que, a pesar de la aparente aleatoriedad de la serie temporal, el sistema recorre una estructura ordenada en el espacio de fases. La reconstrucción del flujo no sólo nos permite realizar predicciones de valores futuros, sino que también permite estudiar la geometría y topología del flujo, lo cual nos brinda información física relevante del sistema. En algunos casos incluso es posible ajustar un sistema de ecuaciones diferenciales que modelan la evolución de los datos en el espacio de fases [24–27].

1.4. La era del *Machine Learning*

Hoy en día, el modelado basado en datos está dominado por algoritmos de *Machine Learning* (Aprendizaje Automático), particularmente por redes neuronales [28, 29]. Estos modelos de tipo “caja negra” se utilizan como algoritmos de predicción y clasificación en todo tipo de series temporales, sin diferenciar su naturaleza [30–38]. El éxito y la popularidad de estos modelos se debe principalmente a dos factores. Uno es la creciente potencia de cálculo disponible, la cual permite utilizar modelos de gran escala que se entrenan sobre una enorme cantidad de datos. El otro es el vertiginoso desarrollo de nuevos algoritmos, cuyas implementaciones se encuentran disponibles de manera abierta en librerías y repositorios web.

Una de las ventajas de las redes neuronales por sobre otros tipos de algoritmos es su capacidad para procesar directamente datos complejos y no estructurados, como pueden ser imágenes, texto o audio [28, 29]. Esto significa que las redes son capaces de aprender a identificar cuáles son las *features* o características relevantes de los datos de forma automática, sin necesidad de un preprocesamiento supervisado por un humano. En el contexto de dinámica, distintas arquitecturas de redes neuronales han sido utilizadas para modelar exitosamente toda clase de sistemas, mostrando incluso buenos resultados en la predicción de sistemas caóticos y en la predicción de películas de fluidos turbulentos, dos problemas intrínsecamente difíciles [39–45].

A pesar de su indudable éxito y utilidad, la gran desventaja de esta clase de modelos *data-driven* es su difícil interpretación. Las redes neuronales son funciones matemáticas no lineales que suelen tener miles o millones de parámetros. Al igual que en el caso del análisis espectral de Kelvin sobre los datos de las mareas, es muy difícil dilucidar información del problema subyacente a partir de los parámetros del modelo. Existen entonces tareas para las cuales podemos entrenar modelos que las resuelven de manera exitosa, pero desconocemos el mecanismo de funcionamiento. Un caso paradigmático sucede en el área de neurociencias, en donde, por ejemplo, existen modelos capaces de reconstruir la imagen que está observando un paciente a partir de mediciones de resonancia magnética funcional (*fMRI*) de su cerebro, pero sin embargo estamos lejos de poder plantear y entender un modelo de primeros principios del sistema [46–49]. En la figura 1.3 se presenta a modo de ejemplo el esquema de uno de estos experimentos y las reconstrucciones de las imágenes que logra a partir de los datos de *fMRI* [46].

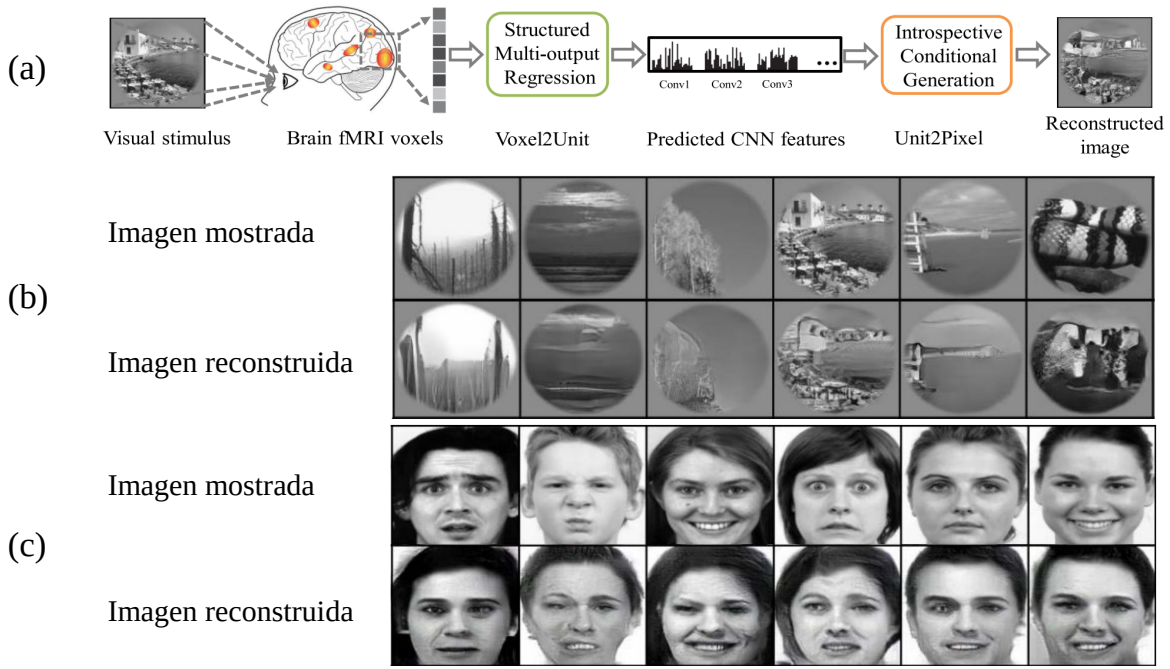


Figura 1.3: (a) Esquema experimental de un trabajo donde se utilizan redes neuronales profundas para reconstruir las imágenes que está viendo un sujeto a partir de la medición de su cerebro con fMRI. (b) Comparación entre imágenes de paisajes naturales mostradas al sujeto y las respectivas reconstrucciones realizadas por el modelo. (c) Idem ítem anterior pero con imágenes de caras. Todas las figuras fueron adaptadas de [46]

Es por este motivo que existen grandes esfuerzos en la comunidad científica de inteligencia artificial por desarrollar metodologías que permitan analizar e interpretar cómo aprenden las redes neuronales, así como también por desarrollar modelos que brinden como salida no solo el resultado deseado, sino también el motivo por el cual llegaron al mismo [50–55].

1.5. *Machine Learning* y modelos interpretables

Cuando trabajamos con datos provenientes de sistemas dinámicos, el objetivo último es poder encontrar cuáles son las variables relevantes del problema y cuál es la ecuación diferencial que gobierna su evolución. Un programa de investigación sumamente activo en los últimos años es aquel que busca utilizar las ventajas de los algoritmos modernos de *Machine Learning* para hacerlo [27, 56–62]. En contraposición a la idea de utilizar modelos de *Machine Learning* como una caja negra que predice el siguiente estado del sistema, el objetivo es utilizar estos algoritmos para escribir modelos matemáticos interpretables. En este esquema de trabajo se busca partir de los datos y terminar en un sistema de ecuaciones diferenciales parsimonioso (con pocos términos no nulos), como aquellos que derivamos cuando planteamos el problema desde primeros principios.

Para escribir una ecuación diferencial a partir de datos crudos, un modelo de *Machine Learning* necesita lograr dos objetivos. Primero, identificar cuáles son las variables dinámicas relevantes en el problema y, segundo, ajustar una ecuación diferencial a la evolución temporal de esas variables.

La dificultad para identificar las variables dinámicas dependerá de qué tan explícitas se encuentren las mismas en los datos de entrada. En el caso más sencillo posible, los datos de entrada a nuestro algoritmo podrían ser directamente los valores de las variables dinámicas. En el caso más general, uno podría tener una relación altamente no lineal y no trivial entre los datos de entrada y las variables dinámicas relevantes. A modo de ejemplo, podemos imaginar un caso en el cual contamos con la filmación de un péndulo oscilando. Los datos de entrada son los valores de intensidad de todos los píxeles en cada cuadro del video, mientras que las variables dinámicas relevantes son la posición angular del péndulo y su derivada. En la figura 1.4 se presenta un ejemplo de este caso, tomado de [60]. Para convertir los datos de alta dimensión en la entrada a una representación de baja dimensión que conserve sólo las variables relevantes es necesario utilizar algoritmos de *Machine Learning* de reducción dimensional. Para esta tarea, las redes neuronales profundas demuestran ser el algoritmo que mejor funciona, ya que son capaces de captar relaciones complejas en los datos de entrada y obtener resultados significativamente mejores que los obtenidos con algoritmos lineales como *Proper Orthogonal Decomposition* (POD) [63, 64] o *Principal Component Analysis* (PCA). [65, 66].

Luego, para encontrar una ecuación diferencial que explique la evolución de los datos, se computa la derivada numérica de las variables encontradas y se utiliza algún tipo de algoritmo de regresión para ajustar los términos presentes en la ecuación diferencial. Normalmente esta regresión se encuentra regularizada con el fin de minimizar la cantidad de términos no nulos en las ecuaciones y generar un sistema parsimonioso [27]. Algunos de los algoritmos propuestos para hacerlo son *Orthogonal Least Squares Method* [56], *Compressed Sensing* [58], *Entropic Regression* [62] y *SINDy* [59, 67, 68], este último siendo muy utilizado en la actualidad.

A diferencia de, por ejemplo, una red neuronal, un modelo *data-driven* expresado en forma de una ecuación diferencial nos permite utilizar una gran cantidad de herramientas de análisis matemático para estudiarlo. En un caso ideal, este programa de trabajo nos permitiría aprender leyes de interacción y simetrías del sistema estudiado partiendo únicamente del análisis de datos. Pero en la práctica, todavía existen numerosos desafíos para lograrlo. Un problema recurrente al trabajar con datos experimentales, es que la cantidad de datos accesible no es lo suficientemente grande como para realizar una buena estimación de las ecuaciones. Además, en presencia de ruido o con un intervalo de medición comparable a los tiempos característicos del sistema, es difícil hacer buenas estimaciones numéricas de las derivadas. Otra dificultad, y tal vez la más relevante, es la de interpretar qué significan las variables relevantes encontradas por la red neuronal en términos del sistema estudiado. Esa relación entre los datos de entrada y las variables relevantes queda codificada en los pesos de la red neuronal que realiza la reducción

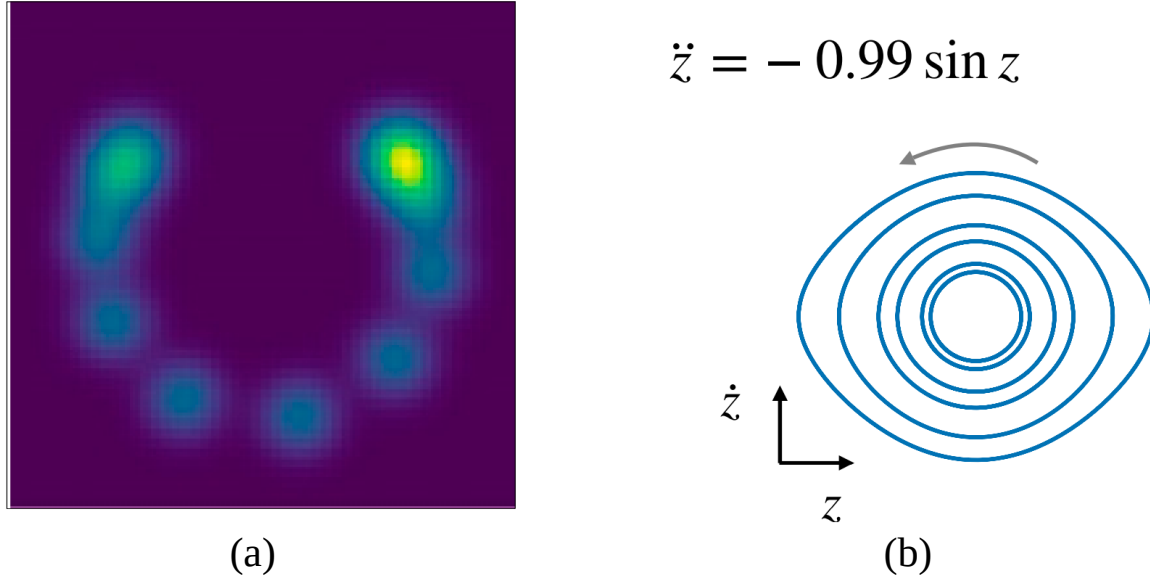


Figura 1.4: (a) Película de un péndulo oscilando generada mediante una simulación del sistema. Se presenta en la imagen la superposición de varios cuadros. Cada cuadro es un dato de entrada en el algoritmo. (b) Gráfico de la evolución de la variable relevante, z , y su derivada. Esta variable fue obtenida mediante la utilización de un *Deep Autoencoder* (red neuronal profunda) sobre los datos de entrada. Luego, la derivada se computó numéricamente a partir de z . Se muestra también la ecuación diferencial ajustada por el algoritmo *SINDy*. Las figuras son adaptadas de [60]

dimensional, pero, nuevamente, resulta muy difícil de interpretar.

1.6. Objetivo de la tesis

Gracias al desarrollo tecnológico en adquisición y almacenamiento, el volumen de datos experimentales en todas las áreas del conocimiento ha crecido notablemente en los últimos años. Si bien esto es algo positivo, también plantea un desafío al momento de extraer información relevante de los mismos. A medida que la cantidad de datos crece, la estrategia de realizar una ingeniería de *features* de forma manual o supervisada se vuelve cada vez mas costosa. Es en este sentido que los algoritmos de *Machine Learning* se vuelven una herramientas fundamental para el análisis y el modelado de datos. Pero si bien estos algoritmos tienen el potencial de extraer *features* de forma automática a partir de datos en crudo, no resulta obvio cuales son variables relevantes en cada problema, ya que esto dependerá de la tarea objetivo y el dominio al cual pertenezcan los datos estudiados.

Este es el marco en el cual se inscribe la presente tesis doctoral. El objetivo de la misma es analizar algoritmos actuales de *Machine Learning* en tareas relacionadas a series temporales

dinámicas, evaluando su manera de operar mediante metodologías provenientes de teoría de sistemas dinámicos. En particular, nos preguntamos si las redes neuronales son capaces de aprender a reconstruir un espacio de fases para series temporales dinámicas así como se hace mediante la metodología propuesta por Takens. Es decir, buscamos evaluar si las redes aprenden a codificar la información de series temporales dinámicas de manera de generar un *embedding* de la misma en su espacio de representación interna.

Para contestar esta pregunta analizamos el flujo reconstruido de series temporales provenientes de sistemas caóticos. Aprovechamos su particular estructura en el espacio de fases para determinar si la codificación aprendida por las redes resulta equivalente a un *embedding* de la serie. Realizamos este procedimiento en dos tipos de redes neuronales distintas. Primero lo hicimos para autoencoders profundos (*deep autoencoders*), un tipo de red neuronal se utiliza como algoritmo de reducción dimensional. Luego, para redes neuronales recurrentes (*RNNs*) entrenadas en una tarea de predicción.

La pregunta planteada en esta tesis tiene relevancia en dos aspectos centrales dentro del contexto actual. Por un lado se suma a los esfuerzos por entender mejor cómo funcionan las redes neuronales, en este caso *deep autoencoders* y *RNNs* aplicadas a tareas en series temporales. Por otro, contribuye al programa de utilizar *Machine Learning* para encontrar modelos *data-driven* en forma de ecuaciones diferenciales, ya que en este programa resulta esencial saber si la codificación de los datos realizada por redes neuronales tiene las características de un flujo dinámico.

1.7. Estructura de la tesis

La estructura de esta tesis doctoral es la siguiente. En el segundo capítulo se presenta un resumen de los conceptos de teoría de sistemas dinámicos que se usarán a lo largo de la tesis. En particular, nos centramos en algunas características de los flujos caóticos y en el Teorema de Takens. En el tercer capítulo se introducen los modelos de redes neuronales en el marco del *Machine Learning*. Se describen brevemente los mecanismos elementales utilizados para su entrenamiento y evaluación. El cuarto capítulo estudia la capacidad de los *autoencoders*, un tipo de red neuronal utilizadas como algoritmo de reducción dimensional, para realizar *embeddings* de series temporales dinámicas [69]. En quinto capítulo se entrenan redes neuronales recurrentes como algoritmos predictivos de series temporales caóticas y se estudia la codificación del flujo en su estado interno [70]. Por último, en el capítulo Conclusiones, se discuten los resultados obtenidos en el contexto actual del campo.

Capítulo 2

Sistemas dinámicos, caos y análisis no-lineal de series temporales

“ *You don't see something until you have the right metaphor to let you perceive it.* ”

James Gleick, *Chaos: Making a New Science*, 1987

En este capítulo se presentarán conceptos de teoría de sistemas dinámicos y análisis no-lineal de series temporales que son relevantes para el trabajo desarrollado en capítulos posteriores. No pretende ser una introducción ninguno de estas dos temáticas, dado que se omitirán muchos aspectos, sino mas bien un compendio de definiciones, resultados y comentarios que serán utilidad para comprender los resultados de la tesis. En este capitulo se siguieron, principalmente, los libros: Wiggins (2003) [71], Strogatz (2018) [72] y Gilmore (2003) [73].

Quiero dedicar este capítulo al físico italiano Giorgio Parisi quien, al momento de escribir esta introducción, fue galardonado con el premio nobel de física. Personalmente, celebro con entusiasmo el reconocimiento, quizás un poco tardío, por parte de la Real Academia Sueca de Ciencias a la teoría de sistemas complejos, teoría imprescindible para entender y modelar la naturaleza.

2.1. Sistemas Dinámicos

2.1.1. Definiciones y consideraciones generales

Cuando nos referamos a sistemas dinámicos a lo largo de esta tesis, nos referimos a sistemas cuyas variables evolucionen según una ecuación diferencial de la siguiente forma:

$$\begin{aligned}\frac{dx_1}{dt} &= f_1(x_1, \dots, x_n) \\ &\vdots \\ \frac{dx_n}{dt} &= f_n(x_1, \dots, x_n),\end{aligned}\tag{2.1.1}$$

es decir que vamos a considerar sistemas autónomos (que no dependen explícitamente de la variable temporal) y de primer orden. Recordemos que cualquier sistema con derivadas de orden superior siempre puede reescribirse como uno de primer orden mediante la inclusión de nuevas variables.

Dado un punto de partida para las variables, tendremos definido un problema de condiciones iniciales. El mismo se puede expresar de manera compacto como:

$$\begin{aligned}\frac{d\bar{x}}{dt} &= f(\bar{x}) \\ \bar{x}(t_0) &= \bar{x}_0\end{aligned}\tag{2.1.2}$$

con $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ y $\bar{x} \in \mathbb{R}^n$. En esta expresión $\bar{x}_0 \in \mathbb{R}^n$ refiere a la condición inicial del problema. Asumiremos además en nuestra definición de sistema dinámico que el campo vectorial f es C^r con $r \geq 1$ en un abierto $U \subset \mathbb{R}^n$ alrededor de todas las condiciones iniciales consideradas.

Bajo estas hipótesis de trabajo, el teorema de existencia y unicidad nos asegura que para cada condición inicial $\bar{x}_0$ existirá una solución única $\bar{x}(t)$ con $t \in [t_0 - \alpha, t_0 + \alpha]$ y $\alpha > 0$. Esta solución será única en el sentido que cualquier otra solución que pase por el mismo punto debe ser la misma solución. Además la trayectoria $\bar{x}(t)$ será una función C^r de t , $\bar{x}_0$ y t_0 . La prueba de este teorema puede encontrarse en [74].

Cuando el campo vectorial f está compuesto por funciones no-lineales, es en general muy difícil resolver analíticamente el problema de condiciones iniciales. Incluso en los escasos casos para los cuales contamos con mecanismos para encontrar las soluciones, las expresiones resultantes son complicadas y no nos brindan una intuición clara del sistema dinámico. Es por este motivo que la teoría de sistemas dinámicos se enfoca en estudiar las características cualitativas de las soluciones de un sistema, como su comportamiento asintótico o su dependencia con los parámetros y condiciones iniciales, en lugar de buscar su forma analítica.

2.1.2. Retrato de fase

Una forma conveniente de analizar el comportamiento de un sistema dinámico, como el definido en 2.1.1, es mediante el estudio de su retrato de fase. El mismo se construye graficando trayectorias, soluciones $\bar{x}(t)$, en el espacio definido por sus variables dinámicas, el espacio de fase. En la figura 2.1 se muestra el diagrama de una trayectoria en el espacio de fase para un sistema de dimensión $n = 2$, noten que el campo vector f será siempre tangente a la curva. Dado un sistema particular, el objetivo es graficar sobre el espacio de fase trayectorias cualitativamente distintas, que nos den una idea de todos los posibles comportamientos de las soluciones. Llamamos flujo a las trayectorias formadas por la evolución de las soluciones y usaremos la notación $\phi(t, x)$ para referirnos a él. El flujo de un sistema dinámico cumple la propiedad $\phi(t + s, \bar{x}_0) = \phi(t, \phi(s, \bar{x}_0))$.

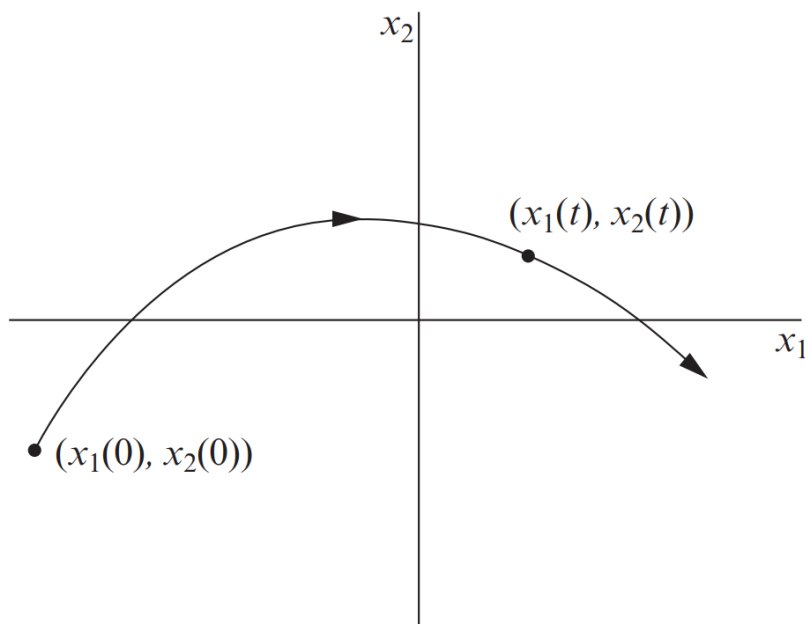


Figura 2.1: Trayectoria de una solución $\bar{x}(t)$ en el espacio de fase para un problema de dimensión $n = 2$ donde $\bar{x} = (x_1, x_2)$. La imagen es adaptada de [72].

Un corolario muy importante del teorema de existencia y unicidad es que las trayectorias en el espacio de fases nunca se cruzan, algo que se deduce automáticamente a partir de que cada punto en este espacio tiene un único futuro posible. Esto impone una fuerte restricción topológica sobre el comportamiento de las soluciones.

La idea de estudiar sistemas dinámicos mediante su representación en un espacio de fases fue desarrollada por el matemático francés Henri Poincaré a finales de 1800. Esta representación permite contestar preguntas relevantes sobre las soluciones, como su estabilidad, mediante un análisis geométrico y topológico de las trayectorias. Su desarrollo significó un gran avance en la comprensión de la dinámica y permitió a Poincaré arrojar luz sobre el centenario problema de

los tres cuerpos [75].

2.1.3. Comportamiento asintótico de las soluciones

Dado un sistema dinámico, una de las preguntas más importantes a responder es cuál será el comportamiento asintótico de sus soluciones. Es decir, qué sucede con una dada solución $x(t)$ cuando $t \rightarrow \infty$. Vamos a definir a $p \in \mathbb{R}^n$ como un punto límite del sistema si existe una secuencia t_i , $t_i \rightarrow \infty$, tal que se cumpla que $\phi(t_i, \bar{x}_0) \rightarrow p$. Al conjunto de todos los puntos límites lo llamaremos un conjunto límite w .

Los conjuntos límite son elementos muy relevantes a la hora de estudiar el comportamiento de un sistema dinámico. Dos ejemplos paradigmáticos de conjuntos límites son los puntos fijos y los ciclos límites. Llamamos a $\bar{x}^* \in \mathbb{R}^n$ un punto fijo del sistema si cumple la condición $f(\bar{x}^*) = 0$, es decir que si la condición inicial del sistema es $\bar{x}^*$, se quedará ahí por siempre. Por otro lado, un ciclo límite es una trayectoria cerrada en el espacio de fases, es decir una solución del sistema $\bar{x}(t)$ para la cuál existe un $P > 0$ tal que $\bar{x}(t + P) = \bar{x}(t)$. En la figura 2.2 se presenta un retrato de fases en de un sistema bidimensional en el cuál se grafican, además de algunas trayectorias, tres puntos fijos y un ciclo límite.

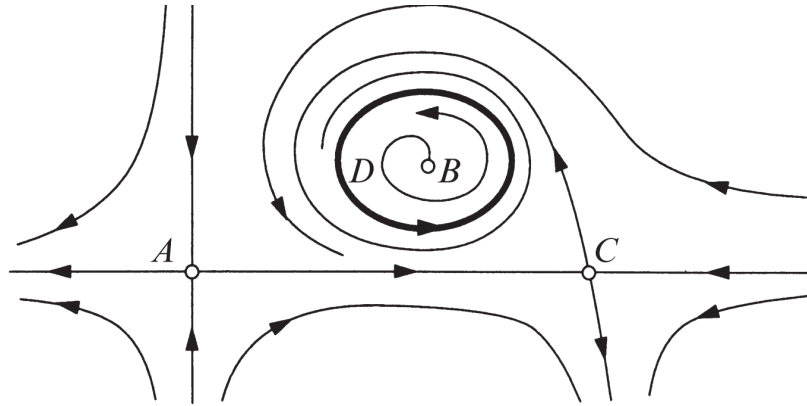


Figura 2.2: Ejemplo de un retrato de fases para un problema bidimensional. Los puntos A, B y C son puntos fijos inestables. La trayectoria D es un ciclo límite estable. La imagen es adaptada de [72].

Una característica importante de estos conjuntos límite es su estabilidad. Decimos que un punto fijo y o un ciclo límite es estable cuando estos son el conjunto límite de todos los puntos en algún abierto que los contenga. Es decir que dado un punto x_0 infinitesimalmente cerca al conjunto límite, $\phi(t, \bar{x}_0)$ tenderá al conjunto cuando $t \rightarrow \infty$. En el ejemplo dado en la figura 2.2, el ciclo límite es estable, ya que las trayectorias que lo rodean se aproximan a él. En cambio, los 3 puntos fijos son inestables, ya que no atraen a todas las trayectorias en un abierto alrededor de ellos.

De manera mas general, podemos investigar propiedades de todos los tipos de conjuntos

límites. Sea M un compacto invariante en el espacio de fase (el término invariante se refiere a que todas las soluciones que inician dentro del mismo se mantienen dentro del mismo), entonces todos los conjuntos límite w dentro de M cumplirán la siguiente lista de propiedades:

- $w \neq \emptyset$.
- w es cerrado.
- w es invariante.
- w es conexo.

Una prueba de estas propiedades puede encontrarse en [71]. Nótese que tanto los puntos fijos como los ciclos límites cumplen con esta lista de propiedades.

Un resultado muy notable de la teoría de sistemas dinámicos para sistemas bidimensionales es el teorema de Poincaré-Bendixón. El mismo establece que, dado un conjunto invariante M en un espacio de fase bidimensional que contiene una cantidad de puntos fijos finitos dentro, entonces un conjunto límite w dentro de M cumple una de estas 3 opciones:

- w es un punto fijo.
- w es Un ciclo límite.
- w es un grupo de puntos fijos unidos por trayectorias.

Una postulación y demostración detallada del teorema puede encontrarse en [71].

Esto quiere decir que la restricción topológica impuesta por el teorema de unicidad a flujos bidimensionales le deja pocas opciones al comportamiento asintótico de las soluciones del sistema dinámico. A grandes rasgos se puede decir que, si las trayectorias no se van al infinito, sólo pueden terminar en un punto fijo o en un ciclo límite. Esto deja de ser válido para sistemas de dimensión 3 o superior, en 2.2.3 se presentará otro tipo posibles de conjunto límite para esta clase de sistemas.

Por último en esta sección, vamos a definir el concepto de atractor. Llamaremos atractor a un cerrado A contenido en un espacio de fases de $\mathbb{R}^n$ que cumpla las siguientes condiciones:

- A es un set invariante.
- A atrae a todas las condiciones iniciales en un abierto U que lo contiene. Es decir que, siendo $A \subset U$, se tiene que $\forall p \in U, \phi(t, p) \rightarrow A$ cuando $t \rightarrow \infty$.
- A es minimal, no hay un subset de A que cumpla las dos condiciones anteriores.

Cabe destacar que un atractor es también un conjunto límite, y que tanto los puntos fijos estables como los ciclos límites estables son atractores.

2.1.4. Cambio de coordenadas

Un sistema físico representado por un sistema dinámico debe ser independiente del sistema de coordenadas que elijamos para modelarlo. Esto significa que debemos ser libres de elegir diferentes sistemas de coordenadas sin riesgo de alterar nuestra descripción del problema. Una de las ventajas de esto es que podemos elegir el sistema que simplifica en la mayor medida posible las dificultades para interpretarlo o resolverlo. Los sistemas de coordenadas que son diferentes pero equivalentes para un mismo conjunto de ecuaciones diferenciales ordinarias están relacionadas por un difeomorfismo.

Sean M y N dos variedades, un difeomorfismo f es un mapa $f : M \rightarrow N$ biyectivo (inyectivo y sobreyectivo), diferenciable (suave) y con inversa $f^{-1} : N \rightarrow M$ también diferenciable [76, 77].

Podemos entonces afirmar que dos sistemas dinámicos n -dimensionales relacionados por un difeomorfismo son equivalentes, cualquier propiedad presente en uno tiene su contraparte en el otro. Por ejemplo, es sencillo corroborar que todas las transformaciones lineales no singulares:

$$x_i \rightarrow x'_i = \sum_j A_i^j x_j \quad (2.1.3)$$

son difeomorfismos.

2.2. Caos y Atractores Extraños

El caos es un comportamiento complejo determinista que puede emerger en sistemas dinámicos muy simples, como un mapa unidimensional o un sistema de tres ecuaciones diferenciales de primer orden autónomas. A pesar de esto, el estudio de los sistemas caóticos es sorprendentemente moderno. Si bien el trabajo realizado por Poincaré en el problema de los tres cuerpos contenía varios conceptos que hoy asociamos a la teoría del caos, recién en 1962 Edward Lorenz sienta las bases del estudio moderno de los sistemas caóticos en su trabajo seminal *Deterministic Nonperiodic Flow* [78]. Pero este artículo pasó prácticamente desapercibido hasta la década de 1970, en la cual trabajos como los de Ruelle [79], Takens [79], Yorke [80], May [81] y Feigenbaum [82] tomaron notoriedad y comenzó una época prolifera en el área.

2.2.1. Comportamiento caótico

Hay varias definiciones de caos, no existe una universalmente aceptada. Antes de dar una definición, vamos primero a listar características esenciales del comportamiento caótico:

1. *Comportamiento asintótico aperiódico.*

Esto quiere decir que las soluciones del sistema, para algún abierto en el espacio de fase, no divergen a infinito, no se acercan a un punto fijo, no se acercan a una órbita periódica, ni presentan un comportamiento quasiperiódico.

2. Determinista.

La aperiodicidad del sistema no viene por la adhesión de ruido o aleatoridad en los parámetros o en las variables, sino que nace de la no linealidad de las ecuaciones.

3. Sensible a las condiciones iniciales.

Esto quiere decir que trayectorias cercanas se separan exponencialmente rápido (Formalizaremos este punto en la sección siguiente).

Definimos entonces como caos al comportamiento aperiódico a largo plazo en un sistema determinista que exhibe dependencia sensible a las condiciones iniciales [72].

2.2.2. Exponentes de Lyapunov

Los exponentes de Lyapunov, también conocidos como exponentes principales, proveen una forma de cuantificar que tanto se acercan o se separan las trayectorias de un sistema [19, 83]. Consideremos dos trayectorias en un espacio de fase en $\mathbb{R}^n$, una con condiciones iniciales $\bar{x}_0$ y otra con condiciones iniciales infinitesimalmente perturbadas respecto a la primera $\bar{x}_0 + \delta(0)$. Pasado un tiempo t , una trayectoria se encontrará en $\bar{x}(t)$ mientras que la perturbada se encontrará en $\bar{x}(t) + \delta(t)$. Si $\delta(t) > \delta(0)$ esto quiere decir que las trayectorias se separaron. En la figura 2.3 se presenta un diagrama de la situación.

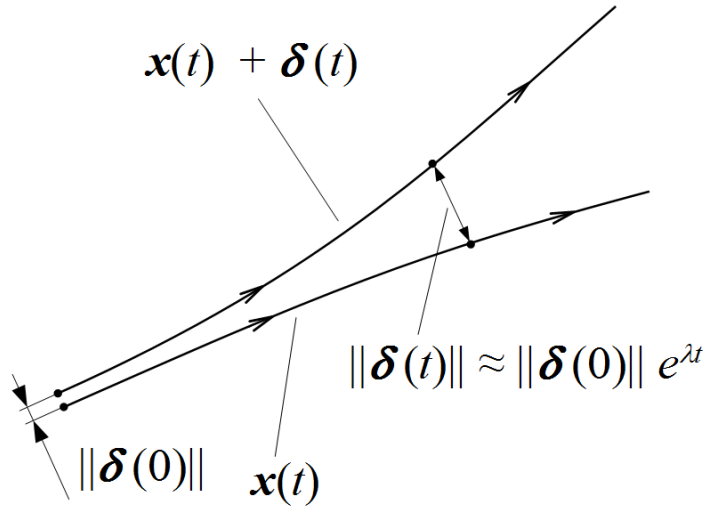


Figura 2.3: Diagrama de la evolución en el espacio de fase de dos trayectorias con condiciones iniciales cercanas. Imagen creada por Yapparina.

La perturbación $\delta(0)$ respecto a $\bar{x}_0$ puede ser en cualquier dirección del abierto infinitesimal alrededor de $\bar{x}_0$. Elegimos la dirección tal que la distancia $\|\delta(t) - \delta(0)\|$ sea máxima, llamaremos

a la perturbación en esta dirección $\delta_1(0)$. Buscamos estimar como el es ratio de separación exponencial entre las curvas en esta dimensión, es decir buscamos encontrar el valor de λ_1 en la expresión:

$$|\delta_1(t)| \approx e^{\lambda_1 t} |\delta_1(0)| \quad (2.2.1)$$

Entonces el exponente maximal de Lyapunov λ_1 se define analíticamente como:

$$\lambda_1 = \lim_{t \rightarrow \infty} \lim_{|\delta_1(0)| \rightarrow 0} \frac{1}{t} \ln \frac{|\delta_1(t)|}{|\delta_1(0)|} \quad (2.2.2)$$

Notemos que el límite $|\delta_1(0)| \rightarrow 0$ garantiza que se puede aproximar linealmente el sistema para todo t . Los demás exponentes $\lambda_2 > \dots > \lambda_n$ pueden calcularse de la misma forma, tomando las subsiguientes direcciones, ortogonales a las ya consideradas, que separen mas las trayectorias.

Como puede observarse de la expresión, el valor de los de los exponentes de Lyapunov varía según la zona del espacio de fases en la cual se los este calculando. Es por este motivo que, para computar los exponentes de Lyapunov correspondientes a un dado atractor, se suele trabajar con el promedio de los exponentes sobre el mismo. Esto es lo que se conoce como el exponente global de Lyapunov [19].

Podemos ahora formalizar la condición (3) de la sección anterior. Para considerar caótico el comportamiento de un sistema, es preciso que al menos uno de los exponentes de Lyapunov sea positivo. Esto quiere decir que, en al menos una dimensión, las trayectorias cercanas se separan exponencialmente rápido. Cuanto mas grande sea el exponente de Lyapunov positivo, mas rápida será esta separación y mas “caótico” será el sistema. En términos de aplicación al mundo real, esta amplificación de errores infinitesimalmente chicos en las condiciones iniciales se traduce en una incapacidad para predecir con precisión el comportamiento del sistema mas allá de un horizonte temporal. Esto se debe a que, para cualquier sistema, resulta inevitable el conocimiento imperfecto de su estado inicial. La escala de tiempo característica para este horizonte de predicción se conoce como el tiempo de Lyapunov y esta definido como $\tau_L = 1/\lambda_1$.

2.2.3. Atractor Caótico

Un atractor caótico es un set atractor, en el sentido definido en 2.1.3, que presenta sensibilidad a las condiciones iniciales del problema, o sea, que sus trayectorias poseen al menos un exponente de Lyapunov positivo. En esta tesis se usarán de manera intercambiable los términos atractor caótico y atractor extraño.

La existencia de este tipo de atractores con comportamiento caótico fue desconocida hasta el articulo de Lorenz en 1963 [78]. Antes de su trabajo, no se consideraba la posibilidad de un sistema dinámico que exhibiese al mismo tiempo las propiedad de ser globalmente estable, localmente inestable y con trayectorias asintóticamente aperiódicas. Es decir, un atractor caótico tiene la propiedad de atraer a todas trayectorias en un abierto U que lo contiene (llamada

región de atracción), pero una vez en su interior, las trayectorias se separan exponencialmente rápido cuando están cercanas entre sí. Este tipo de comportamiento está prohibido en sistemas bidimensionales por el teorema de Poincaré-Bendixon, pero puede aparecer en sistemas de dimensión $n = 3$ en adelante.

En su artículo, Lorenz estudia un problema de convección aproximándolo por un sistema dinámico de dimensión $n = 3$. El famoso sistema tiene la forma:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(x - y) \\ \frac{dy}{dt} &= x(\rho - z) - y \\ \frac{dz}{dt} &= xy - \beta z.\end{aligned}\tag{2.2.3}$$

Para una elección de parámetros $\sigma = 10$, $\rho = 28$ y $\beta = 8/3$, este sistema tiene como solución el atractor que lleva su nombre. En la figura 2.4 se presenta el diagrama de fases para dos condiciones iniciales distintas. Además del atractor caótico, existen otros conjuntos límites que conviven con él en la misma región del espacio de fases, todos ellos inestables. En la figura se representan los 3 puntos fijos que coexisten con el atractor como círculos abiertos, pero también existen orbitas periódicas.

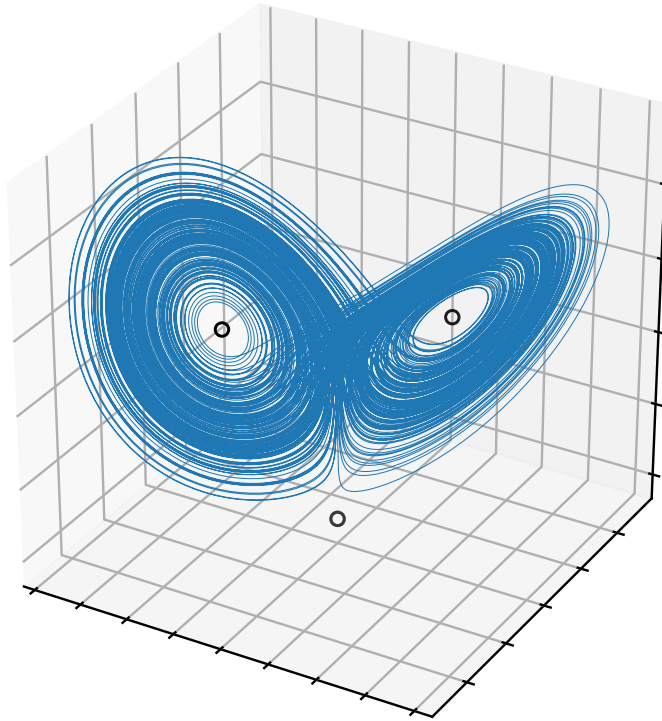


Figura 2.4: Representación del atractor de Lorenz realizada mediante una simulación del flujo para un sistema de Lorenz. Solo se presenta la solución para tiempos muy posteriores al transitorio inicial. Se presentan también los 3 puntos fijos inestables que conviven con el atractor.

Los atractores caóticos poseen una estructura bastante complicada en espacio de fases. Supongamos que se interseca un plano con el atractor y se analiza el set de puntos resultantes,

lo que se conoce como una sección de Poincaré [72]. Los puntos resultantes formarían un set de cantor topológico. Esto quiere decir que, si bien no hay un continuo de puntos (cada punto está aislado), todos los puntos tienen vecinos infinitesimalmente cerca. Es por este motivo que decimos que las trayectorias en un atractor caótico son densas y que los atractores caóticos presentan una estructura fractal. Otra característica fractal que puede encontrarse en los atractores es la de autosimilitud.

Dada su estructura fractal, la dimensión de un atractor extraño resulta generalmente no entera y existen diferentes maneras de estimarla, algunas de ellas son: la *box dimension* [84], la dimensión de correlación [85] y la dimensión de Kaplan-Yorke [86], esta última utiliza el valor de los exponentes de Lyapunov para calcularla. A pesar de ser distintas, todas estas medidas de la dimensión fractal suelen arrojar resultados similares para un mismo atractor, en el caso del de Lorenz $D \approx 2,06$.

Si calculamos los exponentes de Lyapunov para un atractor caótico, sabemos que su exponente maximal λ_1 será positivo. En el caso de estar en $\mathbb{R}^3$, el siguiente exponente λ_2 será cero. λ_2 es el exponente que está asociado a la dirección del flujo, y por ende en esta dirección las trayectorias vecinas no se separan ni acercan exponencialmente. Por ultimo el exponente menor λ_3 será negativo, en esta dirección el flujo se contrae, las trayectorias se acercan entre si. Notemos que, para tener un atractor, es necesario que, al evolucionar el tiempo, el volumen dado por las condiciones iniciales en un abierto alrededor del atractor se vaya reduciendo. Esto quiere decir que $\lambda_1 + \lambda_2 + \lambda_3 < 0$ y que el sistema, a pesar de tener una dirección repulsora, es disipativo.

2.2.4. Mecanismos del caos

Como se discutió en la subsección 2.2.2, un sistema caótico posee al menos un exponente de Lyapunov positivo. Esto quiere decir que, localmente y en esa dirección, las trayectorias se están separando exponencialmente. A este proceso de “estiramiento” del flujo lo llamamos *stretching*.

Pero las trayectorias no divergen exponencialmente hasta el infinito. Las soluciones del sistema están acotadas a la región del atractor caótico. En algún momento estas trayectorias llegan a un máximo de separación y vuelven a acercarse la una a la otra. A este proceso de “acercamiento” del flujo lo llamamos *squeezing*. El comportamiento caótico es generado por la acción repetida de estos dos procesos.

En particular, los atractores caóticos se generan a partir de este proceso de *stretching* y *squeezing* de las trayectorias en el espacio de fases. La repetición del proceso es la responsable de la autosimilitud de los atractores caóticos. Este también es el proceso mediante el cual se generan las órbitas periódicas inestables que conviven con el atractor.

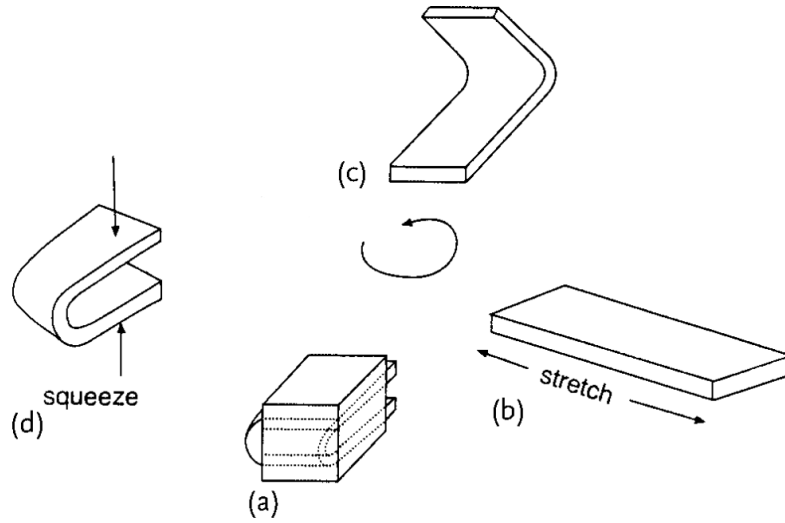


Figura 2.5: Representación del mecanismo de estirar y doblar (*stretch-and-fold*), ver el texto para una explicación detallada del proceso. La imagen es adaptada de [73].

Existen muchos mecanismos distintos de *stretching* y *squeezing*. A modo de ejemplo, en la figura 2.5 presentamos el mecanismo de estirar y doblar (*stretch-and-fold*) en $\mathbb{R}^3$, tomada de [73]. Comenzamos considerando un cubo de condiciones iniciales 2.5(a). A medida que avanza el tiempo el cubo se estira exponencialmente en la dirección del exponente local de Lyapunov positivo y se aplasta en la dirección del exponente local de Lyapunov negativo 2.5(b). Eventualmente, estos puntos alcanzan un máximo de separación y son acercados nuevamente 2.5(c). Luego de un tiempo finito en que sucede este proceso, parte de la imagen del cubo inicial de condiciones iniciales se reinyecta en el mismo cubo 2.5(a). Esta reinyección del cubo en si mismo es la que genera la posibilidad de orbitas periódicas, las cuales serán de periodo 1, 2, 3, etc... en función de la cantidad repeticiones en las cuales la orbita se vuelve a enganchar con ella misma.

La repetición del mecanismo de *stretch-and-fold* es el responsable de generar el comportamiento caótico en muchos sistemas distintos. Entre ellos podemos destacar el mapa de herradura [87] y también el sistema de Rössler. El sistema de Rössler es un sistema de ecuaciones diferenciales de $\mathbb{R}^3$ con una única no linealidad que puede generar comportamiento caótico, el mismo tiene la siguiente forma:

$$\begin{aligned}\frac{dx}{dt} &= -y - z \\ \frac{dy}{dt} &= x + ay \\ \frac{dz}{dt} &= b + z(x - c)\end{aligned}\tag{2.2.4}$$

En la figura 2.6 presentamos el atractor caótico de Rössler generado por los parámetros $a = b = 0,1$ y $c = 14$. A su lado se muestran las órbitas periódicas inestables que conviven

con el atractor de período 1, período 2 y una de las dos posibles de período 3. En este ejemplo puede observarse con claridad el proceso de *textit{stretch-and-fold}* en el flujo. Al mismo tiempo es interesante notar que el flujo visita con frecuencia la vecindad de órbitas periódicas inestables, ya que la densidad de trayectorias es mayor en esas zonas del espacio de fases. Esto es una característica de todos atractores caóticos, uno puede pensar que las órbitas son una suerte de “esqueleto” del atractor.

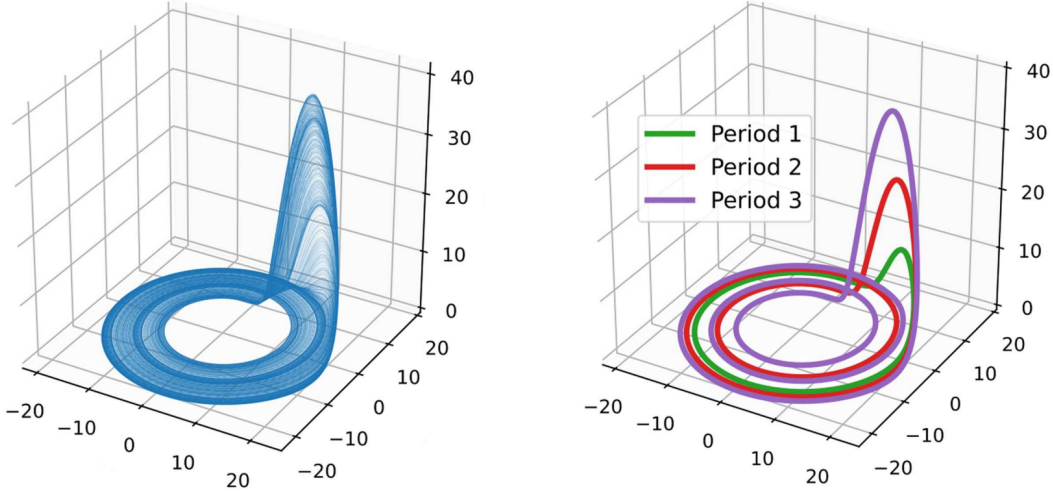


Figura 2.6: A la izquierda se presenta una simulación del flujo para un sistema de Rössler, solo se presenta la solución para tiempos muy posteriores al transitorio inicial. A la derecha se presenta una aproximación numérica de algunas orbitas periódicas inestables en ese mismo sistema.

Antes de pasar a la siguiente sección, vamos a realizar un comentario acerca de la equivalencia topológica entre flujos. Decimos que dos flujos, ϕ_t y ψ_t , son topológicamente equivalentes cuando existe un homeomorfismo, h , capaz de llevar las trayectorias de ϕ_t a ψ_t preservando su orientación [88]. Esto quiere decir que, por ejemplo, dos flujos relacionados por un cambio suave de coordenadas pueden ser geoméricamente distintos pero topológicamente equivalentes. En cambio, entre dos flujos topológicamente inequivalentes, por ejemplo un atractor de Lorenz y uno de Rössler, no existe un cambio de coordenadas suave que nos permita pasar de uno a otro, ambos son cualitativamente distintos.

2.2.5. Organización topológica y *linking numbers*

En esta sección nos enfocaremos únicamente en el caso en que los atractores se encuentran en $\mathbb{R}^3$.

Cada uno de los mecanismos de *stretching* y *squeezing* organiza las órbitas periódicas inestables que coexisten con el atractor caótico de una manera particular y única. Es por este motivo que la mejor manera para identificar mecanismos y caracterizar un atractor es determinar como

se organizan las órbitas que conviven con el. Las trayectorias periódicas son nudos orientados, y su organización es topológica.

Para describir la organización topológica de las órbitas utilizaremos un invariante topológico llamado *linking number*. Este invariante caracteriza la forma en que dos nudos orientados se anudan entre sí. El *linking number* siempre es un entero y para órbitas desenganchadas su valor es 0. En la figura 2.7 presentamos ejemplos de los *linking numbers* para varios pares de curvas.

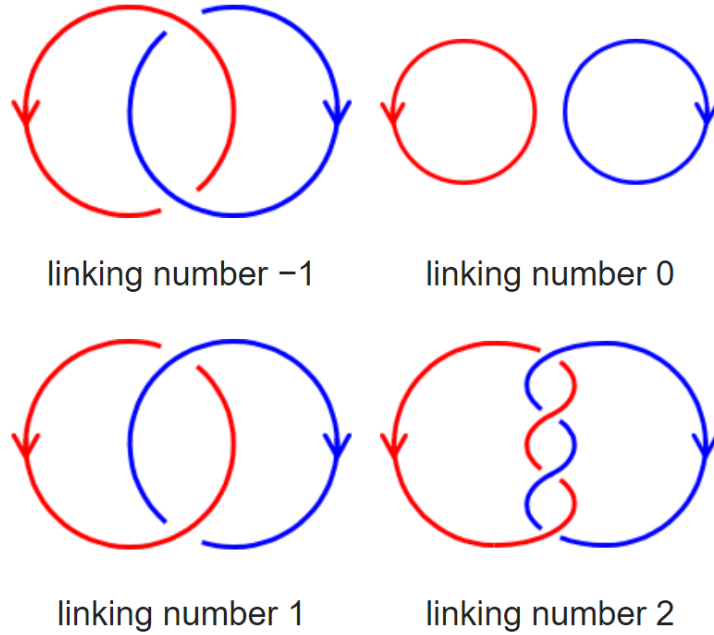


Figura 2.7: Ejemplos de pares de curvas orientadas y sus respectivos *Linking Numbers*. La imagen es de dominio público.

El concepto de *Linking Number*, como tantos otros conceptos la matemática moderna, fue introducido por Carl Friedrich Gauss, quien presentó una forma integral de computarlo [73]. Sean $x_A(s)$ y $x_B(s')$ la parametrización de dos curvas cerradas en $\mathbb{R}^3$, el *Linking Number* entre las mismas quedará definido por:

$$L(A, B) = \frac{1}{4\pi} \oint_A \oint_B \frac{(x_A - x_B) \cdot (dx_A \times dx_B)}{|x_A - x_B|^3}. \quad (2.2.5)$$

Gauss mostró que el resultado de esta integral es un invariante y su resultado es un número entero mientras las curvas no se intersecten entre si, condición que siempre se cumple en las orbitas del atractor. Esta integral es difícil de resolver, mas aún cuando no tenemos la expresión analítica de las órbitas y estamos computándola a partir de datos. Una forma mas sencilla de calcular el *linking number* es usando la representación de los nudos sobre un plano. Estas proyecciones preservan la información de los cruces, y permiten computar el *linking number*. Detallaremos esta metodología en el capítulo 4.

Podemos, entonces, caracterizar un dado atractor caótico por la serie de números corres-

pondientes a los *Linking Number* entre cada par de órbitas periódicas inestables. Estos números funcionan como una huella digital del atractor [89]. Más aún, como existen muchas mas órbitas periódicas en un atractor extraño que mecanismos de *stretching* y *squeezing* para generar atractores caóticos, será en general posible determinar la organización topológica del atractor computando los *linking numbers* únicamente entre un número limitado de pares de orbitas. La organización del resto de las órbitas estará supeditada al mecanismo de generación de caos identificado a partir de esas pocas órbitas [73].

Como último comentario en esta sección, vamos a describir una situación donde se pone en manifiesto la utilidad de los *linking numbers* en el contexto del modelado. Supongamos que logramos reconstruir el flujo de un sistema estudiado a partir de mediciones del mismo, y que en esta reconstrucción se obtiene un atractor caótico (como, por ejemplo, la situación presentada en la figura 1.2 del capítulo anterior). Supongamos, además, que contamos con un sistema de ecuaciones el cual consideramos un buen candidato para modelar el sistema estudiado, y que la solución del mismo resulta ser también un atractor caótico. La pregunta natural en estas circunstancias será: ¿Cómo podemos hacer para validar nuestro modelo a partir de las mediciones?

La validación tendrá como elemento central la comparación entre ambos atractores, el experimental y el teórico. En el caso general, no esperamos que ambos sean idénticos, puede que las variables de nuestro sistema de ecuaciones no se correspondan exactamente con las reconstruidas a partir de los datos, y que exista entonces alguna deformación suave de las trayectorias. Lo que vamos a considerar esencial y exigirle a nuestro modelo es que la organización topológica de ambos sea la misma, ya que el ordenamiento de las órbitas es una característica robusta que determina propiedades importantes del flujo. Podemos entonces computar numéricamente los *linking numbers* entre segmentos que sean aproximaciones de algunas de las órbitas inestables en cada atractor, y evaluar si el conjunto de enteros computados para cada uno coincide o no. Si los mismos no coinciden, consideraremos que el modelo no reproduce adecuadamente la topología del flujo y lo descartaremos. En cambio, si el conjunto de *linking numbers* computados resulta igual para ambos atractores, esto construirá confianza en la hipótesis de que ambos atractores son topológicamente equivalentes y que nuestro modelo captura adecuadamente las características dinámicas del sistema.

2.3. Análisis no-lineal de series temporales

Cómo previamente se comentó en la introducción 1.3, el análisis no-lineal de series temporales son un conjunto de herramientas para predicción y caracterización de series temporales basado en la teoría de sistemas dinámicos. Todas ellas parten de la idea que la serie temporal estudiada surge de la medición de un sistema dinámico. De esta forma, es posible estimar información dinámica del sistema para caracterizarla. Algunas de las metodologías del área para

estudiar series temporales son: los gráficos de recurrencia [90,91], el computo de los exponentes de Lyapunov, el computo de la dimensión fractal y, si es posible, la reconstrucción de un sistema de ecuaciones. Una idea central en todas las metodologías del análisis no-lineal de series temporales es la de reconstrucción del espacio de fases.

2.3.1. Reconstrucción del flujo

La idea de poder reconstruir a partir de una serie temporal univariada el espacio de fases del sistema es notable, y fue sugerida primeramente por Packard [92]. Al poco tiempo, basándose en resultados previos de Withney [93], Takens logró formalizarla [22]. Su resultado, el Teorema de Takens, fue fundamental para el desarrollo del área. Posteriormente, los resultados del teorema fueron extendidos de varias formas, destacamos [94].

La hipótesis subyacente es que, dada una serie temporal de N elementos $\{x_1, x_2, \dots, x_N\}$ provenientes de la solución de un sistema dinámico, un segmento de $N_1 < N$ elementos de la serie $\{x_{k-N_1+1}, x_{k-N_1+2}, \dots, x_k\}$ contiene suficiente información para poder determinar el siguiente segmento $\{x_{k-N_1+2}, x_{k-N_1+3}, \dots, x_{k+1}\}$. El determinismo del sistema dinámico requiere que existe un único valor x_{k+1} para cualquiera de estos segmentos. De esta forma es posible representar el estado de la serie por un punto en $\mathbb{R}^{N_1}$.

Al proceso de reconstrucción fidedigna del retrato de fases correspondiente a una serie temporal se lo llama *embedding*. Formalmente, un *embedding* de una variedad A es un difeomorfismo de clase C^1 desde A hasta su imagen $f(A)$, esto es un mapa uno a uno C^1 con inversa C^1 [94]. En el contexto de reconstrucción del retrato de fases, esto refiere a que el flujo reconstruido preserva la topología del flujo original y existe una correspondencia uno a uno global entre ambos.

2.3.2. Teorema de Takens

Takens fue el primero en formalizar el mecanismo de reconstrucción del flujo, es decir, brindó un teorema que asegura que es posible realizar *embeddings* de series temporales. Probar el teorema de Takens está fuera del alcance de la presente tesis, para una discusión técnica detallada y una prueba del mismo pueden referirse a [22], [95] o [96]. Lo que haremos, en cambio, es presentar su enunciado y discutir su significado.

El teorema original de Takens enuncia lo siguiente [22,95]: Sea M una variedad compacta de dimensión m . Para pares (ϕ, y) , con $\phi \in \text{Diff}^2(M)$, $y \in C^2(M, \mathbb{R})$, es una propiedad genérica del mapa $\Phi_{(\phi, y)} : M \rightarrow \mathbb{R}^{2m+1}$, definido por

$$\Phi_{(\phi, y)}(x) = (y(x), y(\phi(x)), \dots, y(\phi^{2m}(x))) \quad (2.3.1)$$

ser un embedding.

A continuación se especifica al lector el significado de cada uno de los términos utilizados en el enunciado:

- Una variedad es a un espacio topológico que localmente se asemeja a $\mathbb{R}^n$, esto quiere decir que cada punto tiene un abierto en su vecindad que es homeomorfo a $\mathbb{R}^n$.
- $\text{Diff}^r(M)$ refiere a todas las funciones $f : M \rightarrow M$ que son C^r y que tienen inversa C^r , o sea, difeomorfismos de clase C^r .
- $C^r(M, \mathbb{N})$ refiere a todos los mapas C^r que van de M a N .
- “Genérica” en esta definición se refiere a que es topológicamente abierta y densa. A grandes rasgos, significa que vale en casi todos los casos (quitando contadas excepciones).

En el contexto de sistemas dinámicos, $\phi : M \rightarrow M$ es la función que genera la evolución temporal del flujo, es decir el campo vector dado por nuestras ecuaciones diferenciales. Por otro lado, la función $y : M \rightarrow \mathbb{R}$ es nuestra medición, una función que va desde el espacio de fases a $\mathbb{R}$. La serie temporal a partir de la cual buscamos reconstruir el espacio de fases son los valores medidos por y a lo largo del tiempo.

Takens ofrece dos posibilidades para construir $\Phi_{(\phi, y)}$ a partir una serie temporal. Una es inspeccionar el valor de la serie cada una cantidad finita de tiempo τ y tomar $2m + 1$ elementos, es decir tomar los puntos $\{x(t), x(t + \tau), \dots, x(t + 2m\tau)\}$. En general los valores de x suelen ser alguna variable dinámica del problema, por lo cual nuestra función de medición y sería simplemente proyectar el estado en el espacio de fases sobre esa variable. A esta manera de construir el *embedding* se la conoce como el mecanismo de *delays* temporales.

La otra forma es tomar los $2m + 1$ elementos generados por la variable x y sus sucesivas derivas, $\{x(t), \frac{dx(t)}{dt}, \dots, \frac{d^m x(t)}{dt^m}\}$. Esta segunda forma es mucho menos conveniente, ya que el computo numérico de las derivadas es sensible a errores e implica una operación extra a realizar. Por estos motivos no suele ser usada en análisis de datos.

2.3.3. *Embeddings* en la práctica

Gracias al teorema de Takens y sus extensiones, sabemos que, dada una serie temporal proveniente de un sistema dinámico, es posible realizar un *embedding* de la misma usando el mecanismo de *delays* temporales. Pero, para realizarlo, precisamos determinar dos parámetros decisivos: la dimensión del *embedding* y el *delay* temporal que usaremos.

Primero, discutiremos sobre la elección del *delay* temporal a utilizar, τ . En el caso de tener una serie experimental, por lo general, la misma tendrá la forma $\{x(0), x(\delta), \dots, x((N - 1) * \delta)\}$, siendo τ un intervalo de sampleo fijo. Entonces los posible valores del *delay* temporal a elegir serán $\tau = n\delta$, con $n \in \mathbb{Z}$. Una de las excepciones en las cuales el teoremas de *embedding* no

funcionan, es cuando $\tau = P$ o $\tau = 2P$, siendo P el periodo de un ciclo límite en el flujo original. Quitando estos casos particulares y cumpliendo las hipótesis del teorema, en condiciones ideales, cualquier valor de τ funcionaría bien.

Pero en la práctica, esto no es cierto. La calidad de *embedding* depende de manera crucial del valor de τ utilizado. Por un lado, las series suelen medirse con cierto ruido. Si el valor del *delay* temporal es mucho menor los tiempos característicos del sistema, puntos distantes del sistema original quedarán representados en puntos cercanos en el espacio de fases reconstruido. Cuando esto sucede decimos que el flujo no está bien expandido en el espacio reconstruido. En estos casos la reconstrucción se verá fundamentalmente afectada por el ruido, mecanismo que domina la variación de la señal en esa escala temporal. Por otro lado, muchas veces es preciso utilizar un espacio de *embeddings* con dimensión mas baja que aquella en la cual el teorema nos garantiza un *embeddings* correcto. En estas circunstancias, el valor de τ juega un rol central en determinar si la reconstrucción del flujo será topológicamente correcta.

Existen algunos métodos para intentar determinar un buen valor de τ a partir de los datos, pero ninguno asegura la elección de un valor adecuado. Entre ellos los mas comúnmente usados son: el computo del primer mínimo de la autocorrelación de la serie y el computo del primer mínimo de la información promedio mutua de la serie [19].

Por último discutiremos sobre la elección de la dimensión del espacio de *embeddings*, N . La misma debe ser lo suficientemente grande como para garantizar que no haya intersecciones de las trayectorias en el espacio reconstruido. En [94] prueban que es posible relajar la cota del teorema de Takens de $N \geq 2m+1$ a $N > 2m$, donde m es la dimensión original del flujo y puede ser una dimensión no entera (fractal). A pesar de esta mejora en la cota, el valor de N dado por los teoremas de *embedding* resulta, en la mayoría de los casos, no óptimo. Por ejemplo, para un atractor de Lorenz cuya dimensión es $D_0 \approx 2,06$, la cota de garantía dada por los teoremas es $N = 5$. Sin embargo, sabemos que el atractor puede inscribirse en $N = 3$ y que podemos lograr un *embedding* en esta dimensión eligiendo un valor adecuado para el *delay* temporal.

Por otro lado, cuando trabajamos con series de tiempo experimentales, no conocemos a priori la dimensionalidad m del flujo original. Resulta entonces preciso utilizar alguna metodología numérica para estimarla la dimensión de *embedding* N a partir de los datos. La mas utilizada es el computo de “falsos vecinos” [14,97]. Esta es una metodología que logra determinar valores precisos de N para series sin ruido, pero a la cual le cuesta mucho hacerlo a medida que la relación señal ruido crece.

Como conclusión de esta sección se destacan dos observaciones. Por un lado la importancia que tuvieron el teorema de Takens y los subsiguientes teoremas de *embedding* en formalizar el concepto de reconstrucción del espacio de fases y brindar un marco teórico sólido al análisis no-lineal de series temporales. Por otro, que en la práctica, el éxito de un *embeddings* depende sensiblemente del valor de la dimensión N y el *delay* τ . Esto resulta poco conveniente siendo

que, al no contar con una forma rigurosa de calcularlos, la elección de ambos parámetros debe hacerse de manera aproximada.

Capítulo 3

Redes Neuronales

“I checked it very thoroughly” said the computer, “and that quite definitely is the answer. I think the problem, to be quite honest with you, is that you’ve never actually known what the question is.”

Douglas Adams, *The Hitchhiker’s Guide to the Galaxy*, 1979

En este capítulo se introducen los algoritmos de redes neuronales que se utilizarán en los siguientes capítulos. En la primer sección se desarrollará brevemente el paradigma del aprendizaje supervisado. En la siguiente sección se definirán las redes neuronales, se comentará su método de entrenamiento y se discutirán brevemente las razones de su éxito. Las últimas secciones del capítulo se focalizarán en los dos tipos particulares de redes neuronales que utilizamos a lo largo del trabajo de tesis: *Deep Autoencoders* y Redes Neuronales Recurrentes. En ambos casos, se presentará la arquitectura de las mismas y sus antecedentes de aplicación a problemas de dinámica. Para algunos de los temas desarrollados en este capítulo se siguieron los libros: Goodfellow (2016) [29] y Smola (2021) [98].

3.1. Aprendizaje supervisado

3.1.1. El paradigma de aprendizaje supervisado

Un algoritmo de *Machine Learning* es un algoritmo que puede aprender a partir de datos. Para ser precisos, vamos a definir el término “aprender” en este contexto de la siguiente forma: Se dice que el algoritmo aprende de la experiencia E con respecto a alguna clase de tareas T y la medida de desempeño P , si su desempeño en tareas en T , medido por P , mejora con la experiencia E (definición tomada de [99]). Existen muchos tipos de algoritmos que se ajustan

a esta definición, en esta sección nos focalizaremos en algoritmos de aprendizaje supervisado. Típicamente, en esta clase de algoritmos, la tarea T es asociar vectores de entrada $\bar{x} \in \mathbb{R}^n$ a vectores objetivo $\bar{y} \in \mathbb{R}^m$, los vectores $\bar{x}$ también se suelen llamar vectores de *features*. El desempeño P es una función de costo $\mathcal{J}$ que cuantifica la diferencia entre el objetivo real $\bar{y}$ y el predicho por el algoritmo $\hat{\bar{y}}$. La experiencia E consiste en un conjunto de N pares $(\bar{x}, \bar{y})$, $(\bar{X}, \bar{Y})$, que el algoritmo tomará como ejemplos para intentar dilucidar la relación subyacente y aproximar la función objetivo $F : \mathcal{X} \rightarrow \mathcal{Y}$, donde $\mathcal{X}$ es el conjunto de todos los posibles vectores de entrada $\bar{x}$ (del cual $\bar{X}$ es una muestra), e $\mathcal{Y}$ es el conjunto de todos los vectores objetivos $\bar{y}$ asociados.

Normalmente, la aproximación a la función objetivo F se busca dentro de una clase de funciones f_θ , a la que se le suele llamar el *modelo*, donde θ hace referencia a los parámetros en esa clase de funciones. Es decir, se busca el valor de parámetros $\theta^* \in \Theta$ tal que la aproximación a F sea la óptima dentro del conjunto de funciones f_θ dado por todas las posibles parametrizaciones Θ . La aproximación óptima será aquella que minimice la función de costo $\mathcal{J}$ en todo el conjunto de entrenamiento $(\bar{X}, \bar{Y})$. Podemos entonces formalizar la resolución de un problema de aprendizaje supervisado como la búsqueda de los parámetros óptimos θ^* tal que:

$$\theta^* = \arg \min_{\theta \in \Theta} [\mathcal{J}(\bar{y}_i, f_\theta(\bar{x}_i))], \quad (3.1.1)$$

donde $(\bar{x}_i, \bar{y}_i) \in (\bar{X}, \bar{Y})$ son todas las instancias de entrenamiento.

El ejemplo mas sencillo de un algoritmo de aprendizaje supervisado es el de regresión lineal. En este caso, la clase de funciones en que buscamos la aproximación a la función objetivo son las funciones lineales. Es decir $f_\theta = f_{\bar{w}, b} = \bar{w}^T \bar{x} + b$, donde el vector de pesos $\bar{w}$ y la ordenada al origen b son los parámetros θ de la clase. Usualmente, la función de costo es la suma de una función de pérdida $\mathcal{L}$ para cada una de las instancias y, en el caso de una regresión lineal, esta función $\mathcal{L}$ se suele tomar como el cuadrado de la distancia entre el valor predicho y el objetivo, lo que se conoce como cuadrados mínimos:

$$\mathcal{J} = \sum_{(\bar{x}_i, \bar{y}_i)} \mathcal{L}(\bar{y}_i, f_{\bar{w}, b}(\bar{x}_i)) = \sum_{(\bar{x}_i, \bar{y}_i)} (\bar{y}_i - f_{\bar{w}, b}(\bar{x}_i))^2. \quad (3.1.2)$$

Es decir que el problema a resolver en una regresión lineal será encontrar $\bar{w}^*$ y b^* tal que:

$$\bar{w}^*, b^* = \arg \min_{\bar{w}, b} \left[\sum_{(\bar{x}_i, \bar{y}_i)} (\bar{y}_i - f_{\bar{w}, b}(\bar{x}_i))^2 \right]. \quad (3.1.3)$$

El caso de la regresión lineal es uno de los pocos para los cuales contamos con una manera analítica cerrada para encontrar los valores óptimos de los parámetros θ . En el caso general, será preciso usar alguna estrategia de optimización que nos devuelva un valor de parámetros el cual consideremos suficientemente bueno en función de algún criterio.

3.1.2. El compromiso sesgo-varianza

Una de las elecciones mas importantes a la hora de plantear un problema de aprendizaje supervisado es cuál será el modelo a utilizar, en otros términos, sobre qué clase de funciones f_θ buscaremos la aproximación de F . Por lo general, cuantos mas parámetros tenga el modelo, mayor será la variedad de funciones que pueda aproximar. A esta característica se la llama capacidad o expresividad del modelo.

Si un modelo tiene baja capacidad respecto a la tarea objetivo, será imposible lograr una buena aproximación de la función objetivo F , no importa la cantidad o la calidad de datos que tengamos disponibles. A este régimen se lo conoce con el nombre de *underfitting*, y se llama sesgo al error introducido en la aproximación por esta causa. En el otro extremo, si el modelo es demasiado expresivo para la complejidad de la tarea objetivo y la cantidad de datos disponibles, se corre el riesgo de realizar un ajuste en donde el modelo aprende perfectamente todos los datos de entrenamiento y no es capaz de generalizar, desempeñando mal la tarea en datos fuera de ese conjunto. A este otro régimen se lo conoce como *overfitting*, y se llama varianza al error de generalización que se produce por esta causa.

Esto implica que, si buscamos un modelo cuyo error de generalización sea bajo, precisamos encontrar un compromiso en la expresividad del modelo: Queremos que sea lo suficientemente rico como para expresar la estructura subyacente en los datos, pero lo suficientemente simple para evitar que se ajuste a patrones espurios. A este balance se lo conoce como el compromiso sesgo-varianza (*bias-variance tradeoff*). De hecho, para problemas de regresión donde se busca la mejor aproximación en términos del error cuadrático medio y los datos poseen un error ϵ con varianza definida, es posible separar el error de generalización esperado en la suma de tres contribuciones fundamentales: un error irreducible que viene de la varianza de ϵ en los datos, uno que viene por el sesgo del modelo f_θ y otro que viene por su varianza [100].

Para evaluar la capacidad de generalización de un modelo la práctica mas utilizada es separar el conjunto inicial de datos en dos partes. Una parte mayoritaria llamada conjunto de entrenamiento, la cual se utilizará para ajustar los parámetros del modelo. Y otra parte minoritaria, llamada conjunto de testeo, la cual se utilizará exclusivamente para la evaluación del mismo. Un buen ajuste es aquel que minimiza el error cometido por el modelo sobre el conjunto de testeo. En la figura 3.1, se presenta el ajustes de tres modelos con distinta capacidad a un mismo conjunto de datos en una tarea de regresión. Uno de estos modelos está en régimen de *underfitting* (predomina el error de sesgo), otro de estos modelos esta en régimen de *overfitting* (predomina el error de varianza) y, por último, un tercer modelo se encuentra en el óptimo del compromiso, donde el error en sobre conjunto de testeo es mínimo.

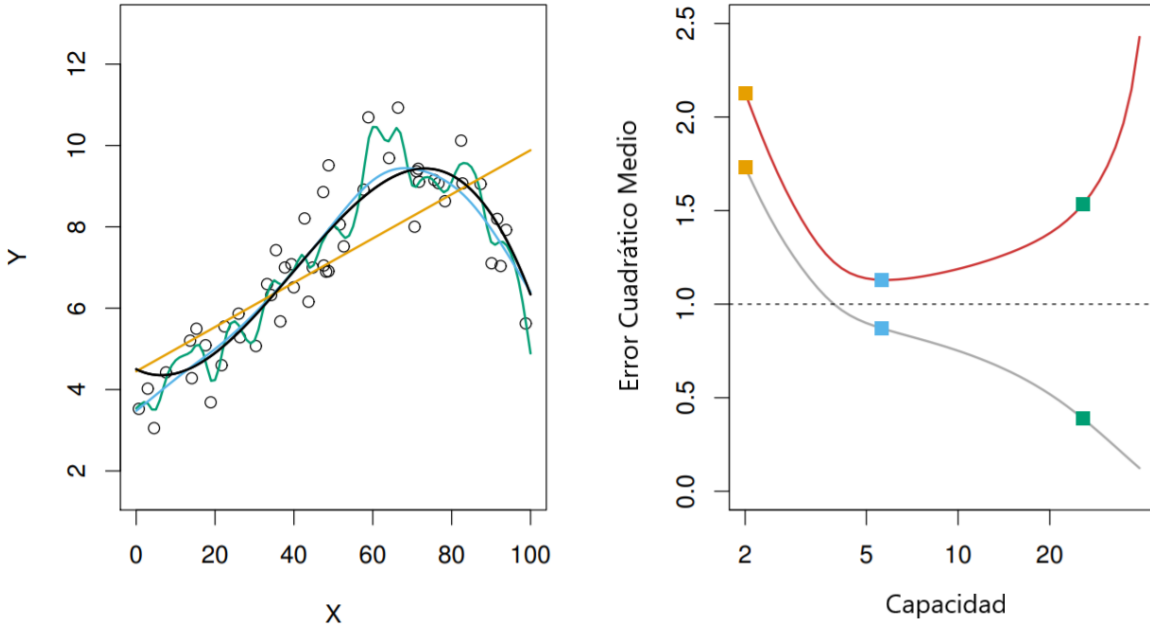


Figura 3.1: En el panel de la izquierda se presentan en círculos negros datos simulados de una función $y = F(x) + \epsilon$, donde ϵ es un término de ruido. Se muestran tres aproximaciones de F realizadas a partir de los datos con modelos de distinta capacidad, una regresión lineal (curva naranja) y dos ajustes de *smoothing spline* (curva azul y curva verde). En el panel de la derecha se muestran el MSE en el conjunto de entrenamiento (curva gris) y el MSE en el conjunto de testeo (curva roja) como función de la capacidad del modelo. Los cuadrados sobre esta curva representan los tres ajustes que se muestran en el panel de la izquierda. También se muestra en línea punteada el mínimo error posible sobre los datos de prueba (el error irreducible). Imagen adaptada de [100].

3.1.3. El teorema de *No Free Lunch*

Un concepto relevante en *Machine Learning* es el hecho de que no existe un algoritmo que sea universalmente mejor que todos los demás. El teorema de *No Free Lunch* (No hay almuerzo gratis) de Wolpert y Macready establece que, promediados sobre todas las posibles distribuciones generadoras de puntos, todo algoritmo de clasificación tiene el mismo error de generalización sobre puntos no observados [101]. Es decir que no será posible diseñar un algoritmo que sea el mejor para todas las posibles tareas.

Pero este resultado solo es cierto cuando se promedia sobre todas las posibles distribuciones generadoras de datos. En la práctica es posible asumir ciertas propiedades sobre las distribuciones de datos en las que se usará un dado algoritmo, y por ende es posible diseñar algoritmos que sean particularmente adecuados para cierto tipo de tareas [29]. Un claro ejemplo de un algoritmo que fue adaptado a distintos formatos de datos son las redes neuronales. Si se desea trabajar con datos secuenciales, las redes recurrentes son una opción adecuada, mientras que, si

se desean utilizar un algoritmo para trabajar con datos en formato de imágenes, probablemente las redes convolucionales sean la mejor alternativa.

3.2. Redes Neuronales y *Deep Learning*

3.2.1. La neurona artificial

El elemento constitutivo fundamental de una red neuronal es la neurona artificial, también llamado perceptrón. La neurona artificial es una función que toma un vector de *features* $\bar{x} \in \mathbb{R}^n$ como entrada y devuelve un valor escalar a la salida. Este valor se computa como:

$$a_{\bar{w},b} = g(\bar{w}^T \bar{x} + b) \quad (3.2.1)$$

donde el vector $\bar{w}$ corresponde a los pesos asignados a cada uno de los *features* de entrada, el valor b es el *bias* y la función g es la función de activación. En la figura 3.2 se presenta un diagrama de una neurona artificial.

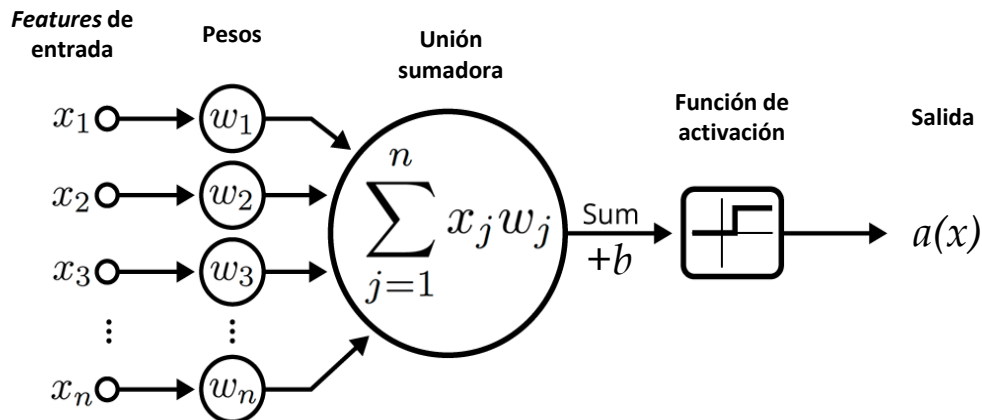


Figura 3.2: Diagrama de una neurona artificial. Primero se realiza el producto interno entre vector de *features* de entrada y el vector de pesos en lo que se denomina la unión sumadora. Luego, se suma el valor del *bias* al resultado. Finalmente se aplica la función de activación para obtener la salida. La imagen es adaptada del sitio becominghuman.ai.

Algunas de las funciones de activación mas utilizadas en neuronas artificiales son:

- Sigmoide: $g(y) = \frac{1}{1+e^{-y}}$.
- Tangente Hiperbólica: $g(y) = \frac{e^y - e^{-y}}{e^y + e^{-y}}$.
- ReLU (*Rectified Linear Unit*): $g(y) = \max(0, y)$.

Notemos que la función de activación es lo que le confiere a la neurona artificial su caracter no-lineal. En el caso de la activación tipo *ReLU*, si bien cada una de las partes de la función es lineal,

la no-linealidad viene por estar definida a trozos. Gracias a la no-linealidad es posible combinar neuronas artificiales para generar modelos que puedan expresar funciones mas complejas entre los *features* de entrada y la salida.

3.2.2. Red neuronal *feedforward*

Las redes neuronales *feedforward* (también conocidas como *fully-connected* o perceptrón multicapa) son un tipo de modelo de *Machine Learning* que se construye a partir de la concatenación de L capas de neuronas artificiales. En este esquema, cada capa de neuronas toma como vector de entrada la salida de la capa anterior. El número de capas L , y los números de neuronas por capas N^l son hiperparámetros que definen lo que llamamos *arquitectura* de la red. El término hiperparámetro refiere a una característica del modelo que se definirá a priori, previo al proceso de entrenamiento. La elección de L y de los N^l determinará la clase de funciones f_θ donde se buscará una buena aproximación de la función objetivo F . En el caso de una red neuronal, los parámetros del modelo son los pesos y los *bias* de cada una de las neuronas en cada capa.

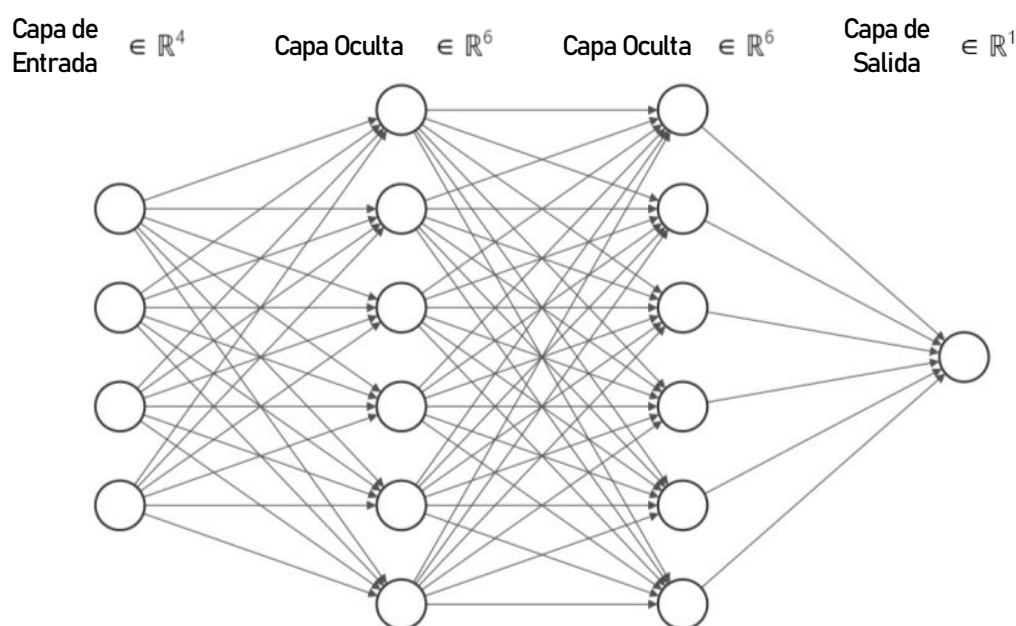


Figura 3.3: Esquema de una red neuronal *feedforward* con 4 capas. Una primer capa de entrada de 4 unidades, seguida de dos capas ocultas con 6 neuronas y una capa de salida con una única neurona.

En la figura 3.3 se presenta a modo de ejemplo el esquema de una red neuronal con 4 capas. Utilizamos una convención donde la primer capa de la red, la capa de entrada, representa el valor del vector de *features* de cada instancia y no está compuesta de neuronas artificiales. Por ende, no posee parámetros entrenables y el número de unidad es igual a la dimensión de

$\hat{x}$, en este caso 4. Las siguientes dos capas son lo que se llaman capas ocultas, y en este caso poseen 6 neuronas cada una. Por último, el número de neuronas en la capa de salida debe corresponderse con la dimensión de los vectores objetivos $\hat{y}$, en este ejemplo es 1. La función de activación elegida para esta última capa suele diferir de la activación para el resto de las neuronas y depende de la tarea para la cual la red neuronal es entrenada. Por ejemplo, en una tarea de regresión normalmente se utiliza una activación lineal $g(y) = y$, ya que esta función posee como imagen todos los reales.

Para una capa cualquiera de una red neuronal l los valores de salida (o activación) de la misma $A^{[l]}$ dependen únicamente del valor de sus parámetros y de los valores de salida de la capa anterior $A^{[l-1]}$. Los mismos se puede expresar como:

$$A^{[l]} = g^{[l]} \left(W^{[l]} A^{[l-1]} + b^{[l]} \right). \quad (3.2.2)$$

Notemos que, sin las funciones de activación no-lineales, una red neuronal *feedforward* sería simplemente una composición de funciones lineales, por ende, una función lineal.

Dada una determinada tarea sobre un conjunto de datos, dilucidar cual es la arquitectura de red adecuada para llevarla a cabo no es trivial, ya que no existe una forma analítica de estimarlo. Normalmente se suele copiar la arquitectura de alguna red que demostró funcionar adecuadamente en un problema similar o, simplemente, se entrenan varias redes con distintas arquitecturas y se elige la arquitectura que consigue un mejor desempeño. A toda red neuronal que posea 2 o mas capas ocultas se la denomina profunda (*deep*), y es por este motivo que se suele denominar como *Deep Learning* al área del *Machine Learning* que se ocupa de los modelos con una gran cantidad de parámetros, normalmente algún tipo de red neuronal con muchas capas [28].

3.2.3. Entrenamiento de una red

Como comentamos en la primer sección del capítulo, no existe un algoritmo analítico cerrado para computar los mejores parámetros de una red neuronal a partir de conjunto de datos y una tarea. Además, dada la cantidad de parámetros y de datos con la cual se suele trabajar en redes neuronales, sería muy difícil de implementar. La forma usual de entrenar redes neuronales es mediante un algoritmo iterativo, utilizando una estrategia de descenso por gradiente. En cada iteración se busca actualizar los parámetros de la red para que la misma se vuelva mejor en términos de la función de costo.

A continuación, describiremos brevemente el algoritmo de optimización de descenso por gradiente estocástico *mini-batch* (*mini-batch SGD*). Una explicación detallada del mismo está fuera del alcance de esta tesis, la misma puede encontrarse en [98]. El algoritmo de *mini-batch SGD* propone encontrar los parámetros óptimos de la red θ^* de forma iterativa. Vamos a partir de unos parámetros inicializados en θ_0 y aplicar una regla de actualización $U : \Theta \rightarrow \Theta$. Esta

regla mueve los pesos en una dirección opuesta al gradiente de la función de costo $\mathcal{L}$, el cual se computa sobre un grupo de m instancias B . B es un subconjunto aleatorio de instancias tomadas del conjunto de entrenamiento $(\bar{X}, \bar{Y})$ al que llamamos *batch*. El tamaño del paso dado en cada iteración es proporcional a un hiperparámetro α , llamado *learning rate*. La forma funcional de esta regla de actualización es:

$$\theta_{(t+1)} = U(\theta_t) = \theta_t - \alpha \cdot \nabla_{\theta} \left[\sum_{x_i, y_i \in B} [\mathcal{L}(y_i, f_{\theta}(x_i))] \right]. \quad (3.2.3)$$

La idea es que cuando $t \rightarrow \infty$ entonces $\theta_t \rightarrow \theta^*$. En general, el paisaje formado por la función de costo en función de los parámetros de la red no es convexo, y por lo tanto no se asegura una convergencia al mínimo global [102]. Pero, sin embargo, el algoritmo puede converger a mínimos locales que son suficientemente buenos. De hecho, en modelos profundos, los mínimos locales pueden resultar casi tan buenos como el mínimo global [103, 104]. Una ventaja de computar las actualizaciones por *batches* y no sobre todo el conjunto de entrenamiento es que de esta forma el costo de cada actualización no escala con el tamaño del conjunto de entrenamiento, permitiéndonos operar con una gran cantidad de datos. Al mismo tiempo, la estocasticidad agregada al computo del gradiente por trabajar con una cantidad pequeña de datos le permite escapar rápidamente de puntos silla en la función de costo y converger a mínimos locales buenos [105, 106]. Por otro lado, calcular el gradiente por *batches* en lugar de instancia a instancia (lo que se denomina *SGD*) resulta mucho mas eficiente, ya que permite explotar el poder de la vectorización y la paralelización en CPUs y GPUs [98].

En el caso de la redes neuronales *feedforward*, el gradiente de la función de costo se computa mediante un proceso llamado *backpropagation* [107]. Al conocer las derivadas de las funciones involucradas, y al encontrarse las neuronas en una configuración *feedforward*, es posible calcular la derivada de la función de costo respecto a cada uno de los parámetros de la red a través de la regla de la cadena. Se comienza computando la función de costo para un dado grupo de instancias, y luego se propaga el cálculo de las derivada desde las neuronas de salida hacia las capas anteriores, es por esto que recibe el nombre de *backpropagation*. Para una descripción detallada del proceso pueden referirse a [98] o a [29]. El hecho de que, a pesar de ser modelos con una enorme cantidad de parámetros, sea posible computar el gradiente de forma automática, es una de las principales virtudes de esta clase de modelos, ya que esto nos permite entrenarlos de forma eficiente mediante algoritmos de optimización como el descenso por gradiente. Al conjunto de técnicas utilizadas para calcular gradientes de funciones computacionalmente, como *backpropagation*, se lo denomina *automatic differentiation* [108]. Las dos librerías de programación mas utilizadas para implementar técnicas de *automatic differentiation* son *Pytorch* y *Tensorflow* [109, 110].

Para entrenar las redes neuronales a lo largo de este trabajo de tesis se utilizó un algoritmo de optimización llamado Adam, el cual es una variante de *mini-batch SGD* [111]. La idea del

algoritmo es similar, pero en lugar de tener un *learning rate* fijo y global, el valor del mismo es adaptado para cada parámetro utilizando el primer y segundo momento de valores anteriores del gradiente. Dado este esquema de entrenamiento, los hiperparámetros mas relevantes a definir para el proceso serán:

- Tamaño del *batch*: Es la cantidad de instancias m sobre la cual computamos la función de costo en cada actualización de los parámetros. Los valores típicos son 16, 32 y 64.
- Cantidad de *epochs*: Llamamos *epoch* al hecho de utilizar todas las instancias del conjunto de entrenamiento para actualizar el modelo. La cantidad de *epochs* es una medida de cuanto se entrenó la red. Si decimos que la red se entrenó k *epochs*, esto quiere decir que la cantidad de actualizaciones que sufrieron los parámetros es del orden de $k * (N/m)$, donde N es la cantidad de instancias en el conjunto de entrenamiento.
- Valor del *learning rate*: Modula el tamaño del paso en el algoritmo de descenso por gradiente.
- Valor del primer y segundo momento (β_1 y β_2): Controlan en cuanto influyen el primer y el segundo momento a la modulación del *learning rate* en el algoritmo Adam.

3.2.4. ¿Por qué redes neuronales profundas?

Desde un punto de vista teórico, solo se precisa una red con única capa oculta para poder modelar “casi” cualquier tipo de función objetivo F . Esto está garantizado por el teorema de aproximación universal, el cual establece que una red *feedforward* con capa de salida lineal y al menos una capa oculta con cualquier función de activación “aplastante” (como, por ejemplo, la función sigmoidea) puede aproximar cualquier función Borel-medible de un espacio de dimensión finita a otro con cualquier nivel de error deseado distinto de cero, siempre y cuando la red tenga suficientes unidades en la capa oculta [112, 113]. Pero, en la práctica, el número de neuronas necesarias para lograr una buena aproximación de funciones complejas utilizando este tipo de arquitectura es, en general, muy grande. En cambio, el uso de modelos profundos con varias capas ocultas puede reducir drásticamente la cantidad de unidades requeridas para representar la función deseada [114]. Al mismo tiempo, de manera empírica se ha demostrado que el error de generalización alcanzado por redes profundas en una gran variedad de tareas suele ser mas bajo que el alcanzado por modelos no profundos [29].

Al seleccionar un determinado modelo f_θ en un problema de *Machine Learning*, implícitamente estamos asumiendo un conjunto de creencias previas sobre el tipo de función objetivo F , nuestro sesgo. Elegir como modelo una red profunda expresa una creencia general de que la función objetivo debe involucrar la composición de varias funciones más simples. Desde el punto de vista del aprendizaje de representación (*representation learning*), esto significa que creemos que el problema de aprendizaje consiste en que cada capa de la red sea capaz de descubrir *features*

simples de la capa anterior, y que los *features* mas complejos de los datos serán obtenidos en las capas mas profundas a través de una consecutiva composición de *features* simples. Dado el buen desempeño de las redes neuronales profundas, este sesgo parece ser adecuado en muchos tipos de tareas para las cuales se utiliza *Machine Learning*.

En la actualidad, existe una gran esfuerzo por entender la manera en que las redes neuronales profundas aprenden a codificar información [51, 115–118], y también por estudiar el motivo por el cual los algoritmos de descenso por gradiente estocástico encuentran buenos mínimos de la función de costo [103–106, 119, 120]. Se cree que la cantidad de parámetros (*overparameterization*) juega un rol central en este proceso [121–123]. Pero, a pesar de estos avances, al momento de escribir esta tesis, no es posible explicar completamente desde la teoría por qué estos modelos son capaces de lograr un error de generalización tan bajo en una variedad de tareas tan diversas. El motivo del éxito de las redes neuronales profundas es aún un debate abierto.

3.3. *Deep Autoencoders*

3.3.1. Definición

El término *autoencoder* se refiere a una red neuronal *feedforward* que aprende a copiar a su salida la misma información que se presenta en su entrada [29, 124]. Cuando el número de capas ocultas en este tipo de redes es mayor que uno, se las denominan *Deep Autoencoders* o *Stacked Autoencoders*. En su interior, un *autoencoder* genera una codificación de los datos de entrada en cada una de sus capas ocultas. Existe una capa particular, que se suele posicionar en el medio de la red, a la cual nos referimos como el espacio latente. Podemos entonces pensar que un *autoencoder* está compuesto por dos partes: Un *encoder* $\hat{h} = e(\hat{x})$ que transforma los datos de entrada a la representación en el espacio latente $\hat{h}$, y un *decoder* $\hat{r} = d(\hat{h})$ que genera una reconstrucción de los datos a partir del estado en su espacio latente.

Lo que hace que este tipo de redes sea interesante es que el espacio latente posee una dimensionalidad mas baja que los datos de entrada. Es decir, la capa correspondiente a su espacio latente tiene un numero de neuronas menor que n , siendo los datos de entrada $\bar{x} \in \mathbb{R}^n$. De esta forma la red funciona como un cuello de botella el cual se entrena para minimizar la pérdida de información, provocando una compresión de la información de los datos originales. Por lo general, la dimensión del espacio latente se elige de manera tal de que tenga el menor número posible de unidades pero, al mismo tiempo, intentando mantener una diferencia aceptable (medida en términos de la función de costo) entre los datos de entrada $\bar{x}$ y su reconstrucción a la de salida de la red $\hat{r} = d(\hat{h}) = d(e(\hat{x}))$. En la figura 3.4 como ejemplo el esquema de un *Deep Autoencoder* de 7 capas y un espacio latente de dimensión 3.

Los *autoencoder* son considerados algoritmos de aprendizaje no supervisado, ya que la tarea

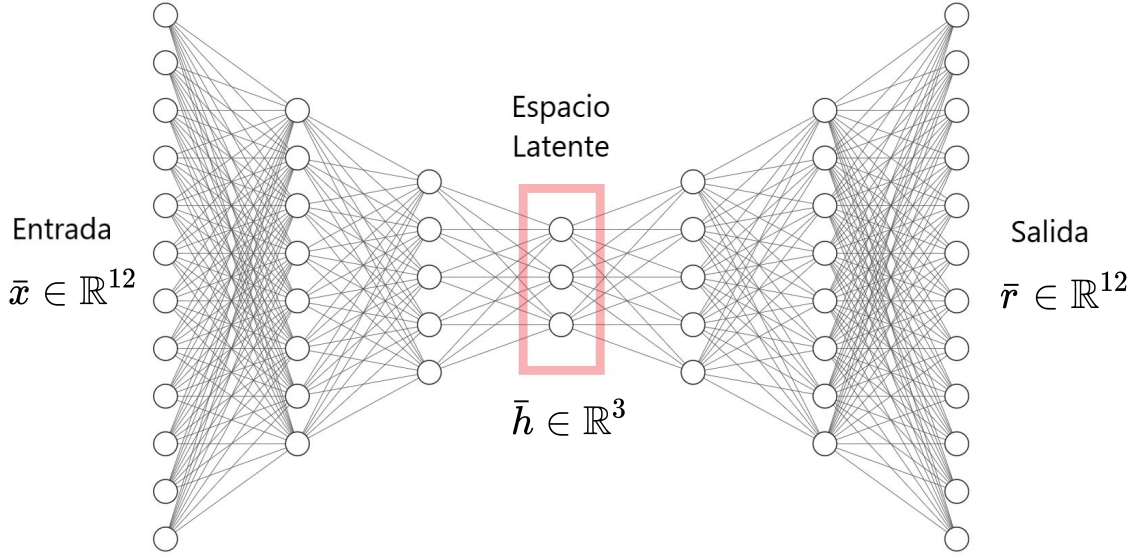


Figura 3.4: Esquema de un *Deep Autoencoder* con 7 capas. En este ejemplo los datos de entrada tienen dimensión 12, mientras que el espacio latente donde se generará la representación comprimida de los mismos tiene dimensión 3.

que llevan a cabo no pasa por encontrar la función que relaciona a cada instancia $\hat{x}$ un vector objetivo $\hat{y}$. Pero, en términos de implementación, los *autoencoders* no dejan de ser un caso particular de una red neuronal *feedforward*, por lo cual se entrenan utilizando las técnicas y consideraciones discutidas en el capítulo anterior. Existen distintas variantes de *autoencoders* y, si bien uso mas extendido es el de algoritmo de reducción de dimensionalidad o de aprendizaje de *features*, esta misma arquitectura puede ser utilizada también como algoritmo generativo o como algoritmo de reducción de ruido [125–129].

3.3.2. Aplicaciones a dinámica

Recientemente, distintas variantes de *Deep Autoencoders* se han utilizado para intentar descubrir las variables relevantes de sistemas dinámicos directamente a partir de datos [60, 130–136]. En estos trabajos se utilizan *Deep Autoencoders* para convertir datos de muy alta dimensión, como por ejemplo los cuadros de una película de fluidos, a una representación de dimensión más baja que preserve la información relevante. El objetivo en este plan de trabajo es luego usar la representación aprendida en el espacio latente como un proxy del espacio de fases del sistema. A diferencia de técnicas lineales como *Proper Orthogonal Decomposition* (POD) [63,64] o *Principal Component Analysis* (PCA). [65,66], las redes neuronales son capaces de aproximar funciones no lineales muy complejas, es por eso que los *Deep Autoencoders* tienen el potencial de proporcionar mejoras cualitativas en nuestra capacidad para extraer información dinámica directamente de datos crudos.

Pero sin embargo, nada garantiza que la representación aprendida por la red en el espacio latente guarde relación con las variables dinámicas originales del problema. Ni siquiera es posible garantizar que la representación de los datos en el espacio latente tenga las propiedades de un flujo dinámico. Además, no existe una manera sencilla de interpretar qué es lo que están codificando las variables del espacio latente de los datos de entrada. De hecho, debido a la aleatoriedad del *mini-batch SGD*, la representación aprendida varía entre entrenamiento a entrenamiento para una misma arquitectura de red sobre un mismo conjunto de datos. Evaluar la relación entre la codificación aprendida en el espacio latente a partir de los datos y el flujo dinámico del sistema original es uno de los objetivos de la presente tesis doctoral.

3.4. Redes neuronales recurrentes

3.4.1. Definición

Las redes neuronales recurrentes (RNNs) son redes diseñadas para lidiar con información secuencial. La característica principal de las RNNs es la de poseer variables de estado que almacenan información sobre el pasado de la secuencia. Estas variables de estado, las cuales se conocen como el estado oculto (*hidden state*) h de la red, se combinan con la entrada en cada paso t para determinar un nuevo estado oculto $h_t = f_h(h_{t-1}, x_t)$. La idea es que el estado oculto h funcione como una memoria persistente en donde se conserve la información relevante de los pasos anteriores de la secuencia y cuya lectura $o_t = f_o(h_t)$ permita realizar la tarea deseada, como por ejemplo la predicción del valor del futuro elemento de la secuencia. Como el estado oculto computado en el paso anterior se utiliza como entrada en el siguiente paso, las redes recurrentes se suelen representar como redes con un bucle (*loop*) de realimentación que reinyecta la salida a la entrada de la red (ver parte izquierda de la identidad en la figura 3.5). Las redes recurrentes han demostrado un excelente desempeño en tareas de predicción y clasificación de series temporales provenientes de áreas tan diversas como finanzas, biología, consumo energético o tráfico. [30–34].

En su forma más elemental, las ecuaciones que definen una red neuronal recurrente de una única celda son:

$$\begin{aligned} h_t &= \tanh(W_h [h_{t-1}, x_t] + b_h) \\ O_t &= g_O(W_O h_t + b_O). \end{aligned} \tag{3.4.1}$$

En esta expresión g_O es la función de activación elegida para la salida, mientras que las matrices W_h y W_O y los vectores b_h y b_O son los parámetros entrenables del modelo. Los corchetes refieren a una concatenación de vectores. Cada valor de entrada x_t puede ser un escalar o un vector dependiendo de los datos del problema, mientras que la dimensión del estado interno h será un hiperparámetro de la red a elegir. En configuraciones más complicadas se pueden “apilar” varias celdas, conectando la secuencia de salida de una celda a la entradas de

la próxima.

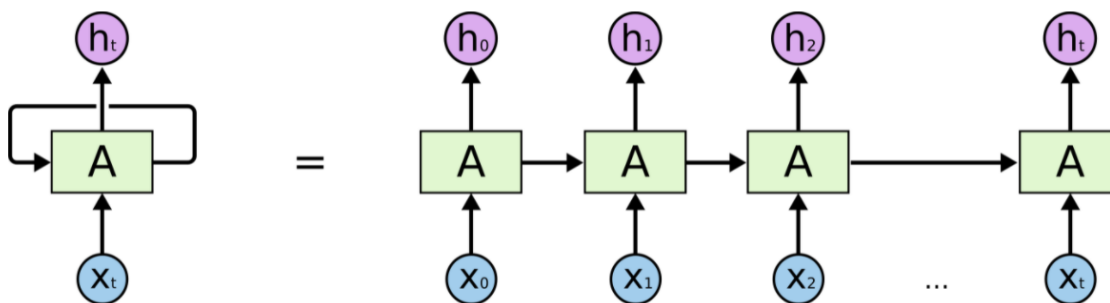


Figura 3.5: Esquema de una red neuronal recurrente. A la izquierda su representación como una red que posee un bucle de realimentación. A la derecha, la representación de la red en forma “desenrollada”.

Al tener un bucle de realimentación, este tipo de red parecería, en principio, fundamentalmente distinto a las redes *feedforward*. Pero, en realidad, uno puede pensar a la red recurrente como una red *feedforward* profunda en la cual todas las capas comparten los mismos pesos. Esta forma “desenrollada” de pensar la red puede observarse a la derecha de la identidad en la figura 3.5. En la misma, cada capa recibe como entrada tanto los valores de salida de la capa anterior h_{t-1} , como el valor de la secuencia en un dado tiempo x_t . Resulta entonces posible computar el gradiente de la función de costo respecto a los parámetros mediante el proceso de *backpropagation*, al cual en este contexto se lo llama *backpropagation* a través del tiempo, y entrenar la red recurrente con un algoritmo de descenso por gradiente estocástico *mini-batch* de la misma forma que se hace en una red *feedforward*.

En esta forma elemental, las redes recurrentes funcionan de manera correcta para detectar correlaciones de corto plazo en series de datos, pero sufren de un problema fundamental a la hora de ser entrenadas para detectar relaciones entre elementos que están separados por una cantidad grande de pasos. Este problema es conocido como el desvanecimiento del gradiente, e impide a la red aprender dependencias de largo plazo en los datos mediante descenso por gradiente [137, 138]. Con el fin de superar esta problemática se desarrollaron otras variantes de redes recurrentes, como la *Long Short Term Memory*.

3.4.2. *Long Short Term Memory*

De los diferentes tipos de redes recurrentes disponibles, las redes *Long Short Term Memory* (LSTM) son una de las más utilizadas y mas exitosas [35–38]. Las mismas fueron diseñadas para superar el problema del desvanecimiento del gradiente y aprender dependencias de largo plazo en los datos. A diferencia de las RNNs elementales, las celdas en las redes *Long Short Term Memory* poseen dos vectores de variables de estado, el espacio oculto h y el estado de

celda c . La idea es que la propagación del gradiente a través del estado de celda c le facilite a la red el aprendizaje de dependencias de largo plazo.

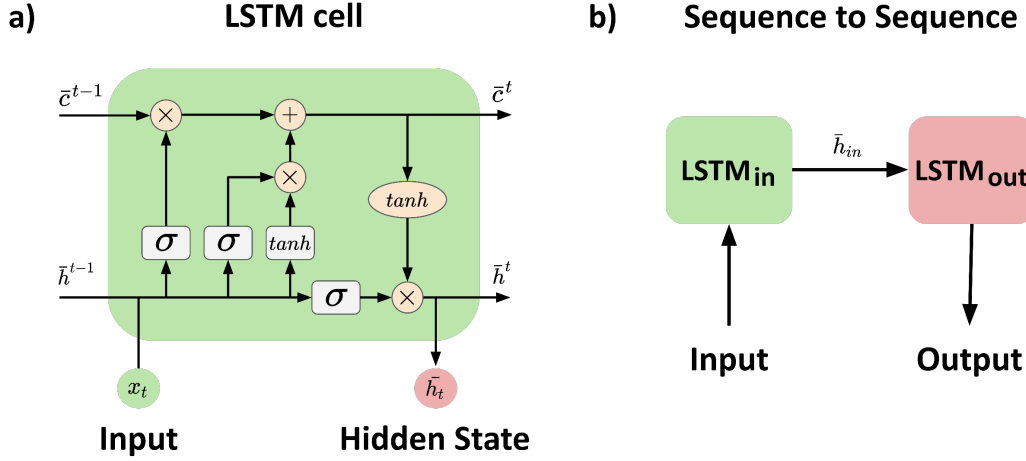


Figura 3.6: (a) Diagrama del flujo de los datos y las operaciones en una celda *Long Short Term Memory* (LSTM). (b) Esquema de una red recurrente en arquitectura *sequence-to-sequence*

La forma de computar el estado oculto h_t y el estado de celda c_t para un dado elemento t a partir del valor de entrada x_t y los valores internos anteriores h_{t-1} y c_{t-1} en una celda LSTM es la siguiente:

$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\
 \tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\
 c_t &= f_t * c_{t-1} + i_t * \tilde{c}_t \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\
 h_t &= o_t * \tanh(c_t) .
 \end{aligned} \tag{3.4.2}$$

En esta notación, $*$ representa una multiplicación elemento a elemento y σ la función sigmoidea. Las expresiones f , i y o suelen llamarse compuerta de olvido, compuerta de entrada y compuerta de salida respectivamente, pero son solo valores intermedios que sirven para computar h y c . Finalmente, la salida de la red suele resultar de una lectura sobre el estado oculto h :

$$O_t = g_O(W_O h_t + b_O) . \tag{3.4.3}$$

Dentro de las distintas arquitecturas en las cuales se utilizan celdas LSTM, la arquitectura *sequence-to-sequence* (secuencia a secuencia) representa un enfoque adecuado para muchas tareas secuenciales [139–146]. En este tipo de arquitectura, una celda de entrada LSTM_{in} lee y codifica la secuencia de entrada en un vector de estado, h_{in} , este vector luego se pasa a una

celda de salida $LSTM_{out}$ la cual produce secuencialmente las salida deseada. Un diagrama de este tipo de arquitectura se presenta en la figura 3.6(b).

3.4.3. Aplicaciones a dinámica

En los últimos años, un número creciente de estudios han investigado el poder predictivo de las redes neuronales recurrentes cuando se entrenan para modelar series de tiempo provenientes de sistemas dinámicos. A continuación haremos una breve recapitulación de algunos de los trabajos mas relevantes en esta temática.

En [38], [147], [39] y [42] los autores investigaron las propiedades a largo plazo de las series producidas por redes LSTM luego de ser entrenadas para predecir series temporales de un sistemas dinámicos. Para su evaluación, estudiaron el error de predicción y las características estocásticas de las señales generadas por la red. En [148], los autores evaluaron las predicciones realizadas por redes LSTM cuando los estados del sistema que se buscan predecir se encuentran levemente por fuera de aquellos que se usaron durante el entrenamiento. Para esto, estudiaron las propiedades dinámicas de las series predichas por la red, como la presencia de bifurcaciones y la estimación de exponentes característicos. En [149], [41] y [150] los autores compararon el rendimiento de diferentes tipos de arquitecturas (*feedforward*, RNN, LSTM y GRU) en la predicción de distintos sistemas con dinámica no lineal.

Estos trabajos ponen en evidencia la capacidad que poseen los algoritmos de redes neuronales recurrentes para capturar dinámicas no lineales complejas a partir de series temporales. Pero todos ellos se enfocan en evaluar la predicciones realizadas por el algoritmo, sin indagar en cómo es que la red logra generar una representación de la dinámica del sistema a partir de la serie. Uno de los objetivos del presente trabajo de tesis es investigar como se codifica la información dinámica de las series en los estados internos de este tipo de redes neuronales.

Capítulo 4

Codificación en Autoencoders

“ Sospecho, sin embargo, que no era muy capaz de pensar. Pensar es olvidar diferencias, es generalizar, abstraer. En el abarrotado mundo de Funes no había sino detalles, casi inmediatos. ”

Jorge Luis Borges, *Ficciones*, 1944

En este capítulo presentaremos y analizaremos una metodología para la reconstrucción de flujos dinámicos a partir de series temporales mediante *autoencoders*, una arquitectura de red neuronal ampliamente utilizada para la reducción de dimensionalidad en el campo del Aprendizaje Automático. Mostraremos que, cuando su entrenamiento resulta exitoso, este tipo de red neuronal es capaz de aprender un *embedding* adecuado de datos provenientes de un sistema dinámico caótico [69].

4.1. Introducción

En el capítulo anterior se mencionaron ejemplos de trabajos donde algoritmos de *autoencoders* fueron utilizados con el fin de comprimir datos de entrada de alta dimensionalidad provenientes de sistemas dinámicos, con la esperanza de encontrar en el espacio latente las variables relevantes del problema, aunque nada garantiza que la información codificada en este espacio reducido sea de alguna manera equivalente al espacio de fases original del sistema. En este esquema de trabajo, se asume que cada uno de los elementos en los datos de entrada posee información de todas las variables dinámicas. Por ejemplo, se asume que en cada uno de los cuadros de la película se tiene de alguna manera codificado el estado de todas las variables involucradas en la dinámica del sistema. Pero este no es el caso general, muchas veces la medición

que realizamos de un sistema únicamente captura la evolución de una o algunas de las variables dinámicas.

En este capítulo usaremos los *autoencoders* para una tarea levemente distinta: reconstruir en su espacio latente una representación multivalueada a partir de la serie temporal de una única variable de un sistema dinámico. Vamos a poner a prueba, mediante experimentos numéricos, la hipótesis de que esa representación es equivalente a la que se obtiene por medio de un *embedding*. Para hacerlo, desarrollaremos una metodología capaz de evaluar numéricamente si la representación de la serie temporal en el espacio latente del *autoencoder* posee una topología que puede considerarse equivalente a la del flujo dinámico original del cual proviene la serie temporal.

Las series temporales que utilizaremos para realizar esta evaluación serán series provenientes de atractores caóticos, cuya particular estructura en el espacio de fases nos ayudará a comparar la representación en el espacio latente del *autoencoder* con el flujo original del sistema. Como análisis adicional, también se comparará el exponente maximal de Lyapunov del atractor en el sistema original con el exponente de los atractores reconstruidos por el *autoencoder* en el espacio latente a partir de la serie temporal. Por último se evaluará la susceptibilidad de esta metodología de reconstrucción de flujo con *autoencoders* ante la adhesión de ruido a las series temporales.

Es importante destacar que, dentro del programa de descubrir ecuaciones a partir de datos, la condición de reconstruir un flujo topológicamente equivalente al original es totalmente necesaria ya que dos flujos con topologías distintas describen dinámicas cualitativamente distintas.

4.2. Metodología

Dada una serie temporal de N elementos $\{x_1, x_2, \dots, x_N\}$, entrenaremos un *autoencoder* en la tarea de reducir la dimensionalidad de vectores de N_1 elementos formados a partir de la serie de la siguiente forma:

$$\{x_1, x_2, \dots, x_{N_1}\}, \{x_2, x_3, \dots, x_{N_1+1}\}, \dots, \{x_{N-N_1}, x_{N-N_1+1}, \dots, x_N\} \quad (4.2.1)$$

Si N_1 es lo suficientemente grande, sabemos que se cumplen las condiciones de los teoremas de *embedding* y el determinismo queda asegurado, es decir, cada segmento se corresponde únicamente con un estado posible del sistema. Tomaremos entonces el espacio multivalueado generado por el valor de la activación de las N_k neuronas en el espacio latente, $Y \in \mathbb{R}^{N_k}$, como nuestro espacio de *embedding*. Es decir, a cada segmento de la serie lo asociaremos a un punto de $\mathbb{R}^{N_k}$:

$$(x_1^i, x_2^i, \dots, x_{N_1}^i) \rightarrow Y^i = (y_1^i, y_2^i, \dots, y_{N_k}^i), \quad i = 1, \dots, N - N_1 \quad (4.2.2)$$

La dimensión del espacio de *embedding*, N_k , se computará a partir de inspeccionar el error

de reconstrucción cometido por el *autoencoder*. Se tomara como dimensión adecuada aquella a partir de la cual, al aumentar la dimensionalidad, el error ya no se reduzca significativamente. Es decir, se busca la mínima dimensión para la cual el *autoencoder* realiza una buena tarea de reconstrucción.

En las siguientes subsecciones se detallaran las metodologías utilizadas en nuestros experimentos numéricos para generar las series temporales, implementar las redes neuronales y calcular el *linking number* entre pares de orbitas periódicas.

4.2.1. Generación del Dataset

Para generar nuestro conjunto de datos, integramos un sistema de ecuaciones de Rössler,

$$\begin{aligned}\frac{dx}{dt} &= -y - z \\ \frac{dy}{dt} &= x + ay \\ \frac{dz}{dt} &= b + z(x - c)\end{aligned}\tag{4.2.3}$$

con $a = b = 0,1$ y $c = 14$ durante 15000 unidades temporales, generando una serie de datos de valores escalares correspondientes a valores consecutivos de la variable x , en la figura 4.1(a) se muestra parte de esta solución. Dividimos la serie de tiempo en segmentos de $N_1 = 256$ puntos, correspondientes a 10,26 unidades de integración, que fueron tomados de la serie de la forma:

$$\{x_1, x_2, \dots, x_{N_1}\}, \{x_2, x_3, \dots, x_{N_1+1}\}, \dots \{x_{N-N_1}, x_{N-N_1+1}, \dots, x_N\}.\tag{4.2.4}$$

Uno de estos segmentos puede observarse en la Figura 4.1(b). La ventana de tiempo considerada resulta aproximadamente $4/3$ de la duración de la recurrencia característica más corta, o sea, la órbita de periodo 1. Este proceso resultó en 374488 elementos que usamos para entrenar y probar un codificador automático. El primer setenta por ciento de los datos se utilizó para entrenar la red y el último treinta por ciento se utilizó para probarla.

Notemos que cada uno de los segmentos considerados cumple sobradamente la hipótesis del teorema de Takens, ya que $N_1 \gg 2m+1$. Esto quiere decir que contiene información redundante como para determinar el estado de sistema. Lo mismo hubiese sucedido si N_1 hubiese tomado el valor 200 o 300. La metodología es robusta frente a la elección de este parámetro, solo es necesario seleccionar un valor que exceda ampliamente la dimensionalidad esperada para el espacio de *embedding*. Esto contrasta con la metodología de *delays* temporales de Takens, donde la selección del parámetro τ es crítica.

4.2.2. Implementación de la Red Neuronal

La red se implementó en la versión 2.3.1 de Keras, utilizando Tensorflow 1.15.2 como *backend*. Se utilizó una activación de tipo *ReLU* en todas las neuronas [151], excepto las neuronas

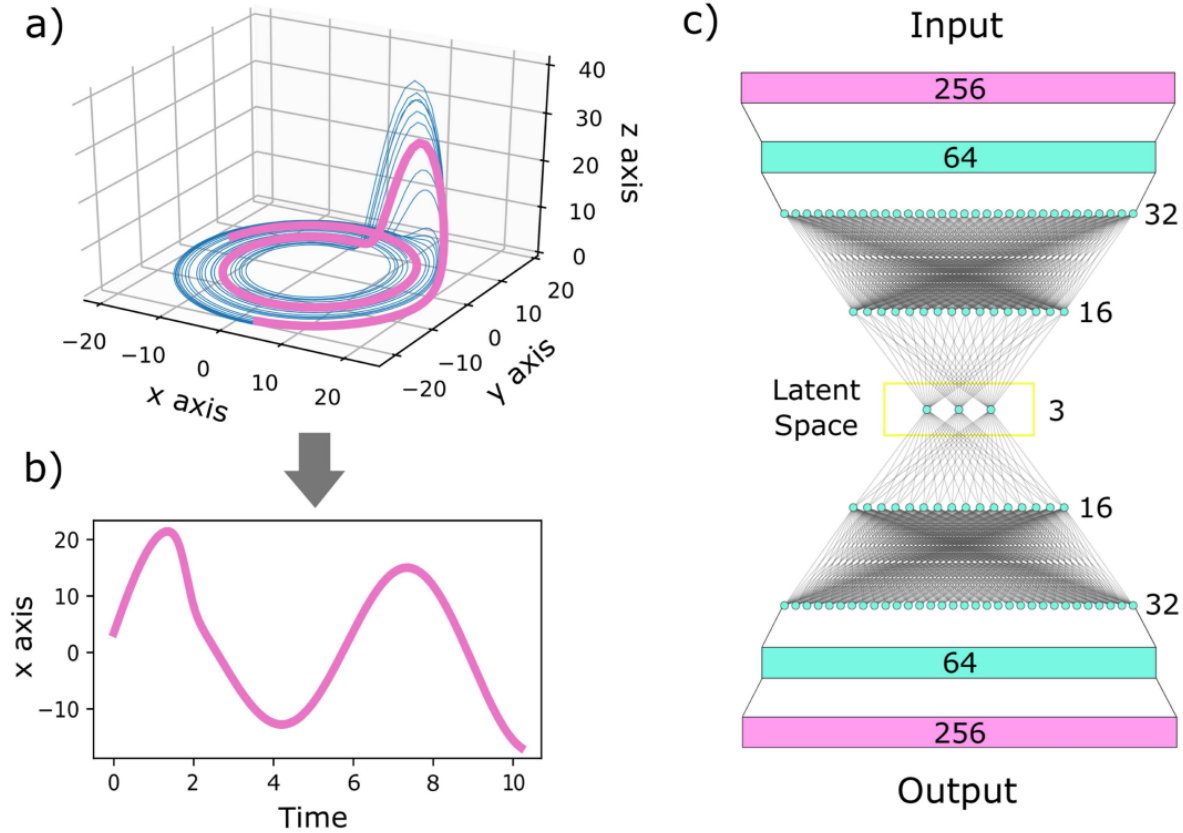


Figura 4.1: Método de *embedding* con *autoencoder*. (a) Simulación de un atractor extraño utilizando el sistema dinámico Rössler. (b) Se usaron fragmentos de 256 puntos para entrenar la red neural. (c) La arquitectura del *autoencoder* con 9 capas de neuronas, el número indica la cantidad de unidades en cada capa. La función de costo que se minimiza es el error cuadrático medio entre las capas de entrada (input) y de salida (output).

de la capa media en las cuales se utilizó una activación lineal. Como función de costo se eligió el Error Cuadrático Medio (MSE), la red se entrenó para minimizar este error entre las capas de entrada (input) y de salida (output). El algoritmo de optimización utilizado fue Adam, con tasa de aprendizaje $lr = 0,001$, $\beta_1 = 0,9$ y $\beta_2 = 0,999$ [111]. El tamaño de *batch* fue de 32 muestras y se utilizó un inicializador *Glorot* uniforme para la inicialización de los pesos de las neuronas [152]. Nótese que en cada uno de los modelos entrenados los pesos iniciales de las redes serán distintos. Los *autoencoders* fueron entrenados durante 91 *epochs*. Este criterio de finalización del entrenamiento fue elegido al ver que, para este valor de *epochs*, en la mayoría de los modelos, el cambio porcentual promedio del MSE en el dataset de testeo era menor al 1 %. No se realizó una búsqueda sistemática en el espacio de hiperparámetros para optimizar el MSE sobre el dataset de testeo.

4.2.3. Cálculo del *linking number*

Un elemento central en nuestro programa de trabajo es la posibilidad de comparar la estructura topológica de dos atractores extraños de una manera sistemática y categórica a través del ordenamiento de las órbitas periódicas inestables que conviven con ellos. Para esto precisaremos calcular los *linking numbers* entre esas órbitas, el invariante topológico que describe su ordenamiento.

La definición de *linking number* y su forma integral de calcularlo fue presentada en la sección 2.2.5. Pero en este trabajo utilizaremos una forma algorítmica mas sencilla para computarlo, la cual se detalla a continuación.

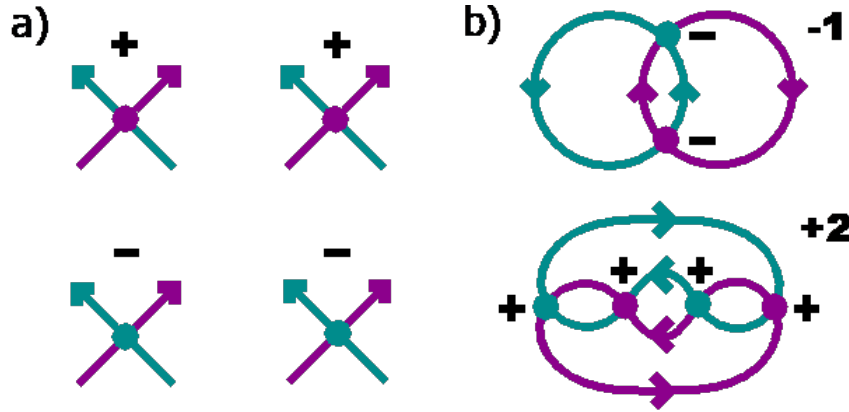


Figura 4.2: Compueto del *linking number* entre dos curvas orientadas en $\mathbb{R}^3$. (a) Los cuatro tipos posibles de intersecciones en las curvas proyectadas al plano bidimensional. El color del círculo en la intersección indica cuál es la curva superior, mientras que el signo indica si la intersección contribuye como $+1/2$ o $-1/2$ al *linking number*. (b) Ejemplo del compueto en dos pares de órbitas una con *linking number* igual a -1 y la otra a 2 .

Dadas dos órbitas orientadas en un espacio tridimensional el *linking number* entre las mismas se puede calcular algorítmicamente a partir de una proyección de ambas orbitas a un plano bidimensional en donde se preserva la información de los cruces [15, 73]. En cada punto de cruce entre las curvas, se dibujan vectores tangentes a los segmentos de curva superior e inferior en la dirección del flujo. El cruce se etiqueta $+$ ($-$) si los vectores tangentes forman un sistema de coordenadas de mano derecha (izquierda) en el plano de proyección. El ejemplo de los cuatro tipos posibles de cruces se presenta en la figura 4.2. Luego, el *linking number* entre estas dos órbitas es el número entero resultante de la suma de todos los cruces, donde cada cruce contribuye con $+1/2$ o $-1/2$, según su signo. Recordemos que el valor de este invariante es entero, los cruces siempre vienen de a pares.

Con el fin de poder computar el 4.2 de forma sistemática para órbitas cerradas arbitrarias, se desarrolló una función de Python. Su código se encuentra disponible en el siguiente repositorio público: github.com/gon-uri/linking_number. La función recibe como entrada las órbitas de

manera numérica, es decir, como vectores de puntos de $\mathbb{R}^3$ ordenados según la evolución del flujo. En teoría, cualquier plano de proyección elegido debería funcionar bien. Pero en la práctica hay algunos planos donde las curvas están mas expandidas en el espacio. Esto lleva a que la cantidad de cruces sea menor y que los mismo estén mas distanciados entre si, haciendo menos probables los errores numéricos. Para seleccionar de manera automática un buen plano de proyección, donde las órbitas estén mas expandidas, nuestra función selecciona el plano cartesiano en el cual los puntos correspondientes a las órbitas poseen la mayor desviación absoluta mediana (MAD) [153]. Para mas detalles sobre el funcionamiento, referirse al [repositorio](#).

4.3. Resultados

4.3.1. Estimación de la Dimensionalidad

En este caso particular, al trabajar con un dataset generado de manera sintética, sabemos a priori que el número mínimo de dimensiones en las que se puede inscribir el flujo correspondiente al sistema dinámico subyacente es $N_k = 3$. En una situación general, se puede estimar la dimensión del espacio latente (la dimensión del *embedding*) mediante el estudio del error de reconstrucción. Esperamos que este error baje significativamente cuando la dimensión N_k sea lo suficientemente grande como hacer un mapeo uno a uno entre los estados del sistema y los valores en el espacio latente.

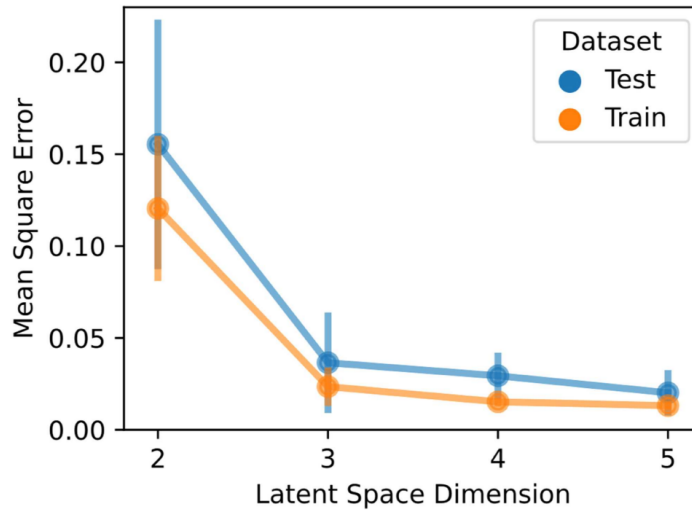


Figura 4.3: Grafico del error de reconstrucción en función del numero de unidades en el espacio latente. La dimensionalidad fue elegida como la mínima que garantiza una ganancia cualitativa en términos de reducción de MSE. Con un espacio latente de dos unidades, el MSE es un orden de magnitud mayor que con 3 o más.

Por ejemplo, en nuestro caso, la evaluación de redes con N_k igual a 2, 3, 4 y 5 muestra que se produce una mejora cualitativa cuando usamos tres unidades o más: el MSE para dos unidades

es un orden de magnitud mayor que el MSE obtenido con tres unidades. En otras palabras, dos unidades en la capa intermedia no permiten una reconstrucción adecuada del flujo, ya que la pérdida de información es demasiado grande. Los resultados se presentan en la figura 4.3.

4.3.2. Evaluación de la topología

De las simulaciones numéricas de nuestro sistema dinámico original, destacamos tres segmentos que eran buenas aproximaciones de las órbitas periódicas inestables que coexistían con la solución del sistema. En la figura 4.4 se presenta el flujo y la aproximación de sus orbitas periódicas. Para hacerlo definimos una sección de Poincaré en $y = 0$ y buscamos un segmento del flujo original donde la distancia entre dos intersecciones consecutivas del flujo con la sección era mínima. Esta es la aproximación de la órbita de período 1, la cual llamaremos solución P1. Luego, repetimos el procedimiento para una órbita que cruza la sección dos veces y recién en la segunda vuelta la distancia entre los puntos de intersección al plano es mínima (período 2, solución P2). Lo mismo con una órbita que cruza tres veces (período 3, solución P3). Notemos que es una de las dos soluciones del período tres que se pueden encontrar en el sistema de Rössler.

Calculamos numéricamente los *linking numbers* entre estas aproximaciones de órbitas periódicas en el sistema de Rössler utilizando la función descripta en los métodos. El resultado obtenido fue: $ln(P1,P2) = -1$, $ln(P1,P3) = -1$, $ln(P2,P3) = -2$. En este trabajo, caracterizamos la topología del atractor mediante esta terna de *linking numbers* entre órbitas de periodo bajo, las cuales podemos aproximar de manera precisa. Aunque no es necesario computar el ordenamiento entre todos los pares de orbitas para caracterizar un atractor (ver sección 2.2.5), en términos estrictos, estos tres valores no definen inequívocamente al atractor. Pero, sin embargo, resultan útiles para distinguirlo categóricamente de un flujo con topología distinta. Por ejemplo, un atractor de Lorenz presenta $ln(P1,P2) = 0$ [73], mientras que un atractor de Lü presenta $ln(P2,P3) = 1$ [154].

Luego, para evaluar si la representación realizada por los *autoencoders* es equivalente a un *embedding*, comparamos esta organización de los segmentos en el espacio de fase original, con la organización de los segmentos en el espacio latente (es decir, el entorno multivaluado donde las coordenadas son las actividades de las neuronas en la capa media del *autoencoder*). En la figura 4.4, mostramos el flujo reconstruido en el espacio latente para toda la serie de tiempo en una de las sesenta redes entrenadas, y en la misma figura también mostramos la reconstrucción de los tres segmentos que se aproximan a las soluciones periódicas. En este ejemplo, el entrenamiento del *autoencoder* conduce a un *embedding*: las órbitas periódicas en el espacio latente tienen una organización topológica idéntica a la de las órbitas originales.

Nos interesa explorar cómo la red neuronal logra realizar un *embedding* de la serie temporal. Con este fin, entrenamos sesenta *autoencoders* en la tarea de reconstrucción. Para cada uno de

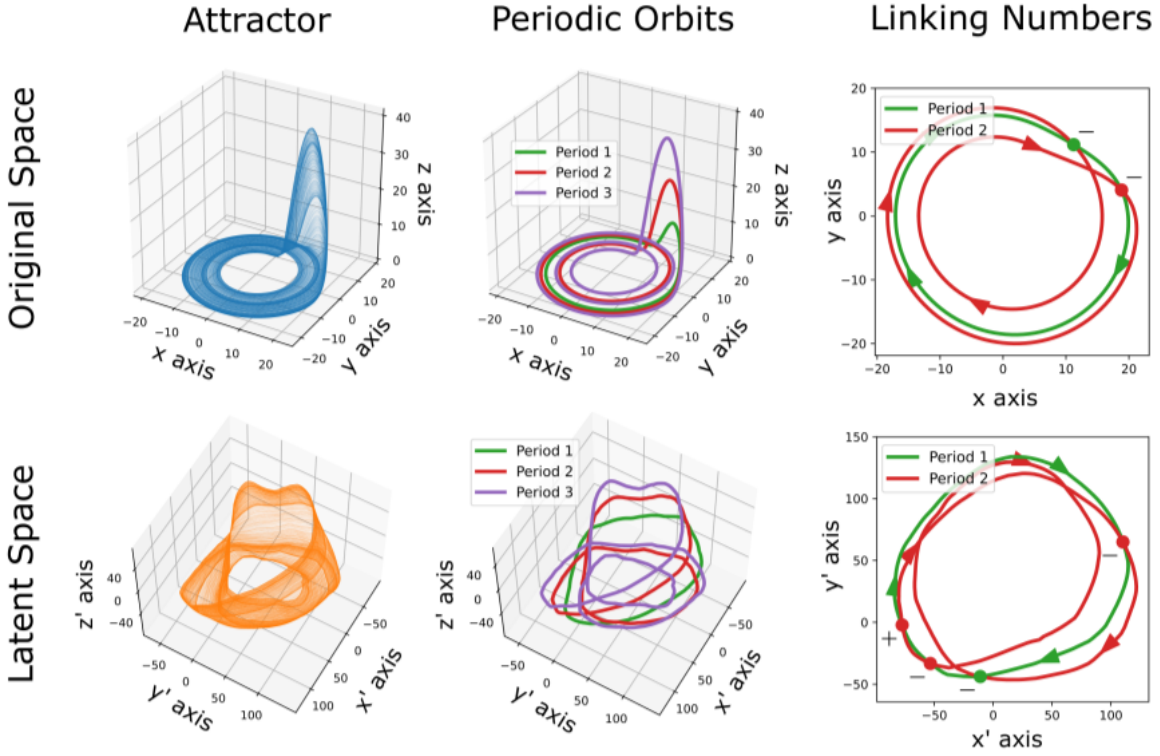


Figura 4.4: Comparación entre el flujo original y el flujo reconstruido con un *autoencoder*. En la primer columna, se muestra el atractor original y el reconstruido. En la segunda, se presentan las aproximaciones de las orbitas periódicas. En la tercera, se muestra el calculo del *linking number* entre las órbitas de periodo 1 y periodo 2. En este ejemplo, tanto para el atractor original como para el reconstruido $ln(P1,P2) = -1$.

estos, inspeccionamos la estructura topológica de las órbitas reconstruidas en el espacio latente en diferentes épocas durante el proceso de entrenamiento. El gráfico de esta evaluación puede verse en la figura 4.5(a). Luego de un único *epoch* de entrenamiento, los sesenta *autoencoders* dieron lugar a topologías incorrectas en el espacio latente. Durante los siguientes *epochs* de entrenamiento, el MSE de los modelos fue típicamente disminuyendo y, al mismo tiempo, el porcentaje de redes que conducen a una topología correcta fue aumentando. Después de noventa *epochs*, 53 modelos condujeron a una organización topológica correcta entre las órbitas reconstruidas, mientras que 7 no. Para asegurarnos de que no estábamos cortando el entrenamiento demasiado pronto, continuamos entrenando modelos con topología incorrecta durante 90 épocas más. Ninguno de ellos terminó en una topología correcta al final del proceso. En la figura 4.5(b) mostramos la evolución del MSE durante el procedimiento de entrenamiento para los 60 modelos. Diferenciamos para cada *epoch* en este gráfico entre los modelos que llevan a una estructura topológica correcta y los modelos que no. Observe que el MSE típicamente alcanza una meseta después de noventa épocas.

En la Figura 4.5(c) se presenta como ejemplo el detalle del proceso de entrenamiento para

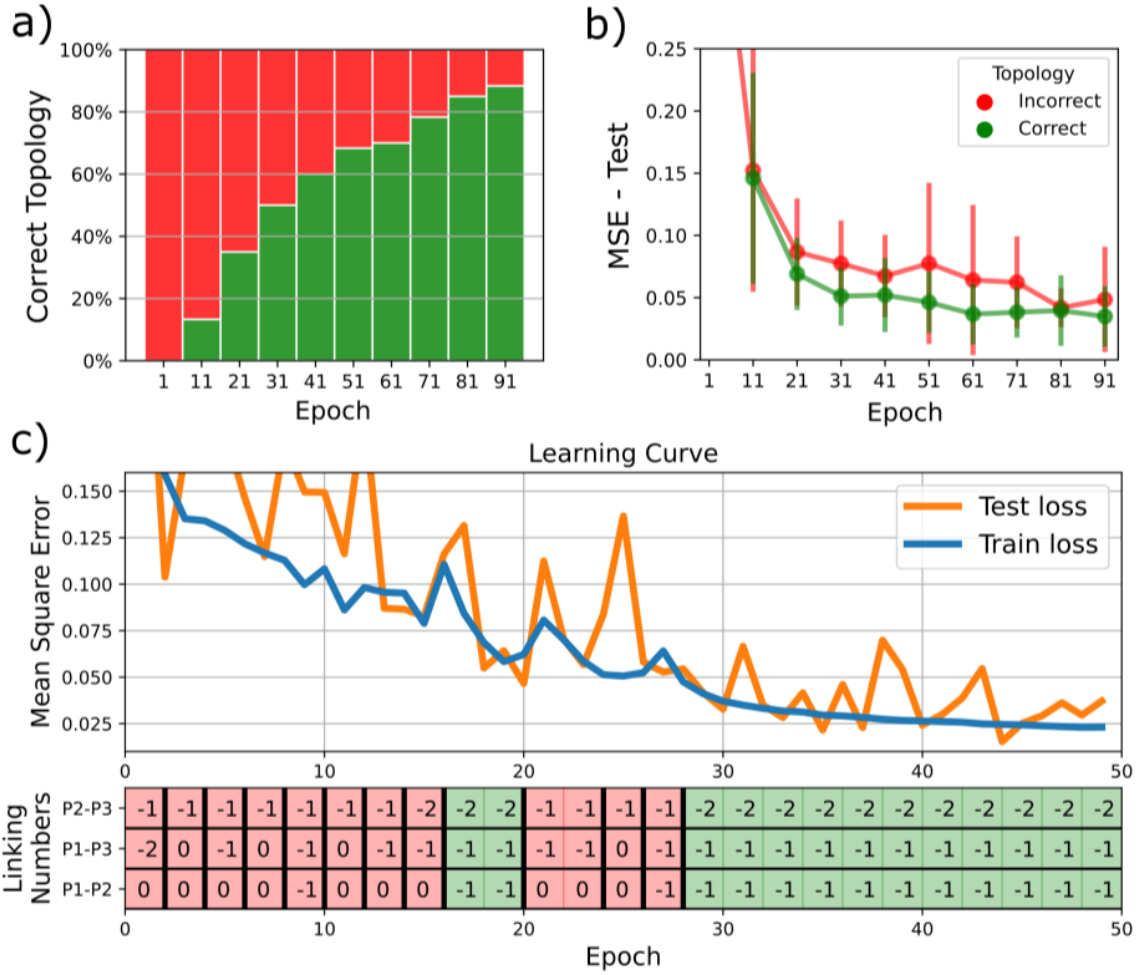


Figura 4.5: Resultado del entrenamiento para los 60 *autoencoders*. (a) Proporción de atractores reconstruidos con organización topológica correcta para distintos *epochs*. (b) Evolución de MSE durante el entrenamiento discriminada según la estructura topológica del flujo reconstruido. Los puntos indican el MSE medio y su desvío estándar. (c) Ejemplo del proceso de entrenamiento para uno de los modelos. En el panel de arriba mostramos la evolución del MSE para los dataset de entrenamiento y testeo. En el panel de abajo se presentan los *linking numbers* entre las orbitas periódicas del flujo reconstruido. Las celdas verdes corresponden a índices topológicos equivalentes a los del flujo original.

uno de los modelos entrenados. En esta figura se grafican únicamente los primeros 50 *epochs* de entrenamiento. Además de la evolución del error en los dataset de entrenamiento y testeo, se presenta el valor de los *linking numbers* entre las órbitas periódicas reconstruidas cada 2 *epochs*. En este experimento numérico en particular, el *autoencoder* conduce a una reconstrucción adecuada de la topología luego de veintinueve *epochs* y se mantiene así hasta el final del entrenamiento.

Es notable que la mayoría de los *autoencoders* proporcionen la organización topológica correcta entre las órbitas reconstruidas. Pero también es importante notar que más del 10 %

de ellos no lo hacen, incluso después de que su MSE haya alcanzado una meseta, un punto de referencia comúnmente utilizado para finalizar el procedimiento de entrenamiento. Además, los modelos que no constituyen una integración adecuada pueden presentar un MSE más bajo que el de otros que conducen a uno correcto, algo que se puede observar en las barras de error de la figura 4.5(b). Es decir, no identificamos en nuestros experimentos un valor crítico del MSE tal que por debajo de él se asegura una reconstrucción correcta.

4.3.3. Evaluación de intersecciones

Es interesante analizar qué sucede en las reconstrucciones que conducen a una topología incorrecta. Con este fin, exploramos si los vectores que describen el flujo en una pequeña región del espacio reconstruido eran en su mayoría paralelos. De no ser así, esto podría indicar la existencia de intersecciones en las trayectorias del espacio de fases reconstruido [155].

Dado un punto en una trayectoria del espacio reconstruido, se computaron las 3 trayectorias vecinas más cercanas. Luego se calculó la media del coseno del ángulo formado por las direcciones del flujo entre la trayectoria original y estas tres trayectorias. Este proceso se repitió para 25000 puntos del conjunto de test en cada uno de los atractores reconstruidos.

Para la mayoría de los puntos, este valor está muy cerca de 1, lo que indica direcciones casi paralelas (es decir, los puntos cercanos comparten futuros similares). Pero, si miramos los 50 puntos con los valores más bajos en cada *autoencoder*, hay una diferencia significativa entre aquellos con topología correcta, 0,93(0,88 – 0,96), y aquellos con una incorrecta 0,79(0,46 – 0,87). Los valores se expresan como mediana (rango intercuartílico). Este resultado sugiere que hay regiones en el espacio latente donde los *autoencoder* que conducen a topologías incorrectas presentan auto-intersecciones en las trayectorias reconstruidas, ya que los puntos en una pequeña región del espacio de fases muestran futuros cualitativamente diferentes [155].

Notemos que, teóricamente, el hecho de tener trayectorias que se cruzan en el espacio de fases debería impactar negativamente en el error de reconstrucción, ya que dos puntos correspondientes a series distintas están siendo representados en el mismo punto del espacio latente. Pero basta con una única región del espacio donde esto suceda para que la topología global del atractor sea incorrecta. De hecho, la cantidad de pares de puntos en los cuales esto parece estar sucediendo en los modelos con topología incorrecta es muy baja (menos de 50 en 25000). Por ende, proporcionalmente, esto aporta una diferencia muy pequeña en términos del MSE.

4.3.4. Evaluación del Exponente de Lyapunov

Los atractores extraños también se pueden caracterizar por los exponentes de Lyapunov, que describen la sensibilidad del flujo a diferentes condiciones iniciales y permiten reconstruir propiedades métricas como la dimensión del flujo (ver sección 2.2.2). En la tabla 4.3.1 presen-

Parámetros			Número de Autoencoders		1º Exponente de Lyapunov		
a	b	c	Topología Correcta	Topología Incorrecta	Sistema Original	Espacio Latente	
						Topología Correcta	Topología Incorrecta
0.1	0.1	14	53	7	0.072	0.076 (0.072 - 0.094)	0.09 (0.081 - 0.208)
0.12	0.12	9	17	3	0.068	0.069 (0.067 - 0.084)	0.074 (0.073 - 0.087)
0.11	0.11	13	16	4	0.086	0.093 (0.088 - 0.0121)	0.248 (0.22 - 0.302)

Cuadro 4.3.1: Valor del primer exponente de Lyapunov para tres sistemas de Rössler con parámetros distintos. Se presenta el valor del mismo computado tanto en el sistema original como en las diferentes reconstrucciones generadas en el espacio latente, cuyos valores se segregan según si el ordenamiento topológico de las órbitas es correcto o no. Los resultados se expresan como mediana (rango intercuartílico).

tamos los valores del primer exponente de Lyapunov para el atractor de Rössler en el espacio original y en el espacio latente. Para estimar los exponentes de Lyapunov utilizamos la metodología numérica propuesta por Eckmann y Ruelle [83, 156]. Implementamos un algoritmo en Python, tomando como punto de partida en código de la biblioteca *Nonlinear Measures for Dynamical Systems* (nolds) [157].

Adicionalmente, entrenamos 20 *autoencoders* con datos provenientes de otros dos sistemas de ecuaciones, los cuales corresponden a soluciones caóticas del sistema de Rössler pero con distintos valores de los parámetros a, b y c. Los resultados también se muestran en la tabla 4.3.1. A diferencia de los *linking numbers*, los valores de los exponentes de Lyapunov cambian para los diferentes parámetros del sistema. Los resultados obtenidos muestran que los atractores reconstruidos con topología correcta presentan valores cercanos al calculado para el atractor original, mientras que aquellos con organización topológica incorrecta presentan valores significativamente más altos.

4.3.5. Robustez Frente a Ruido

En vistas de esta nueva metodología, resulta relevante analizar si el éxito en la realización del *embedding* se ve afectado por la presencia de ruido en las series temporales. Para explorar esta temática decidimos volver a entrenar 20 modelos siguiendo el mismo procedimiento detallado en los métodos, pero esta vez agregando ruido a la serie temporal. Antes de la etapa de entrenamiento le sumamos a cada punto de la serie un valor aleatorio tomado de una dis-

tribución gaussiana, $\mathcal{N}(\mu, \sigma^2)$, con media cero, $\mu = 0$, y un desvío estándar, σ , equivalente al 5 % del desvío estándar de la serie temporal completa. Es decir que utilizamos un ruido blanco gaussiano aditivo del 5 %. El proceso se repitió para cada modelo entrenado, generando 20 series ruidosas distintas. Luego, analizamos la topología de los atractores reconstruidos en el espacio latente a partir de estas series de la misma manera que lo hicimos para las series sin ruido. Encontramos que 17 autocodificadores (un 85 %) dieron lugar a la topología correcta, una relación muy similar a la obtenida con datos de series temporales no ruidosas.

Si bien el tema precisa mas investigación, este resultado sugiere que la metodología de reconstrucción del espacio de fases con *autoencoders* es robusta ante la suma de ruido estocástico a la señal. Nótese además que al reconstruir un segmento de la serie temporal ruidosa con un *autoencoder* debidamente entrenado, el resultado será un segmento temporal el cual resulta una buena aproximación del segmento sin ruido. De hecho, los *autoencoders* son comúnmente utilizados como mecanismo de *denoising*: al tener que comprimir la información relevante en su espacio latente, el modelo aprende a ignorar particularidades de cada instancia y se focaliza en codificar features generales que le permitan explicar la mayor cantidad de varianza en todo el dataset de entrenamiento.

4.4. Discusiones

En este capítulo presentamos y evaluamos una metodología por la cual es posible realizar *embeddings* de series temporales utilizando *autoencoders*. Mostramos que la probabilidad de tener un *embedding* correcto en el espacio latente crece a medida que la red se entrena en la tarea de comprimir la información presente en segmentos de la serie temporal. Esto significa que la reducción del error en la reconstrucción en términos del MSE lleva a una representación topológicamente correctas en el espacio latente. Pero al mismo tiempo mostramos que esta relación entre la función de costo y la topología no es trivial, existen configuraciones donde aún teniendo un error relativamente bajo en términos de reconstrucción, el *embedding* en el espacio latente puede resultar incorrecto.

Cuando se reconstruye el flujo de un sistema dinámico a partir de datos de una serie temporal mediante un mecanismo de *embedding*, no se reconstruye un atractor idéntico al original, sino uno topológicamente equivalente. Es importante destacar el valor de los *linking numbers* como un invariante que nos permite determinar esta equivalencia de manera categórica, y que, además, se puede computar fácil y robustamente a partir de los datos.

En las siguientes subsecciones discutimos particularidades del trabajo presentado en este capítulo así como también posibles enfoques para continuar la línea de trabajo.

Función de Costo

La elección de la función de costo a utilizar es un elemento central en cualquier algoritmo de *machine Learning*. Esta elección determina que cantidad vamos a buscar optimizar mediante el proceso de entrenamiento. En nuestro trabajo optamos por utilizar el MSE, ya que es la función de costo que se utiliza mas comúnmente en problemas de regresión. Una continuación natural de este trabajo sería preguntarse como depende el éxito del *embedding* de la función de costo utilizada. ¿Existe una función de costo mas adecuada que el MSE si lo que buscamos es una reconstrucción topológicamente correcta en el espacio latente? En este sentido, durante la realización del trabajo exploramos de manera no rigurosa otras 2 funciones de costo, las funciones $L = \sum_i |\hat{X}^i - X^i|^l$ con $l = 1$ y $l = 1/2$. Observamos, para ambas funciones de costo, un comportamiento cualitativamente equivalente al reportado usado MSE ($l = 2$). Existen trabajos posteriores a [69] que indagaron en funciones de costo diseñadas con el objetivo de reconstruir adecuadamente atractores, pero utilizando otro tipo de metodología para su evaluación [158]. Sin duda, la exploración de funciones de costo específicas para esta tarea es una dirección de investigación interesante para trabajos futuros.

Arquitectura de la red neuronal

Para realizar este trabajo decidimos optar por una arquitectura de red donde el único limitante para poder realizar una buena reconstrucción de la señal fuese el cuello de botella del *autoencoder*, es decir, la dimensión del espacio latente. Elegimos una red con una cantidad de parámetros tal que, si el espacio latente tiene un numero de unidades mayor que la dimensionalidad de los datos de entrada, el error de reconstrucción fuese muy bajo. Esto se puede observar en la figura 4.1, donde se muestra que el error de reconstrucción RMSE a la salida es menor a 0,05 para redes con espacios latentes de dimensión 3 o mayor. Este es un error relativamente bajo si tenemos en cuenta que los valores de la serie temporal oscila aproximadamente entre -20 y 20 con un desvío estándar de 11,5. En trabajos futuros sería interesante explorar como afecta la complejidad de la red utilizada, en términos de profundidad y numero de neuronas por capa, en el éxito para realizar *embeddings* topológicamente correctos.

Generalización a series multivaluadas

La principal desventaja de metodología de reconstrucción del flujo con *autoencoders* frente a la de *delays* temporales propuesta por Takens es su complejidad: debemos entrenar correctamente una red neuronal para realizar el *embedding*. Pero también ofrece varias posibles ventajas, algunas de las cuales ya se discutieron en este capítulo: Nos independiza de la selección de τ (un parámetro sensible en la metodología de *delays* temporales), parece ser robusta frente a la presencia de ruido, y admite trabajar con datos que no son necesariamente una serie temporal

unidimensional. Esta última es la principal ventaja de esta metodología, ya que los datos de entrada pueden ser tensores de alta dimensión con información de una o mas variables dinámicas. Esto posibilita la realización directa de *embeddings* a partir de, por ejemplo, películas de la dinámicas de un fenómeno sin la necesidad de tener que computar previamente los modos espaciotemporales del mismo.

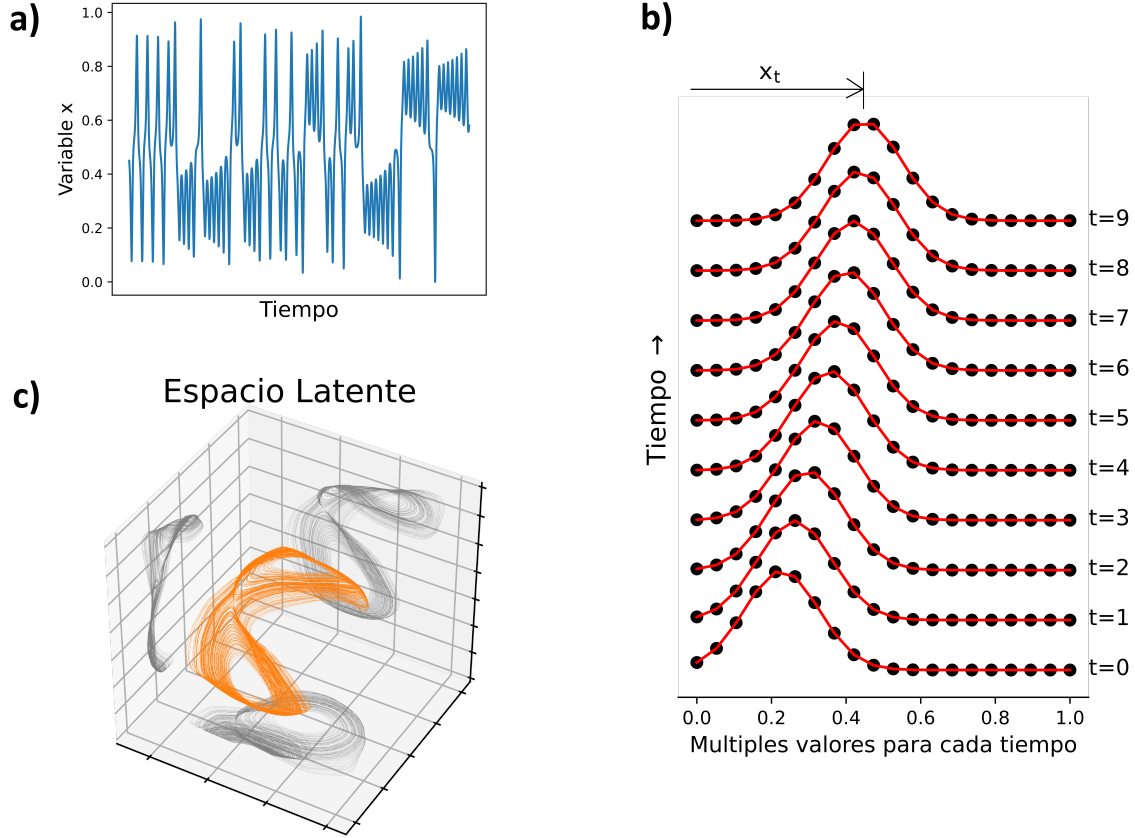


Figura 4.6: Ejemplo de reconstrucción de flujo con *autoencoder* para serie temporal multivaluada. (a) Serie temporal de la coordenada x en un atractor de Lorenz. (b) Vectores de entrada $\bar{v}_t$ para los primeros 10 pasos temporales. Los mismos poseen valores correspondientes a un pulso gaussiano cuya posición está determinada por el valor de x_t . (c) Representación de los datos en el espacio latente del *autoencoder* luego de la finalización del proceso de entrenamiento.

Para finalizar este capítulo presentamos, a modo de ejemplo, la reconstrucción en el espacio latente generada por un *autoencoder* para el caso de una serie temporal multivaluada. En este ejemplo tomamos como punto de partida la proyección al eje x de una solución correspondiente a un atractor de Lorenz normalizada entre 0 y 1, ver figura 4.6(a). Se construye una serie x_t inspeccionando el valor de la variable cada 0,16 unidades temporales. Luego, cada uno de estos valores se utiliza para determinar la posición de un pulso gaussiano de altura 1 en el intervalo $[0, 1]$, al cual se samplea en 20 puntos equiespaciados para generar un vector $\bar{v}_t \in \mathbb{R}^{20}$. En la figura 4.6(b) se presentan los vectores $\bar{v}_t$ generados para los primeros 10 pasos

temporales. Notemos que este es un ejemplo no trivial, ya que, si bien la serie multivaluada $\bar{v}_t$ posee codificada en el conjunto de sus 20 elementos la información sobre los valores x_t , ninguno de estos elementos por separado contiene suficiente información como para averiguar x_t .

Realizamos entonces el mismo procedimiento que en la sección anterior, pero esta vez tomando segmentos de 10 elementos consecutivos de la serie $\bar{v}_t$. Los valores correspondientes a estos segmentos se aplanan para generar vectores de 200 números, los cuales serán las instancias de entrenamiento. El modelo y el proceso de entrenamiento fue el mismo que el utilizado con anterioridad, con la única salvedad de que las capas de entrada y salida del *autoencoder* poseen 200 unidades en lugar de 256. Luego del entrenamiento, la representación de los datos en el espacio latente resulta semejante al flujo de un atractor de Lorenz, presentando sus dos lóbulos característicos. El flujo reconstruido por el *autoencoder* se presenta en la figura 4.6(c), se puede comparar su morfología con la del atractor de Lorenz en la figura del capítulo anterior 2.4. Con este ejemplo, mostramos que la metodología propuesta permite la reconstrucción del flujo incluso en casos donde la relación entre los datos de entrada ($\bar{v}_t$) y la variable del sistema codificada en ellos (x_t) es no trivial.

Capítulo 5

Codificación en Redes Neuronales Recurrentes

“ *That was the story I was telling myself... Isn't that interesting? The past is just a story we tell ourselves.* ”

Spike Jonze, *Her*, 2013

En este capítulo se estudiará la forma en que las redes neuronales recurrentes codifican series temporales dinámicas en su estado interno cuando son entrenadas para realizar predicciones sobre las mismas. Mostraremos, a través de experimentos numéricos, que estas redes son capaces de generar representaciones de los datos de entrada en su espacio interno que conservan propiedades topológicas del flujo del sistema dinámico del cual proviene la serie temporal. Además, se mostrará que la fidelidad de esta representación depende de la cantidad de pasos futuros usados durante el entrenamiento [70].

5.1. Introducción

En este capítulo, estudiaremos redes LSTM en una arquitectura *sequence-to-sequence* entrenadas en una tarea de predicción de series de tiempo generadas a partir de un sistema dinámico caótico. Analizaremos la codificación de la señal realizada por la celda LSTM en el espacio oculto, h_{in} , y pondremos a prueba la hipótesis de que la codificación en este entorno multivaluado posee características topológicas equivalentes a las del flujo del sistema dinámico original. Para evaluar esta equivalencia, realizamos experimentos numéricos donde comparamos la organización topológica de las órbitas inestables que coexisten con el atractor caótico mediante el valor de los *linking numbers* entre ellas (ver sección 2.2.5). Interpretamos la coincidencia entre la

organización topológica de las órbitas del sistema original y las generadas en el espacio oculto como una signatura de que el flujo original y el reconstruido poseen topologías equivalentes.

Mostraremos que, bajo ciertas condiciones, la representación de la serie en el espacio oculto de los modelos entrenados es equivalente a un *embedding*. Notablemente, este resultado sugiere que el proceso aprendido por la red neuronal recurrente para hacer predicciones se asemeja a los métodos de pronóstico estándar desarrollados por los dinamistas, en los que la serie de tiempo es primeramente representada en un espacio fases para ser estudiada.

El capítulo está organizado de la siguiente manera. En la segunda sección, describimos el proceso de generación de conjuntos de datos, la arquitectura de las redes entrenadas y los índices topológicos que usamos para comparar flujos. La tercera sección, Resultados, presenta los resultados obtenidos para dos escenarios de entrenamiento diferentes (predicción de un segmento futuro y predicción de los siguiente N pasos) y analiza las condiciones bajo las cuales el flujo en el espacio oculto resulta ser equivalente a un *embedding* de los datos de entrenamiento. En la sección final, discutimos los alcances de nuestros resultados, cómo puede ayudarnos a interpretar lo aprendido por las redes neuronales recurrentes y a estimar algunos hiperparámetros.

5.2. Métodos

5.2.1. Generación del Dataset

Al igual que en el capítulo anterior, el primer paso para generar nuestro conjunto de datos fue integrar un sistema Rössler,

$$\begin{aligned}\frac{dx}{dt} &= -y - z \\ \frac{dy}{dt} &= x + ay \\ \frac{dz}{dt} &= b + z(x - c),\end{aligned}\tag{5.2.1}$$

con $a = b = 0,1$ y $c = 14$ usando un método de Runge-Kutta de cuarto orden. La serie de tiempo corresponde al valor escalar de la variable x , muestreado cada $\tau = 0,08$ unidades de tiempo (ver figura 5.1). Nótese que, al provenir de un atractor caótico, la serie de tiempo nunca se repite, y la tarea de predecir su estado futuro no es trivial. Para simular datos más realistas y agregar robustez a los resultados, agregamos ruido a la serie de tiempo en el proceso de entrenamiento. El ruido agregado fue ruido blanco gaussiano, con media cero y una desviación estándar equivalente al 2 % de la desviación estándar la serie completa de valores de x (diferente para cada modelo entrenado).

Para entrenar la red, dividimos la serie de tiempo en segmentos de $L = 32$ puntos:

$$\{x_1, x_2, \dots, x_L\}, \{x_2, x_3, \dots, x_{L+1}\}, \dots \{x_{N-L+1}, x_{N-L+2}, \dots, x_N\}.\tag{5.2.2}$$

. Cada uno de estos segmentos incluye datos correspondientes a un lapso de tiempo de aproximadamente la mitad de la duración de la órbita de periodo 1 del atractor. Luego, tomamos

pares de segmentos subsecuentes: el primero para ser usado como entrada de la red, $\bar{x} \in \mathbb{R}^{32}$, y el segundo como salida deseada, $\bar{y} \in \mathbb{R}^{32}$. Un esquema de esto se muestra en la figura 5.1(b). Dada que la dimensionalidad del atractor en este sistema es $D \approx 2,02$, el teorema de Takens garantiza que cada uno de estos segmentos tiene suficiente información para representar inequívocamente un estado particular del sistema.

El resultado de este proceso fueron 62461 pares de elementos $(\bar{x}, \bar{y})$, los conjuntos de datos de entrada y de destino $(\bar{X}, \bar{Y})$. Finalmente, se realizó una división de los datos entre conjunto de entrenamiento y testeo. Usamos los datos correspondientes al primer 80 % de la serie temporal para entrenar la red y los datos del último 20 % para testearla.

5.2.2. Implementación de la Red Neuronal

La arquitectura “desenrollada” de nuestra red *sequence-to-sequence* se muestra en la figura (ver figura 5.1). La celda LSTM de entrada lee los 32 elementos de una secuencia de entrada, $\bar{x}^i \in \mathbb{R}^{32}$, y genera una codificación, $\bar{h}^i \in \mathbb{R}^H$, que es igual al último valor de activación de las neuronas en su espacio oculto. Luego, una celda LSTM de salida recibe este estado y genera secuencialmente 32 estados ocultos. Estos estados son leídos por una neurona de salida (indicada como *Dense* en la figura 5.1) que transforma cada uno de estos estados en un valor unidimensional, generando la salida predicha $\bar{y}^i \in \mathbb{R}^{32}$. Elegimos la dimensión del espacio oculto $H = 3$ en ambas celdas LSTM ya que de esta forma nos permite usar el 2.2.5 para estudiar la codificación de las series. Además, esta la dimensión óptima para realizar un *embedding* de datos que provienen de un atractor de Rössler.

Para implementar las redes neuronales, usamos Keras 2.3.1 con Tensorflow 1.15.2 como backend. La función de costo minimizada durante el entrenamiento fue el Error Cuadrático Medio (MSE) entre la salida deseada $\bar{y}$ y la salida de la red $\bar{y}_{pred}$. La función de activación utilizada para las neuronas de las celdas LSTM fue la tangente hiperbólica, mientras que la de la neurona de salida fue lineal. Usamos el algoritmo Adam como optimizador, con una tasa de aprendizaje $lr = 0,001$ y tasas de decaimiento exponencial $\beta_1 = 0,9$ y $\beta_2 = 0,999$ [111]. Se utilizó un tamaño de *batch* de 64 y los modelos se entrenaron durante 201 *epochs*. No se realizó una búsqueda sistemática y exhaustiva en el espacio de hiperparámetros para minimizar el MSE en el conjunto de testeo.

5.2.3. Cómputo de Linking Numbers

Al igual que en el trabajo presentado en el capítulo anterior, utilizamos los *linking numbers* entre las órbitas periódicas inestables que conviven con el atractor caótico como una forma de examinar si el flujo reconstruido en el espacio oculto presenta características equivalentes al flujo del sistema original. La forma de computarlos fue utilizando la función en Python descrita en la sección 4.2.3 y disponible en el repositorio github.com/gon-uri/linking_number.

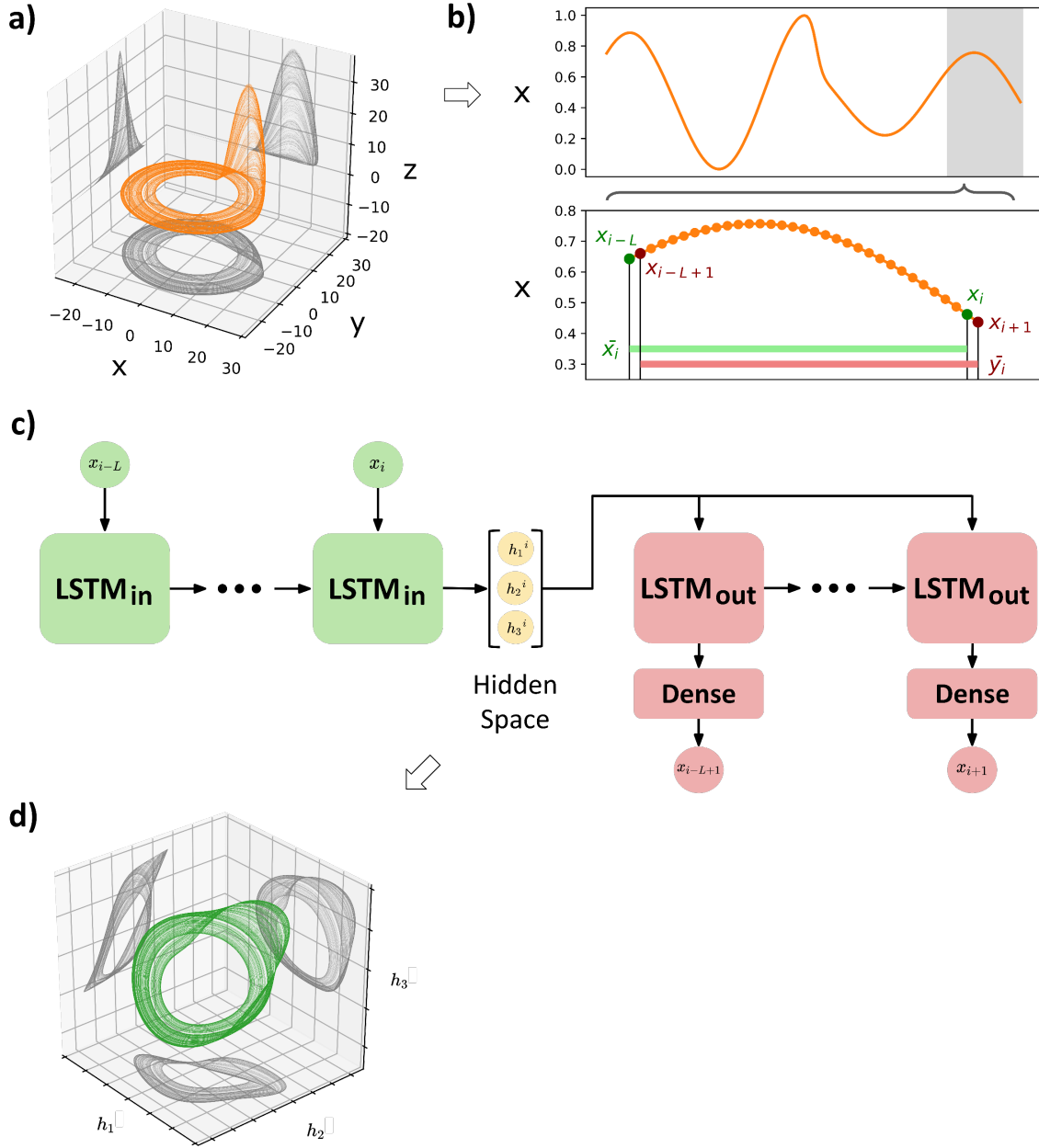


Figura 5.1: *Embeddings* generados por redes LSTMs en entrenadas para pronóstico. (a) Atractor caótico generado a partir de la resolución numérica del sistema de Rössler. (b) Series de tiempo generadas a partir de los valores de la variable x para entrenar la red neuronal: para cada instancia de entrenamiento, se usaron L puntos consecutivos como entrada y los siguientes L puntos consecutivos como salida deseada. (c) Arquitectura de la red neuronal utilizada. Una celda LSTM de entrada lee los datos de entrada y genera *embeddings* en el espacio oculto, luego una celda LSTM de salida conectada a una capa densa constituida por una sola neurona genera el pronóstico. La función de costo que se minimiza es el error cuadrático medio entre esta salida y el segmento objetivo de la serie original. (d) Ejemplo de un atractor caótico reconstruido en el espacio oculto.

Primero, identificamos segmentos que eran buenas aproximaciones las órbitas periódicas inestables de período 1,2 y 3 del atractor de Rössler original. Los Números de enlace entre estas aproximaciones de las órbitas resultan: $\ln(P1,P2) = -1$, $\ln(P1,P3) = -1$, $\ln(P2,P3) = -2$. Nos referimos como P1 a la solución del período 1, P2 a la solución del período 2 y P3 a una de las dos soluciones del período 3. Luego, se calcularon los *linkin numbers* del mismo conjunto de órbitas periódicas pero en la representación de la serie temporal realizada por las redes recurrentes en su espacio oculto (ver figura 5.2). Para comparar la topología de la codificación realizada por las redes recurrentes con la del flujo original, se compararon los valores de los *linkin numbers* en ambos casos. Si alguno de estos índices topológicos no coincide, se determina categóricamente que la representación en el espacio oculto no es equivalente al flujo original. En caso de coincidir los 3 valores, lo consideraremos como un indicio de que la topología es equivalente.

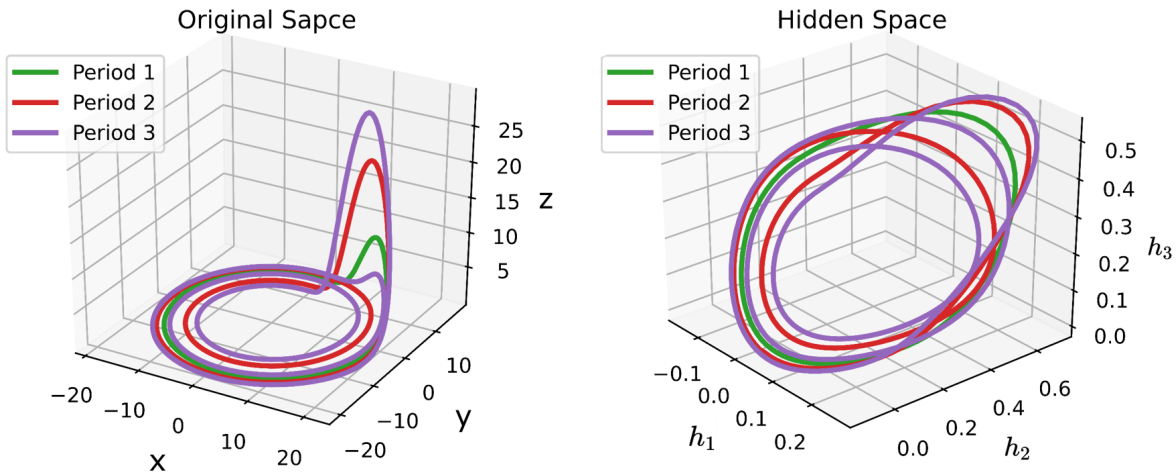


Figura 5.2: Comparación de la topología del flujo entre el sistema original y la codificación de la serie en el espacio oculto de la red LSTM. Se muestra la aproximación de las orbitas periódicas inestables de periodo 1,2 y 3 que conviven con el atractor. En este ejemplo, las órbitas formadas en el espacio oculto poseen los mismo *linkin numbers* que en el atractor original: $\ln(P1,P2) = -1$, $\ln(P1,P3) = -1$ y $\ln(P2,P3) = -2$.

5.3. Resultados

5.3.1. Predicción del próximo segmento

Primero, entrenamos 70 modelos *sequence-to-sequence* de acuerdo con el proceso descrito en la sección 5.2.2. Nótese que, a pesar de seguir el mismo procedimiento, cada proceso de entrenamiento es diferente debido a su naturaleza estocástica (inicialización de parámetros, generación de ruido y barajado del conjunto de entrenamiento). En la figura 5.3 presentamos el resultado del proceso de entrenamiento para 201 épocas. En rojo y azul, evolución del Error Cuadrático

Medio durante el entrenamiento para el conjunto de prueba y el conjunto de entrenamiento; las líneas indican la mediana y la sombra el rango intercuartílico. Una vez finalizado el proceso de entrenamiento, el error cuadrático medio de los conjuntos de entrenamiento y prueba ha alcanzado una meseta para la mayoría de los modelos.

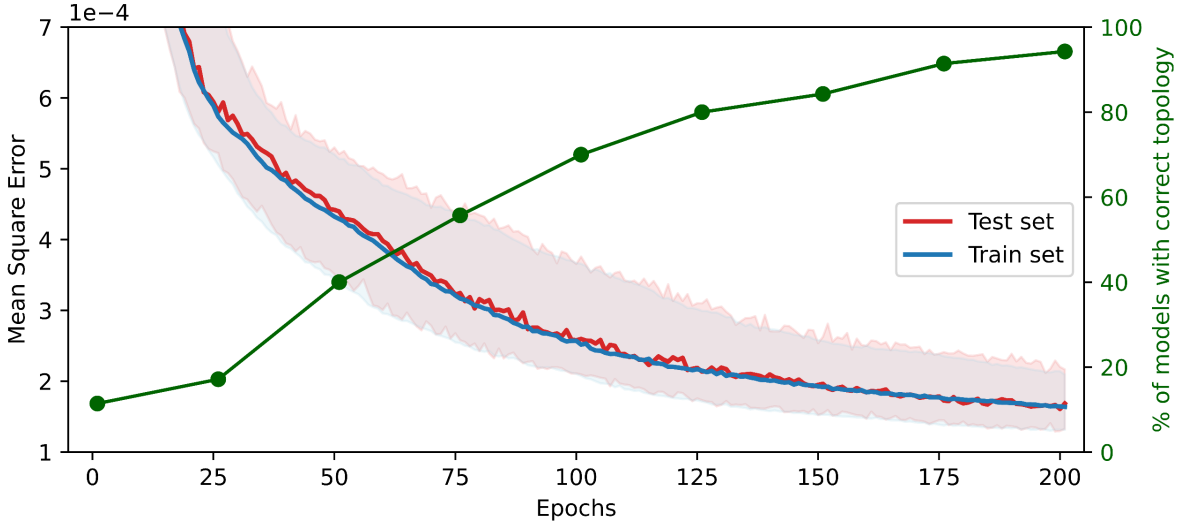


Figura 5.3: Resultados del proceso de entrenamiento para los 70 modelos. En rojo y azul se presenta la evolución del MSE para el conjunto de testeo y el conjunto de entrenamiento respectivamente. Las líneas indican la mediana y las sombras el rango intercuartílico. En verde mostramos el porcentaje de modelos cuya organización topológica de las órbitas inestables en el espacio oculto resulta la misma que en el sistema original.

Para estudiar la codificación de la serie temporal realizada por la red LSTM, tomamos la activación de las neuronas ocultas en el último paso de la entrada LSTM, $(h_1)^i$, $(h_2)^i$ y $(h_3)^i$, como una representación multivaluada del estado de la serie. En la figura 5.3 mostramos la representación del todo el conjunto de entrenamiento en el espacio oculto para una de las redes luego de finalizar su proceso de entrenamiento. Evaluamos la organización topológica de las órbitas en el espacio oculto calculando los números de enlace de las órbitas periódicas en esta nueva representación. En la Figura 5.2, mostramos la reconstrucción de los tres segmentos que se aproximan a las soluciones periódicas. En el caso que usamos como ejemplo, el entrenamiento de la red da como resultado una organización topológica de las órbitas periódicas en el espacio oculto es idéntica a la de las órbitas originales, siendo entonces equivalente a lo que se obtiene mediante un *embedding*.

Examinamos la estructura topológica de las órbitas en el espacio oculto de todos los modelos entrenados en diferentes épocas durante el proceso de entrenamiento y calculamos la proporción de modelos cuya codificación de la serie temporal en este espacio resulta ser un flujo equivalente a un *embedding*. El porcentaje de modelos con topología correcta en función del *epoch* de entrenamiento se muestra en la figura 5.3. Observe cómo aumenta el número de modelos con

organización topológica correcta a medida que las redes aprenden a hacer mejores predicciones y el MSE entre el segmento objetivo predicho y el real disminuye. Después del primer *epoch*, solo el 11 % de los modelos conducen a la organización topológica correcta, mientras que después de 201 *epochs*, el 94 % de ellos lo hace.

El 6 % de los modelos con topología incorrecta fueron modelos que no alcanzaron un buen mínimo de la función de costo durante la fase de entrenamiento. Existe una diferencia significativa entre el error cuadrático medio del conjunto de pruebas en estos modelos, $4,94(4,25 - 5,21) \times 10^{-4}$, y los que tienen la topología correcta, $1,59(1,29 - 2,12) \times 10^{-4}$. Los valores se presentan como mediana (rango intercuartílico).

En este primer experimento numérico, se entrenaron las redes recurrentes con el objetivo de, dado un segmento de entrada $\{x_{i-L+1}, \dots, x_i\}$, tener que predecir el siguiente segmento $\{x_{i-L+2}, \dots, x_{i+1}\}$, con $L = 32$. Es destacable que, para lograrlo, el LSTM genera una codificación en el espacio oculto que es topológicamente equivalente al atractor original. En el texto a continuación, discutimos las razones y condiciones para que esto suceda.

A la luz del teorema de Takens, sabemos que cada instancia tanto los datos de entrada como los de destino, $(\bar{X}, \bar{Y})$, se encuentran en variedades que son un *embedding* de la serie temporal en $\mathbb{R}^L$, y resultan equivalentes al atractor de Rössler original. Llamemos $M_{in} \subset \mathbb{R}^{32}$ y $M_{out} \subset \mathbb{R}^{32}$ a las variedades de entrada y de destino. Entonces, nuestra función objetivo, F , es un mapeo uno a uno de puntos en M_{in} a puntos en M_{out} , es decir, $F : M_{in} \rightarrow M_{out}$. Sabemos que es uno a uno ya que cada punto posee un único futuro. Al entrenar nuestro modelo, al que llamamos f_θ , estamos tratando de encontrar un conjunto de parámetros, θ^* , de modo que nuestra red neuronal recurrente sea una buena aproximación de nuestra función objetivo F en términos del error cuadrático medio. Pero nuestra red tiene una restricción importante: la información sobre la entrada debe ser transportada hacia la salida de la red en el espacio oculto $\bar{h} \subset \mathbb{R}^H$. Entonces, para que f_θ pueda hacer un mapeo uno a uno de M_{in} a M_{out} , H tiene que ser igual o mayor que la dimensión del atractor original. En nuestro caso, $H = 3$ es el número entero óptimo para cumplir esta condición.

Si la red entrenada f_{θ^*} resulta una buena aproximación de la función objetivo F , también generará un mapeo uno a uno entre la entrada y los datos objetivo, M_{in} y M_{out} . Esto significa que, si nuestro modelo f_θ es lo suficientemente complejo y está debidamente entrenado, asegurará una biyección entre M_{in} y la representación de los datos en el espacio oculto, M_h , que actúa como un cuello de botella para la información que fluye de la entrada a la salida. Esta biyección es una condición clave necesaria para que la codificación realizada por el LSTM resulte equivalente a un *embedding*.

Los atractores también se pueden caracterizar por el exponente máximo de Lyapunov (MLE), una cantidad que mide la tasa de separación de trayectorias infinitesimalmente cercanas y está relacionada con la noción de predictibilidad para un sistema dinámico, ver sección 2.2.2.

Como análisis adicional, calculamos el MLE para el atractor de Rössler original, $L = 0,074$, y para los atractores reconstruidos en el espacio oculto. Encontramos $L = 0,074(0,072 - 0,077)$ para modelos donde el entrenamiento condujo a una organización topología correcta de las órbitas y $L = 0,084(0,081 - 0,087)$ para modelos donde el entrenamiento condujo a una incorrecta. Podemos entonces afirmar que los modelos que alcanzaron un buen mínimo durante el entrenamiento presentan un valor para el mayor exponente de Lyapunov que resulta cercano al calculado para el atractor original. Sin embargo, este tema requiere más investigación. En trabajos futuro se podrían explorar diferentes propiedades métricas, como la dimensión fractal y la entropía métrica relativa, en atractores con diferentes valores de parámetros y niveles de ruido [159–161].

5.3.2. Predicción de N pasos adelante

Es interesante explorar lo que sucede cuando, en lugar de predecir el siguiente segmento, entrenamos una red recurrente para predecir cierto número de pasos adelante en la serie de tiempo. Esta es una tarea convencional para la cual se usan este tipo de redes.

Usamos la misma red y esquema de entrenamiento que en la sección anterior, pero esta vez para cada segmento de entrada $\bar{x}_i$ de $L = 32$ puntos, $\{x_{i-L+1}, \dots, x_i\}$, la salida objetivo $\bar{y}_i$ está compuesta por los siguientes N puntos en la serie de tiempo, $\{x_{i+1}, \dots, x_{i+N}\}$. Exploramos diferentes valores de pasos adelante en la predicción, $N = 1, 2, 3$ y 6 , entrenando 50 modelos para cada uno de estos valores.

De la misma forma que hicimos en la sección anterior, evaluamos la estructura topológica de las órbitas periódicas en el espacio oculto para todos los modelos entrenados. Calculamos el porcentaje de modelos con organizaciones correctas al final del proceso de entrenamiento para los diferentes valores de N explorados. Los resultados se muestran en la figura 5.4(a). Para $N = 1$, solo el 64 % de los modelos presentan una correcta organización topológica en el espacio oculto después de 201 *epochs*. Pero esta proporción aumenta cuando aumenta el número de pasos adelante. Para $N = 3$ y $N = 6$, el porcentaje de modelos con topología correcta alcanza el 95 %. En la figura 5.4(b) presentamos el error cuadrático medio en los conjuntos de entrenamiento y de testeo para los diferentes valores de N utilizados. Observe que la tarea de predecir pasos adicionales se vuelve cada vez más difícil de lograr para la red neuronal.

Si comparamos el error cuadrático medio en el conjunto de prueba entre los modelos con organización topológica incorrecta con el de los modelos con organización topológica correcta para $N = 1$, no hay diferencia significativa. Para modelos con topología incorrecta, el MSE es $5,57(5,45 - 6,06) \times 10^{-5}$, mientras que para modelos con topología correcta es $5,64(5,41 - 6,40) \times 10^{-5}$. Esto significa que incluso los modelos que alcanzaron un buen mínimo de la función de costo durante la fase de entrenamiento terminan con topologías incorrectas.

A diferencia del caso descrito en la sección anterior, en este caso los puntos en el conjunto

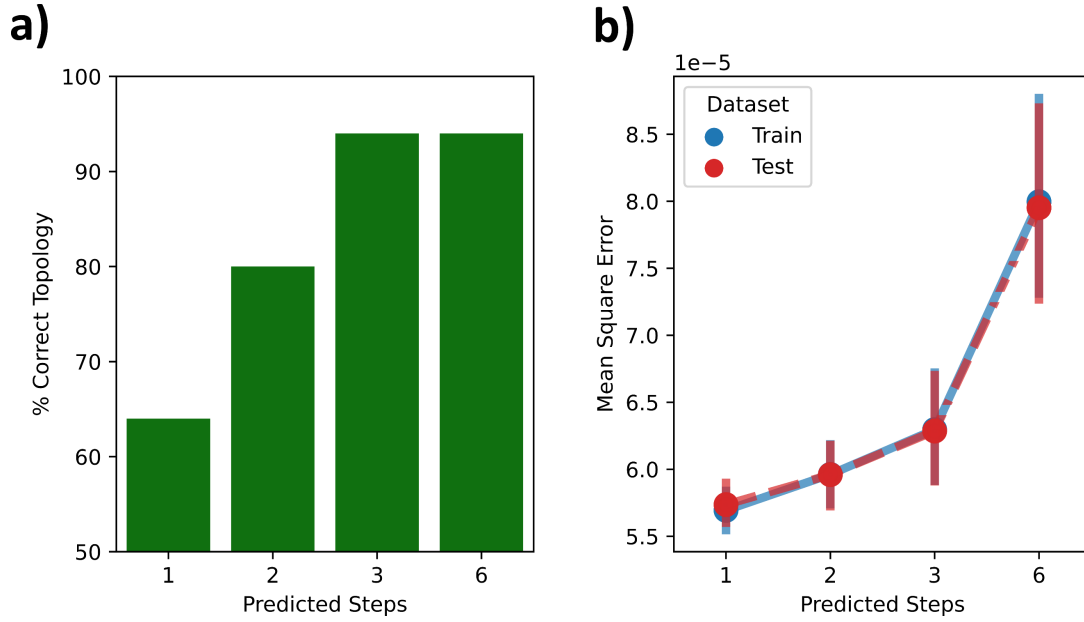


Figura 5.4: Resultado correspondiente a los modelos LSTM entrenados para predecir 1, 2, 3 y 6 pasos futuros. Para cada uno de estos valores, se entrenaron 50 redes. (a) Porcentaje de los modelos cuya organización topológica de las órbitas inestables en el espacio oculto al final del entrenamiento resulta la misma que en el sistema original. (b) Error de predicción (MSE) en función del número de pasos predichos para el conjunto de entrenamiento y el conjunto de testeo.

de datos de destino $\bar{Y}$ no definen inequívocamente un estado del sistema para todos los casos. Dependiendo de la elección del valor de N , la variedad objetivo $M_{out} \subset \mathbb{R}^N$ puede o no ser un *embedding* de la serie de tiempo. Analicemos los casos $N = 1$ y $N = 2$. Sabemos que la variedad objetivo $M_{out} \subset \mathbb{R}^N$ no puede ser un homeomorfismo del atractor de Rössler original. Esto se debe a que el atractor de Rössler posee una dimensionalidad mayor a 2. Por la misma razón, también sabemos que la función objetivo F no es un mapeo uno a uno entre puntos de M_{in} y puntos de M_{out} . Algunos puntos distintos de M_{in} irán a parar al mismo punto de M_{out} . Así, para estos valores de N , que un modelo entrenado f_{θ^*} sea una buena aproximación de F no implica necesariamente exista una biyección entre M_{in} y la representación de los datos en el espacio oculto M_h . Es por este motivo que una proporción importante de modelos entrenados terminan con una organización incorrecta de las órbitas periódicas en el espacio oculto, independientemente de lo bien que estén cumpliendo la tarea de predicción.

5.4. Discusiones

En este capítulo mostramos que una red neuronal recurrente en configuración *sequence-to-sequence* entrenada en una tarea de predicción puede aprender a realizar una codificación de una serie temporal dinámica en su espacio oculto que resulta equivalente a realizar un

embedding. Es decir, mostramos que existe una relación entre minimizar la función de costo de la red y aprender una representación topológicamente correcta de la serie en el espacio oculto. Notablemente, el proceso realizado por la red recurrente se asemeja al tradicionalmente realizado por los dinamicistas, en cual, para realizar predicciones de una serie temporal, se realiza primero un *embedding* de la misma. En general este *embedding* es realizado por medio del mecanismo de *delays* temporales propuesto por Takens, donde el problema no trivial es encontrar un intervalo de muestreo adecuado τ para que el *embedding* resulte óptimo.

También exploramos cuales son las condiciones necesarias para que la red recurrente realice una codificación equivalente a un *embedding* de la serie. Encontramos una dependencia con la cantidad de pasos que utilizamos como objetivo de predicción de la red. Si el número de pasos no es suficiente para definir inequívocamente un estado del sistema, no se garantiza que la representación de los datos en el hidden space sea un flujo topológicamente equivalente al de el sistema dinámico subyacente. Y esto resulta independiente de si la red realiza, o no, un buen trabajo en términos del error de predicción. Por ende, si pretendemos que la codificación en el espacio latente de la red recurrente tenga las características del flujo del sistema dinámico original, es preciso entrenar la red en la tarea de predecir mas allá de únicamente el primer paso en el futuro.

Decidimos utilizar celdas LSTM ya que es una de las variantes de redes recurrentes más utilizadas, pero los resultados presentados pueden extenderse a modelos secuenciales utilizando otros tipos de celdas (RNN clásicas, GRU). Nótese que las discusiones dadas en el capítulo son independientes de las características particulares del modelo f_θ (siempre y cuando f_θ posea la complejidad suficiente para poder aproximar adecuadamente a F).

Lo mismo sucede con la arquitectura de la red. Si bien en este capítulo decidimos utilizar una arquitectura simple para procesar secuencias de valores escalares, también es posible utilizarlas para procesar datos secuenciales más complejos, como series de tiempo multivariadas o películas (secuencias de imágenes). En los casos donde los datos de entrada poseen una dimensionalidad alta, una practica común es combinar las celdas recurrentes con otro tipo de redes que procesan los datos antes de entrar a las celdas, como pueden ser redes neuronales convolucionales o *autoencoders*. La idea es que esa primer parte de la red reduzca la dimensionalidad de los datos de entrada, aprendiendo a preservar únicamente las características relevantes de cada elemento en la secuencia, y luego las celdas recurrentes operen relacionando temporalmente esas características relevantes. Aún en estas arquitecturas mas complejas, para que la codificación en el espacio oculto tenga las características de un *embedding*, siguen siendo válidas la condiciones discutidas sobre la dimensión del espacio oculto y el numero de pasos a predecir.

Capítulo 6

Conclusiones

“ There is no real ending. It's just the place where you stop the story. ”

Frank Herbert, *Fullerton Interview*, 1969

El desarrollo de nuevas tecnologías en los últimos años nos permitió medir y acceder a una cantidad de datos experimentales sin precedentes, y todo indica que esta tendencia de incremento en el volumen de datos disponibles continuará durante los próximos años. Cualquier tipo de procesamiento manual o supervisado de esta cantidad de información resulta cada vez más costoso, y es por este motivo que el uso de *machine learning* para analizar datos experimentales se irá volviendo cada vez más indispensable en muchas áreas de investigación. Pero la adopción de estas nuevas herramientas de análisis no es inmediata, es preciso adaptar la manera en que usamos estos algoritmos a las características de los datos y a las tareas específicas en cada área. El trabajo desarrollado a lo largo de esta tesis doctoral ha contribuido a este proceso en el campo de sistemas dinámicos.

En este trabajo se analizaron algoritmos de redes neuronales en tareas relativas a series temporales dinámicas. Los resultados del mismo se separan en dos partes. Una primera parte en la cual se estudiaron redes neuronales en una arquitectura de *deep autoencoders*, y una segunda en la cual se estudiaron redes neuronales recurrentes.

En la primera parte, se mostró que es posible realizar un *embedding* de una serie temporal dinámica utilizando un *deep autoencoder*. Esto se logra entrenando al modelo para que aprenda a comprimir la información de segmentos de la serie temporal en su espacio latente. Además se mostró que, inspeccionando el error de reconstrucción alcanzado por el modelo, este mismo proceso permite estimar la mínima dimensión de un espacio euclidiano en la que puede inscribirse el flujo correspondiente a la serie, es decir, estimar su dimensionalidad.

Esta nueva metodología para reconstruir espacios de fases a partir de series temporales se

presenta como una alternativa a la de *delays* temporales propuesta por Takens [22]. Si bien la misma cuenta con la desventaja de tener que entrenar exitosamente un modelo de *machine learning* para realizar el *embedding*, al mismo tiempo presenta propiedades que pueden resultar muy beneficiosas. En primer lugar, nos independiza de la selección del *delay* temporal y la dimensionalidad, parámetros sensibles en la metodología de *delays* temporales. En segundo lugar, el algoritmo se muestra robusto ante ruido. En último lugar, y más importante, con la metodología de *embedding* de *deep autoencoders* no es necesario que la serie temporal corresponda a la evolución de una única variable dinámica del sistema, cada elemento de la serie puede ser un tensor de alta dimensión que contenga información de una o más variables.

Dado lo expuesto en el párrafo anterior, esta metodología permite reconstruir las variables dinámicas de un sistema a partir de secuencias de datos no estructurados, como un video, donde cada elemento de la secuencia contenga información parcial del sistema. Esto le otorga un gran potencial para combinarse con los algoritmos de búsqueda de ecuaciones diferenciales a partir de datos, como por ejemplo *Compressed Sensing* o *SINDy*, siendo incluso posible entrenar de manera conjunta los parámetros de ambos modelos [58, 59].

En la segunda parte de este trabajo, se analizaron las condiciones bajo las cuales una red recurrente, entrenada en una tarea de predicción, puede generar un *embedding* correcto una serie temporal dinámica en su espacio de representación interna (su *hidden space*). Se mostró que esto depende de la cantidad de pasos a predecir para la cual se entrena la red. En la mayoría de los casos, entrenarla para predecir un único paso futuro no es suficiente para garantizar un *embedding* topológicamente correcto. Estos hallazgos pueden resultar de gran interés para las personas que trabajan con redes neuronales recurrentes en todo tipo de sistemas dinámicos [38–40, 150, 162, 163].

Tener una representación correcta de los datos en el *hidden space* permite acceder a un retrato de fase equivalente al del sistema original del que proviene la serie temporal, el cual suele ser desconocido. Por ejemplo, en los últimos años existen numerosos trabajos en los cuales se utilizan redes neuronales recurrentes para predecir la evolución de películas de fluidos [43, 164–166]. Nuestros resultados indican que, bajo las condiciones adecuadas, la dinámica de baja dimensión en el *hidden space* se puede estudiar como un *proxy* de la dinámica de los modos activos involucrados en el problema real bajo investigación.

Además, tener un *embedding* de una serie temporal dinámica nos asegura que cada punto en el *hidden space* corresponde exclusivamente a un estado del sistema. Esto puede ser muy beneficioso si se quiere realizar una tarea de clasificación o *clustering* sobre este espacio. Queda pendiente para trabajos futuros investigar si una representación correcta del flujo en el espacio oculto también contribuye a tener una predicción a largo plazo más robusta y precisa del comportamiento del sistema.

Como observación final sobre los resultados del presente trabajo vale la pena destacar que la

interacción entre ambos campos, sistemas dinámicos y *machine learning*, resultó beneficiosa en ambos sentidos. Por un lado, los modelos de *machine learning* aportaron a los dinamicistas una alternativa flexible y poderosa a los mecanismos propuestos por Takens para generar *embeddings* de señales temporales. Por el otro, la teoría de sistemas dinámicos aportó una metodología novedosa para analizar la manera en que se codifica la información de una serie temporal en una red neuronal, contribuyendo así en los esfuerzos por interpretar los mecanismos detrás del éxito de este tipo de algoritmos. Además, se mostró que conocer las características dinámicas de una serie temporal puede ayudar a estimar cotas para los valores de ciertos hiperparámetros en los algoritmos que se usen para modelarlas.

Bibliografía

- [1] R. DesLauriers, *Brief history of scientific developments in the study of tides*, *The Canadian Surveyor* **13** (1956) 165.
- [2] J. Case, *Understanding tides—from ancient beliefs to present-day solutions to the laplace equations*, *SIAM News* **33** (2000) 1.
- [3] D. Cartwright, *On the origins of knowledge of the sea tides from antiquity to the thirteenth century*, *Earth sciences history* **20** (2001) 105.
- [4] D. E. Cartwright, *Tides: a scientific history*. Cambridge University Press, 2000.
- [5] S. W. Thomson, E. Roberts, G. Deacon, S. G. Airy, H. Law, J. Redman et al., *Discussion on tidal instruments*, in *Minutes of the Proceedings of the Institution of Civil Engineers*, vol. 65, pp. 26–64, Thomas Telford-ICE Virtual Library, 1881.
- [6] B. Parker, *The tide predictions for d-day*, *Physics Today* **64** (2011) 35.
- [7] M. Mudelsee, *Trend analysis of climate time series: A review of methods*, *Earth-science reviews* **190** (2019) 310.
- [8] D. S. Stoffer and H. Ombao, *Special issue on time series analysis in the biological sciences*, 2012.
- [9] B. Ghosh, B. Basu and M. O’Mahony, *Multivariate short-term traffic flow forecasting using time-series analysis*, *IEEE transactions on intelligent transportation systems* **10** (2009) 246.
- [10] E. I. Vlahogianni, M. G. Karlaftis and J. C. Golias, *Short-term traffic forecasting: Where we are and where we’re going*, *Transportation Research Part C: Emerging Technologies* **43** (2014) 3.
- [11] T. G. Andersen, T. Bollerslev, P. Christoffersen and F. X. Diebold, *Volatility forecasting*, 2005.
- [12] P. Grassberger, T. Schreiber and C. Schaffrath, *Nonlinear time sequence analysis*, *International Journal of Bifurcation and Chaos* **1** (1991) 521.

- [13] M. Casdagli, *Nonlinear prediction of chaotic time series*, *Physica D: Nonlinear Phenomena* **35** (1989) 335.
- [14] G. Mindlin, H. Solari, M. Natiello, R. Gilmore and X.-J. Hou, *Topological analysis of chaotic time series data from the belousov-zhabotinskii reaction*, *Journal of Nonlinear Science* **1** (1991) 147.
- [15] G. M. Mindlin and R. Gilmore, *Topological analysis and synthesis of chaotic time series*, *Physica D: Nonlinear Phenomena* **58** (1992) 229.
- [16] G. Sugihara and R. M. May, *Nonlinear forecasting as a way of distinguishing chaos from measurement error in time series*, *Nature* **344** (1990) 734.
- [17] G. Sugihara, *Nonlinear forecasting for the classification of natural time series*, *Philosophical Transactions of the Royal Society of London. Series A: Physical and Engineering Sciences* **348** (1994) 477.
- [18] H. D. Abarbanel, R. Brown and J. B. Kadtko, *Prediction in chaotic nonlinear systems: Methods for time series with broadband fourier spectra*, *Physical Review A* **41** (1990) 1782.
- [19] H. Abarbanel, *Analysis of observed chaotic data*. Springer Science & Business Media, 2012.
- [20] H. Kantz and T. Schreiber, *Nonlinear time series analysis*, vol. 7. Cambridge university press, 2004.
- [21] E. Bradley and H. Kantz, *Nonlinear time-series analysis revisited*, *Chaos: An Interdisciplinary Journal of Nonlinear Science* **25** (2015) 097610.
- [22] F. Takens, *Detecting strange attractors in turbulence*, in *Dynamical systems and turbulence, Warwick 1980*, pp. 366–381, Springer, (1981).
- [23] L. M. Pecora and T. L. Carroll, *Driving systems with chaotic signals*, *Physical review A* **44** (1991) 2374.
- [24] P. D. Mininni, D. O. Gomez and G. Mindlin, *Stochastic relaxation oscillator model for the solar cycle*, *Physical review letters* **85** (2000) 5476.
- [25] P. D. Mininni, D. O. Gomez and G. B. Mindlin, *Simple model of a stochastically excited solar dynamo*, *Solar Physics* **201** (2001) 203.
- [26] J. P. Crutchfield and B. McNamara, *Equations of motion from a data series*, *Complex systems* **1** (1987) 417.

- [27] Y.-C. Lai, *Finding nonlinear system equations and complex network structures from data: A sparse optimization approach*, *Chaos: An Interdisciplinary Journal of Nonlinear Science* **31** (2021) 082101.
- [28] Y. LeCun, Y. Bengio and G. Hinton, *Deep learning*, *nature* **521** (2015) 436.
- [29] I. Goodfellow, Y. Bengio and A. Courville, *Deep learning*. MIT press, 2016.
- [30] B. Lim and S. Zohren, *Time-series forecasting with deep learning: a survey*, *Philosophical Transactions of the Royal Society A* **379** (2021) 20200209.
- [31] J. C. B. Gamboa, *Deep learning for time-series analysis*, *arXiv preprint arXiv:1701.01887* (2017) .
- [32] O. B. Sezer, M. U. Gudelek and A. M. Ozbayoglu, *Financial time series forecasting with deep learning: A systematic literature review: 2005–2019*, *Applied Soft Computing* **90** (2020) 106181.
- [33] H. Wang, Z. Lei, X. Zhang, B. Zhou and J. Peng, *A review of deep learning for renewable energy forecasting*, *Energy Conversion and Management* **198** (2019) 111799.
- [34] R. Yu, Y. Li, C. Shahabi, U. Demiryurek and Y. Liu, *Deep learning: A generic approach for extreme condition traffic forecasting*, in *Proceedings of the 2017 SIAM international Conference on Data Mining*, pp. 777–785, SIAM, 2017.
- [35] S. Hochreiter and J. Schmidhuber, *Long short-term memory*, *Neural computation* **9** (1997) 1735.
- [36] A. Graves, *Generating sequences with recurrent neural networks*, *arXiv preprint arXiv:1308.0850* (2013) .
- [37] C. Olah, *Understanding lstm networks*, .
- [38] M. Sangiorgio and F. Dercole, *Robustness of lstm neural networks for multi-step forecasting of chaotic time series*, *Chaos, Solitons & Fractals* **139** (2020) 110045.
- [39] M. Madondo and T. Gibbons, *Learning and modeling chaos using lstm recurrent neural networks*, in *MICS 2018 Proceedings*, 2018.
- [40] A. Chattopadhyay, P. Hassanzadeh and D. Subramanian, *Data-driven predictions of a multiscale lorenz 96 chaotic system using machine-learning methods: reservoir computing, artificial neural network, and long short-term memory network*, *Nonlinear Processes in Geophysics* **27** (2020) 373.
- [41] J. de Jesús Serrano-Pérez, G. Fernández-Anaya, S. Carrillo-Moreno and W. Yu, *New results for prediction of chaotic systems using deep recurrent neural networks*, *Neural Processing Letters* **53** (2021) 1579.

- [42] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott et al., *Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics*, *Neural Networks* **126** (2020) 191.
- [43] A. T. Mohan and D. V. Gaitonde, *A deep learning based approach to reduced order modeling for turbulent flow control using lstm neural networks*, *arXiv preprint arXiv:1804.09269* (2018) .
- [44] P. A. Srinivasan, L. Guastoni, H. Azizpour, P. Schlatter and R. Vinuesa, *Predictions of turbulent shear flows using deep neural networks*, *Physical Review Fluids* **4** (2019) 054603.
- [45] J. N. Kutz, *Deep learning in fluid dynamics*, *Journal of Fluid Mechanics* **814** (2017) 1.
- [46] C. Du, C. Du, L. Huang, H. Wang and H. He, *Structured neural decoding with multitask transfer learning of deep neural network representations*, *IEEE Transactions on Neural Networks and Learning Systems* (2020) .
- [47] Y. Lin, J. Li and H. Wang, *Dcnn-gan: Reconstructing realistic image from fmri*, in *2019 16th International Conference on Machine Vision Applications (MVA)*, pp. 1–6, IEEE, 2019.
- [48] R. VanRullen and L. Reddy, *Reconstructing faces from fmri patterns using deep generative neural networks*, *Communications biology* **2** (2019) 1.
- [49] G. Shen, T. Horikawa, K. Majima and Y. Kamitani, *Deep image reconstruction from human brain activity*, *PLoS computational biology* **15** (2019) e1006633.
- [50] F. Doshi-Velez and B. Kim, *Towards a rigorous science of interpretable machine learning*, *arXiv preprint arXiv:1702.08608* (2017) .
- [51] R. Shwartz-Ziv and N. Tishby, *Opening the black box of deep neural networks via information*, *arXiv preprint arXiv:1703.00810* (2017) .
- [52] G. Montavon, W. Samek and K.-R. Müller, *Methods for interpreting and understanding deep neural networks*, *Digital Signal Processing* **73** (2018) 1.
- [53] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti and D. Pedreschi, *A survey of methods for explaining black box models*, *ACM computing surveys (CSUR)* **51** (2018) 1.
- [54] C. Molnar, G. Casalicchio and B. Bischl, *Interpretable machine learning—a brief history, state-of-the-art and challenges*, in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 417–431, Springer, 2020.
- [55] C. Molnar, *Interpretable machine learning*. Lulu. com, 2020.

- [56] S. Chen, S. A. Billings and W. Luo, *Orthogonal least squares methods and their application to non-linear system identification*, *International Journal of control* **50** (1989) 1873.
- [57] C. Yao and E. M. Bollt, *Modeling and nonlinear parameter estimation with kronecker product representation for coupled oscillators and spatiotemporal systems*, *Physica D: Nonlinear Phenomena* **227** (2007) 78.
- [58] W.-X. Wang, R. Yang, Y.-C. Lai, V. Kovanis and C. Grebogi, *Predicting catastrophes in nonlinear dynamical systems by compressive sensing*, *Physical review letters* **106** (2011) 154101.
- [59] S. L. Brunton, J. L. Proctor and J. N. Kutz, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, *Proceedings of the national academy of sciences* **113** (2016) 3932.
- [60] K. Champion, B. Lusch, J. N. Kutz and S. L. Brunton, *Data-driven discovery of coordinates and governing equations*, *Proceedings of the National Academy of Sciences* **116** (2019) 22445.
- [61] S. L. Brunton and J. N. Kutz, *Data-driven science and engineering: Machine learning, dynamical systems, and control*. Cambridge University Press, 2019.
- [62] A. A. R. AlMomani, J. Sun and E. Bollt, *How entropic regression beats the outliers problem in nonlinear system identification*, *Chaos: An Interdisciplinary Journal of Nonlinear Science* **30** (2020) 013107.
- [63] G. Berkooz, P. Holmes and J. L. Lumley, *The proper orthogonal decomposition in the analysis of turbulent flows*, *Annual review of fluid mechanics* **25** (1993) 539.
- [64] A. Chatterjee, *An introduction to the proper orthogonal decomposition*, *Current science* (2000) 808.
- [65] S. Wold, K. Esbensen and P. Geladi, *Principal component analysis*, *Chemometrics and intelligent laboratory systems* **2** (1987) 37.
- [66] H. Abdi and L. J. Williams, *Principal component analysis*, *Wiley interdisciplinary reviews: computational statistics* **2** (2010) 433.
- [67] B. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. Kutz and S. Brunton, *Pysindy: A python package for the sparse identification of nonlinear dynamical systems from data*, *Journal of Open Source Software* **5** (2020) 2104.
- [68] A. A. Kaptanoglu, B. M. de Silva, U. Fasel, K. Kaheman, J. L. Callahan, C. B. Delahunt et al., *Pysindy: A comprehensive python package for robust sparse system identification*, *arXiv preprint arXiv:2111.08481* (2021) .

- [69] G. Urbarri and G. B. Mindlin, *The structure of reconstructed flows in latent spaces*, *Chaos: An Interdisciplinary Journal of Nonlinear Science* **30** (2020) 093109.
- [70] G. Urbarri and G. B. Mindlin, *Dynamical time series embeddings in recurrent neural networks*, *Chaos, Solitons & Fractals* (2021) 111612.
- [71] S. Wiggins, S. Wiggins and M. Golubitsky, *Introduction to applied nonlinear dynamical systems and chaos*, vol. 2. Springer, 2003.
- [72] S. H. Strogatz, *Nonlinear dynamics and chaos with student solutions manual: With applications to physics, biology, chemistry, and engineering*. CRC press, 2018.
- [73] R. Gilmore and M. Lefranc, *The topology of chaos*, 2003.
- [74] M. W. Hirsch and S. Smale, *Differential equations, Dynamical Systems, and Linear Algebra*. San Diego: Academic Press (1974) .
- [75] H. Poincaré, *The three-body problem and the equations of dynamics: Poincaré's foundational work on dynamical systems theory*, vol. 443. Springer, 2017.
- [76] J. Guckenheimer and P. Holmes, *Nonlinear oscillations, dynamical systems, and bifurcations of vector fields*, vol. 42. Springer Science & Business Media, 2013.
- [77] B. F. Schutz and D. B. F. Schutz, *Geometrical methods of mathematical physics*. Cambridge university press, 1980.
- [78] E. N. Lorenz, *Deterministic nonperiodic flow*, *Journal of atmospheric sciences* **20** (1963) 130.
- [79] D. Ruelle and F. Takens, *On the nature of turbulence*, *Les rencontres physiciens-mathématiciens de Strasbourg-RCP25* **12** (1971) 1.
- [80] T.-Y. Li and J. A. Yorke, *Period three implies chaos*, in *The theory of chaotic attractors*, pp. 77–84, Springer, (2004).
- [81] R. M. May, *Simple mathematical models with very complicated dynamics*, *The Theory of Chaotic Attractors* (2004) 85.
- [82] M. J. Feigenbaum, *Quantitative universality for a class of nonlinear transformations*, *Journal of statistical physics* **19** (1978) 25.
- [83] J.-P. Eckmann and D. Ruelle, *Ergodic theory of chaos and strange attractors*, *The theory of chaotic attractors* (1985) 273.
- [84] B. B. Mandelbrot, *Self-affine fractals and fractal dimension*, *Physica scripta* **32** (1985) 257.

- [85] P. Grassberger, *Generalized dimensions of strange attractors*, *Physics Letters A* **97** (1983) 227.
- [86] J. L. Kaplan and J. A. Yorke, *Chaotic behavior of multidimensional difference equations*, in *Functional differential equations and approximation of fixed points*, pp. 204–227, Springer, (1979).
- [87] S. Smale, *The mathematics of time*, vol. 451. Springer, 1980.
- [88] D. K. Arrowsmith, C. M. Place, C. Place et al., *An introduction to dynamical systems*. Cambridge university press, 1990.
- [89] N. B. Tufillaro, H. G. Solari and R. Gilmore, *Relative rotation rates: Fingerprints for strange attractors*, *Physical Review A* **41** (1990) 5717.
- [90] J.-P. Eckmann, S. O. Kamphorst, D. Ruelle et al., *Recurrence plots of dynamical systems*, *World Scientific Series on Nonlinear Science Series A* **16** (1995) 441.
- [91] N. Marwan, *A historical review of recurrence plots*, *The European Physical Journal Special Topics* **164** (2008) 3.
- [92] N. H. Packard, J. P. Crutchfield, J. D. Farmer and R. S. Shaw, *Geometry from a time series*, *Physical review letters* **45** (1980) 712.
- [93] H. Whitney, *Differentiable manifolds*, *Annals of Mathematics* (1936) 645.
- [94] T. Sauer, J. A. Yorke and M. Casdagli, *Embedology*, *Journal of statistical Physics* **65** (1991) 579.
- [95] J. Huke, *Embedding nonlinear dynamical systems: A guide to takens' theorem*, .
- [96] L. Noakes, *The takens embedding theorem*, *International Journal of Bifurcation and Chaos* **1** (1991) 867.
- [97] M. B. Kennel, R. Brown and H. D. Abarbanel, *Determining embedding dimension for phase-space reconstruction using a geometrical construction*, *Physical review A* **45** (1992) 3403.
- [98] A. Zhang, Z. C. Lipton, M. Li and A. J. Smola, *Dive into deep learning*, *arXiv preprint arXiv:2106.11342* (2021) .
- [99] T. M. Mitchell, *Does machine learning really work?*, *AI magazine* **18** (1997) 11.
- [100] G. James, D. Witten, T. Hastie and R. Tibshirani, *An introduction to statistical learning*, vol. 112. Springer, 2013.

- [101] D. H. Wolpert and W. G. Macready, *No free lunch theorems for optimization*, *IEEE transactions on evolutionary computation* **1** (1997) 67.
- [102] K. C. Kiwiel, *Convergence and efficiency of subgradient methods for quasiconvex minimization*, *Mathematical programming* **90** (2001) 1.
- [103] K. Kawaguchi, *Deep learning without poor local minima*, *Advances in neural information processing systems* **29** (2016) .
- [104] K. Kawaguchi, J. Huang and L. P. Kaelbling, *Effect of depth and width on local minima in deep learning*, *Neural computation* **31** (2019) 1462.
- [105] R. Ge, F. Huang, C. Jin and Y. Yuan, *Escaping from saddle points—online stochastic gradient for tensor decomposition*, in *Conference on learning theory*, pp. 797–842, PMLR, 2015.
- [106] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P. T. P. Tang, *On large-batch training for deep learning: Generalization gap and sharp minima*, *arXiv preprint arXiv:1609.04836* (2016) .
- [107] D. E. Rumelhart, G. E. Hinton and R. J. Williams, *Learning internal representations by error propagation*, tech. rep., California Univ San Diego La Jolla Inst for Cognitive Science, 1985.
- [108] A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, *Automatic differentiation in machine learning: a survey*, *Journal of Machine Learning Research* **18** (2018) 1.
- [109] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito et al., *Automatic differentiation in pytorch*, .
- [110] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean et al., *{TensorFlow}: A system for {Large-Scale} machine learning*, in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pp. 265–283, 2016.
- [111] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, *arXiv preprint arXiv:1412.6980* (2014) .
- [112] K. Hornik, M. Stinchcombe and H. White, *Multilayer feedforward networks are universal approximators*, *Neural networks* **2** (1989) 359.
- [113] G. Cybenko, *Approximation by superpositions of a sigmoidal function*, *Mathematics of control, signals and systems* **2** (1989) 303.
- [114] G. F. Montufar, R. Pascanu, K. Cho and Y. Bengio, *On the number of linear regions of deep neural networks*, *Advances in neural information processing systems* **27** (2014) .

- [115] A. M. Saxe, J. L. McClelland and S. Ganguli, *Exact solutions to the nonlinear dynamics of learning in deep linear neural networks*, *arXiv preprint arXiv:1312.6120* (2013) .
- [116] A. Ansuini, A. Laio, J. H. Macke and D. Zoccolan, *Intrinsic dimension of data representations in deep neural networks*, *Advances in Neural Information Processing Systems* **32** (2019) .
- [117] A. M. Saxe, Y. Bansal, J. Dapello, M. Advani, A. Kolchinsky, B. D. Tracey et al., *On the information bottleneck theory of deep learning*, *Journal of Statistical Mechanics: Theory and Experiment* **2019** (2019) 124020.
- [118] M. Belkin, D. Hsu, S. Ma and S. Mandal, *Reconciling modern machine-learning practice and the classical bias–variance trade-off*, *Proceedings of the National Academy of Sciences* **116** (2019) 15849.
- [119] S. Jastrzebski, Z. Kenton, D. Arpit, N. Ballas, A. Fischer, Y. Bengio et al., *Three factors influencing minima in sgd*, *arXiv preprint arXiv:1711.04623* (2017) .
- [120] M. S. Advani, A. M. Saxe and H. Sompolinsky, *High-dimensional dynamics of generalization error in neural networks*, *Neural Networks* **132** (2020) 428.
- [121] S. Arora, N. Cohen and E. Hazan, *On the optimization of deep networks: Implicit acceleration by overparameterization*, in *International Conference on Machine Learning*, pp. 244–253, PMLR, 2018.
- [122] Z. Allen-Zhu, Y. Li and Z. Song, *A convergence theory for deep learning via over-parameterization*, in *International Conference on Machine Learning*, pp. 242–252, PMLR, 2019.
- [123] S. Bubeck and M. Sellke, *A universal law of robustness via isoperimetry*, *Advances in Neural Information Processing Systems* **34** (2021) .
- [124] P. Baldi and K. Hornik, *Neural networks and principal component analysis: Learning from examples without local minima*, *Neural networks* **2** (1989) 53.
- [125] J. Lim, S. Ryu, J. W. Kim and W. Y. Kim, *Molecular generative model based on conditional variational autoencoder for de novo molecular design*, *Journal of cheminformatics* **10** (2018) 1.
- [126] C. Nash and C. K. Williams, *The shape variational autoencoder: A deep generative model of part-segmented 3d objects*, in *Computer Graphics Forum*, vol. 36, pp. 1–12, Wiley Online Library, 2017.
- [127] X. Lu, Y. Tsao, S. Matsuda and C. Hori, *Speech enhancement based on deep denoising autoencoder.*, in *Interspeech*, vol. 2013, pp. 436–440, 2013.

- [128] J. Li, Z. Struzik, L. Zhang and A. Cichocki, *Feature learning from incomplete eeg with denoising autoencoder*, *Neurocomputing* **165** (2015) 23.
- [129] S. Li, J. Kawale and Y. Fu, *Deep collaborative filtering via marginalized denoising auto-encoder*, in *Proceedings of the 24th ACM international on conference on information and knowledge management*, pp. 811–820, 2015.
- [130] D. Hartman and L. K. Mestha, *A deep learning framework for model reduction of dynamical systems*, in *2017 IEEE Conference on Control Technology and Applications (CCTA)*, pp. 1917–1922, IEEE, 2017.
- [131] F. J. Gonzalez and M. Balajewicz, *Deep convolutional recurrent autoencoders for learning low-dimensional feature dynamics of fluid systems*, *arXiv preprint arXiv:1808.01346* (2018) .
- [132] K. Champion, B. Lusch, N. Kutz and S. Brunton, *Autoencoders for discovering coordinates and dynamics from data*, in *APS Division of Fluid Dynamics Meeting Abstracts*, pp. P10–006, 2019.
- [133] K. Lee and K. T. Carlberg, *Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders*, *Journal of Computational Physics* **404** (2020) 108973.
- [134] B. Lusch, J. N. Kutz and S. L. Brunton, *Deep learning for universal linear embeddings of nonlinear dynamics*, *Nature communications* **9** (2018) 1.
- [135] S. E. Otto and C. W. Rowley, *Linearly recurrent autoencoder networks for learning dynamics*, *SIAM Journal on Applied Dynamical Systems* **18** (2019) 558.
- [136] S. L. Brunton, B. R. Noack and P. Koumoutsakos, *Machine learning for fluid mechanics*, *Annual Review of Fluid Mechanics* **52** (2020) 477.
- [137] Y. Bengio, P. Simard and P. Frasconi, *Learning long-term dependencies with gradient descent is difficult*, *IEEE transactions on neural networks* **5** (1994) 157.
- [138] S. Hochreiter, Y. Bengio, P. Frasconi, J. Schmidhuber et al., *Gradient flow in recurrent nets: the difficulty of learning long-term dependencies*, 2001.
- [139] M. A. Zaytar and C. El Amrani, *Sequence to sequence weather forecasting with long short-term memory recurrent neural networks*, *International Journal of Computer Applications* **143** (2016) 7.
- [140] L. Zhu and N. Laptev, *Deep and confident prediction for time series at uber*, in *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*, pp. 103–110, IEEE, 2017.

- [141] S. H. Park, B. Kim, C. M. Kang, C. C. Chung and J. W. Choi, *Sequence-to-sequence prediction of vehicle trajectory via lstm encoder-decoder architecture*, in *2018 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1672–1678, IEEE, 2018.
- [142] S. Du, T. Li and S.-J. Horng, *Time series forecasting using sequence-to-sequence deep learning framework*, in *2018 9th International Symposium on Parallel Architectures, Algorithms and Programming (PAAP)*, pp. 171–176, IEEE, 2018.
- [143] S. Du, T. Li, Y. Yang, X. Gong and S.-J. Horng, *An lstm based encoder-decoder model for multistep traffic flow prediction*, in *2019 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, IEEE, 2019.
- [144] Z. Xiang, J. Yan and I. Demir, *A rainfall-runoff model with lstm-based sequence-to-sequence learning*, *Water resources research* **56** (2020) e2019WR025326.
- [145] P. Lyu, N. Chen, S. Mao and M. Li, *Lstm based encoder-decoder for short-term predictions of gas concentration using multi-sensor fusion*, *Process Safety and Environmental Protection* **137** (2020) 93.
- [146] I.-F. Kao, Y. Zhou, L.-C. Chang and F.-J. Chang, *Exploring a long short-term memory based encoder-decoder framework for multi-step-ahead flood forecasting*, *Journal of Hydrology* **583** (2020) 124631.
- [147] T. Lee, J.-Y. Shin, J.-S. Kim and V. P. Singh, *Stochastic simulation on reproducing long-term memory of hydroclimatological variables using deep learning model*, *Journal of Hydrology* **582** (2020) 124540.
- [148] R. Cestnik and M. Abel, *Inferring the dynamics of oscillatory systems using recurrent neural networks*, *Chaos: An Interdisciplinary Journal of Nonlinear Science* **29** (2019) 063128.
- [149] V. Shopov and V. Markova, *Identification of non-linear dynamic system*, in *2019 International Conference on Information Technologies (InfoTech)*, pp. 1–3, IEEE, 2019.
- [150] J. Gonzalez and W. Yu, *Non-linear system modeling using lstm neural networks*, *IFAC-PapersOnLine* **51** (2018) 485.
- [151] X. Glorot, A. Bordes and Y. Bengio, *Deep sparse rectifier neural networks*, in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 315–323, JMLR Workshop and Conference Proceedings, 2011.
- [152] X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 249–256, JMLR Workshop and Conference Proceedings, 2010.

- [153] D. C. Howell, *Median absolute deviation, Encyclopedia of statistics in behavioral science* (2005) .
- [154] S. Anastassiou, T. Bountis and Y. Petalas, *On the topology of the lü attractor and related systems, Journal of Physics A: Mathematical and Theoretical* **41** (2008) 485101.
- [155] G. Mindlin and H. Solari, *Topologically inequivalent embeddings, Physical Review E* **52** (1995) 1497.
- [156] J.-P. Eckmann, S. O. Kamphorst, D. Ruelle and S. Ciliberto, *Liapunov exponents from time series, Physical Review A* **34** (1986) 4971.
- [157] C. Schölzel, *Nonlinear measures for dynamical systems*, 2019.
- [158] W. Gilpin, *Deep reconstruction of strange attractors from time series, arXiv preprint arXiv:2002.05909* (2020) .
- [159] J. L. Kaplan and J. A. Yorke, *Chaotic behavior of multidimensional difference equations*, in *Functional differential equations and approximation of fixed points*, pp. 204–227, Springer, (1979).
- [160] P. Frederickson, J. L. Kaplan, E. D. Yorke and J. A. Yorke, *The liapunov dimension of strange attractors, Journal of differential equations* **49** (1983) 185.
- [161] V. S. Anishchenko and S. Astakhov, *Relative kolmogorov entropy of a chaotic system in the presence of noise, International Journal of Bifurcation and Chaos* **18** (2008) 2851.
- [162] P. R. Vlachas, W. Byeon, Z. Y. Wan, T. P. Sapsis and P. Koumoutsakos, *Data-driven forecasting of high-dimensional chaotic systems with long short-term memory networks, Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* **474** (2018) 20170844.
- [163] T. Li, T. Wu and Z. Liu, *Nonlinear unsteady bridge aerodynamics: Reduced-order modeling based on deep lstm networks, Journal of Wind Engineering and Industrial Aerodynamics* **198** (2020) 104116.
- [164] Z. Wang, D. Xiao, F. Fang, R. Govindan, C. C. Pain and Y. Guo, *Model identification of reduced order fluid dynamics systems using deep learning, International Journal for Numerical Methods in Fluids* **86** (2018) 255.
- [165] S. Wiewel, M. Becher and N. Thuerey, *Latent space physics: Towards learning the temporal evolution of fluid flow*, in *Computer graphics forum*, vol. 38, pp. 71–82, Wiley Online Library, 2019.

- [166] K. Hasegawa, K. Fukami, T. Murata and K. Fukagata, *Cnn-lstm based reduced order modeling of two-dimensional unsteady flows around a circular cylinder at different reynolds numbers.*, *Fluid Dynamics Research* **52** (2020) 065501.